

MyBatis 核心

在WEB中应用MyBatis（使用MVC架构模式）

目标：

- 掌握mybatis在web应用中怎么用
- mybatis三大对象的作用域和生命周期
- ThreadLocal原理及使用
- 巩固MVC架构模式
- 为学习MyBatis的接口代理机制做准备

实现功能：

- 银行账户转账

使用技术：

- HTML + Servlet + MyBatis

WEB应用的名称：

- bank

软件版本：

- IntelliJ IDEA：2023.2 (Ultimate Edition)
- Navicat：17.3.8
- MySQL数据库：8.4.8
- MySQL密码：20060329
- MyBatis：3.5.19
- MySQL驱动：8.4.0
- JDK：Java21
- Maven 3.9.12
- JUnit：4.13.2
- Logback：1.5.6
- apache-tomcat-11.0.18

需求描述

银行账户转账

文件 | C:/Users/Administrator/Desktop/index.html

转出账户：

转入账户：

转账金额：

转账

数据库表的设计和准备数据

数据库: powernode

数据表: t_act

- SQL语句:

```
-- 1. 创建数据库 powernode (如果不存在的话) 并切换到该数据库
CREATE DATABASE IF NOT EXISTS `powernode` DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE `powernode`;

-- 2. 如果表已经存在, 先删除
DROP TABLE IF EXISTS `t_act`;

-- 3. 创建 t_act 表
CREATE TABLE `t_act` (
  `id` bigint NOT NULL AUTO_INCREMENT COMMENT '单纯的主键, 自增',
  `actno` varchar(255) DEFAULT NULL COMMENT '账号',
  `balance` decimal(15,2) DEFAULT NULL COMMENT '余额',
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='账户表';

-- 4. 插入初始数据
INSERT INTO `t_act` (`actno`, `balance`) VALUES ('act001', 50000.00);
INSERT INTO `t_act` (`actno`, `balance`) VALUES ('act002', 0.00);
```

我的连接

- 本地MySQL
 - information_schema
 - mybatis_demo
 - mysql
 - performance_schema
 - powernode
 - 表
 - t_act
 - 字段
 - id
 - actno
 - balance

对象: t_act @powernode (本地MySQL) - 表

保存 添加字段 插入字段 删除字段 主键 上移 下移

字段	索引	外键	检查	触发器	选项	注释	SQL 预览
名称	类型	长度	小数点	不是 null	虚拟	键	注释
id	bigint			☒	☐	1	单纯的主键, 自增
actno	varchar	255		☐	☐		账号
balance	decimal	15	2	☐	☐		余额

我的连接

- 本地MySQL
 - information_schema
 - mybatis_demo
 - mysql
 - performance_schema
 - powernode
 - 表
 - t_act
 - t_user
 - 视图

对象: t_act @powernode (本地MySQL) - 表

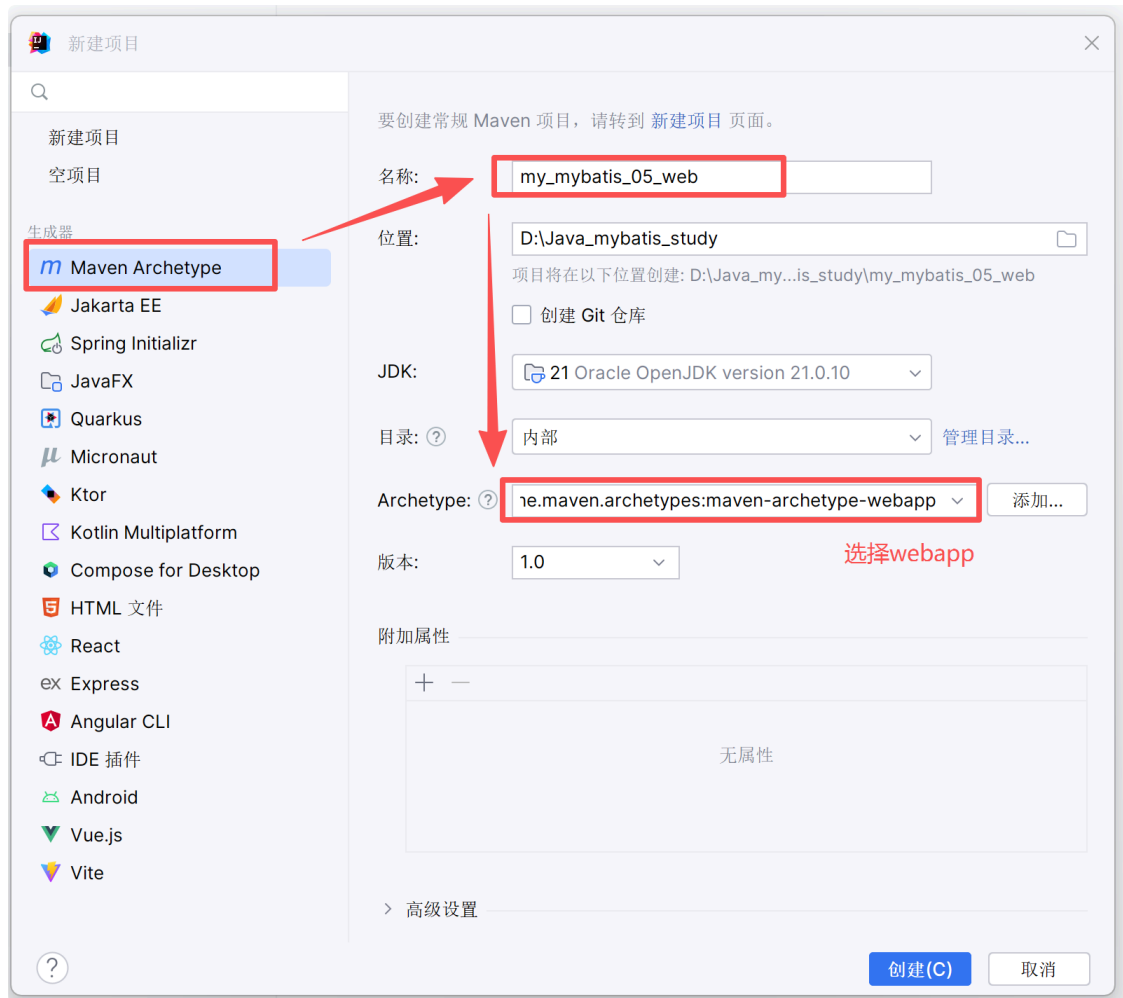
表配置文件 开始事务 单元格编辑器 筛选

id	actno	balance
# bigint	# varchar(255)	# decimal(15,2)
1	act001	50000.00
2	act002	0.00

实现步骤

第一步：环境搭建

- IDEA中创建Maven WEB应用（my_mybatis-05-web）



- Archetype：org.apache.maven.archetypes:maven-archetype-webapp
- IDEA配置Tomcat，这里Tomcat使用11+版本。并部署应用到tomcat。
 - 下载Tomcat 11.0.18
 - 网站：<https://tomcat.apache.org/download-11.cgi>

Search... GO

COMMUNITY CODE

Apache Tomcat

- Home
- Taglibs
- Maven Plugin
- Download**
- Which version?
- Tomcat 11
- Tomcat 10
- Tomcat 9
- Tomcat Migration Tool for Jakarta EE
- Tomcat Connectors
- Tomcat Native
- Taglibs
- Archives

Documentation

- Tomcat 11.0
- Tomcat 10.1
- Tomcat 9.0
- Upgrading
- Tomcat Connectors
- Tomcat Native 2
- Tomcat Native 1.3
- Wiki
- Migration Guide
- Presentations
- Specifications

Tomcat 11 Software Downloads

Welcome to the Apache Tomcat® 11.x software download page. This page provides download links for obtaining the latest version of Tomcat 11.0.x software, as well as links to the archives of older releases.

Unsure which version you need? Specification versions implemented, minimum Java version required and lots more useful information may be found on the ["which version?"](#) page.

Users of Tomcat 11 onwards should be aware that, as a result of the move from Java EE to Jakarta EE as part of the transfer of Java EE to the Eclipse Foundation, the primary package for all implemented APIs has changed from `javax.*` to `jakarta.*`. This will almost certainly require code changes to enable applications to migrate from Tomcat 9 and earlier to Tomcat 11 and later. A [migration tool](#) has been developed to aid this process.

Quick Navigation

[KEYS](#) | [11.0.18](#) | [Browse](#) | [Archives](#)

Release Integrity

You **must** [verify](#) the integrity of the downloaded files. We provide OpenPGP signatures for every release file. This signature should be matched against the [KEYS](#) file which contains the OpenPGP keys of Tomcat's Release Managers. We also provide [SHA-512](#) checksums for every release file. After you download the file, you should calculate a checksum for your download, and make sure it is the same as ours.

Mirrors

You are currently using <https://dldcn.apache.org/>. If you encounter a problem with this mirror, please select another mirror. If all mirrors are failing, there are *backup* mirrors (at the end of the mirrors list) that should be available.

Other mirrors:

11.0.18

Please see the [README](#) file for packaging information. It explains what every distribution contains.

Binary Distributions

- Core:
 - [zip \(pgp, sha512\)](#)
 - [tar.gz \(pgp, sha512\)](#)
 - [Windows zip \(pgp, sha512\)](#)

下载zip

在 Maven 项目中，如果没有显式指定打包方式，Maven 默认会把它当成一个普通的 Java SE 项目，即jar包。因此我们要手动添加war包的代码。不添加war包，IDEA 无法识别项目为 Web 项目，不会提供 Web 相关的部署选项。

- 在pom.xml 文件中：

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>lzt.lztbatis</groupId>
  <artifactId>my_mybatis_05_web</artifactId>
  <version>1.0-SNAPSHOT</version>
  <!-- 添加war包 -->
  <packaging>war</packaging>

  <properties>
    <maven.compiler.source>21</maven.compiler.source>
    <maven.compiler.target>21</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

</project>
```

- 添加 Web 类型的 Artifact
 - 点击文件 -> 项目结构
 - 选择 Web 应用程序: 展开型

项目结构

← →

+

−

📄

项目设置

项目

模块

库

Facet

工件

平台设置

SDK

全局库

问题

添加

JAR >

运行时映像(JLink)

Web 应用程序: 展开型 >

Web 应用程序: 归档

Java EE 应用程序: 展开型

Java EE 应用程序: 归档

EJB 应用程序: 展开型

EJB 应用程序: 归档

JavaFx 应用程序 >

平台特定的软件包 >

JavaFx 预加载程序

其他

空

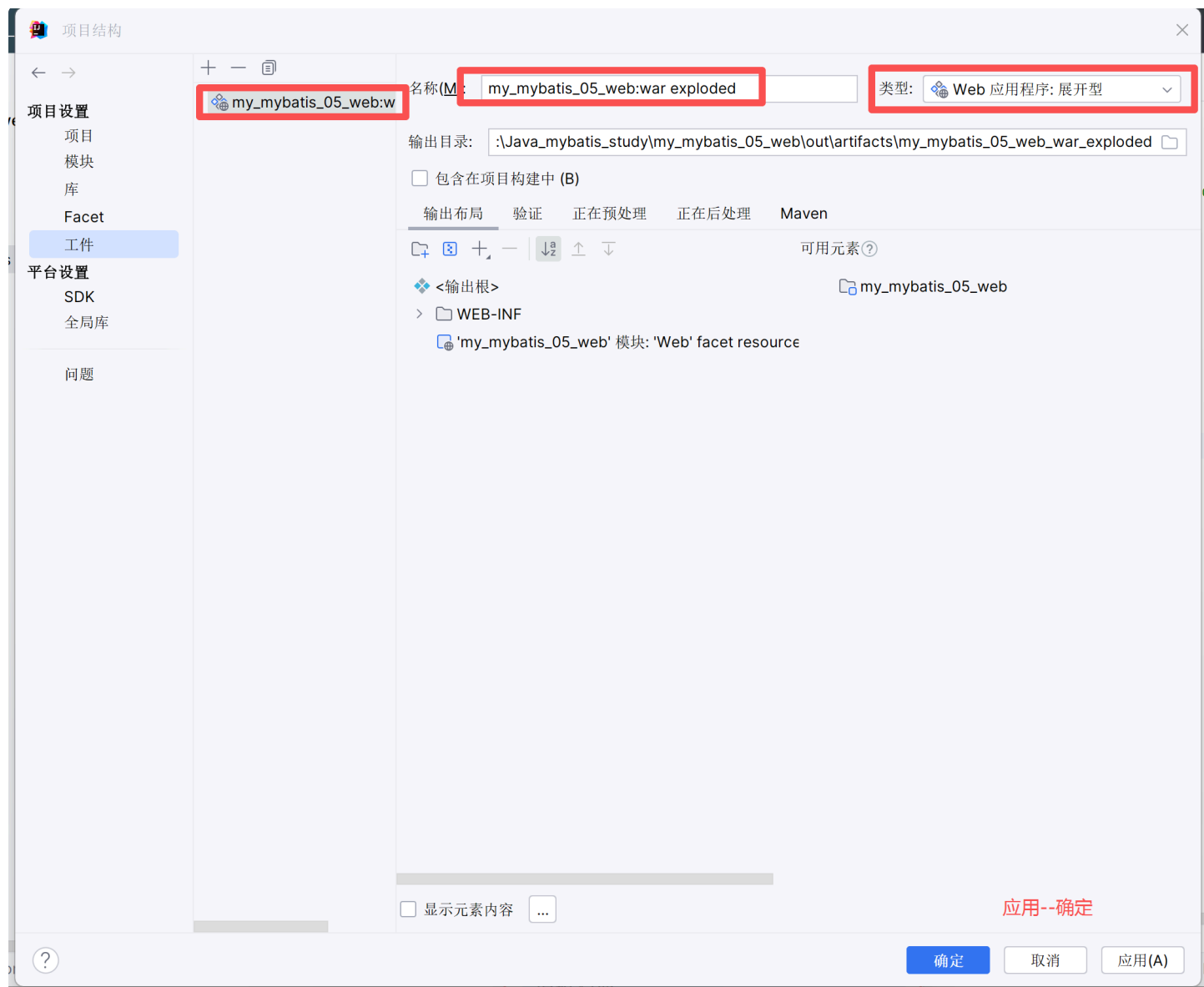
基于模块...

?

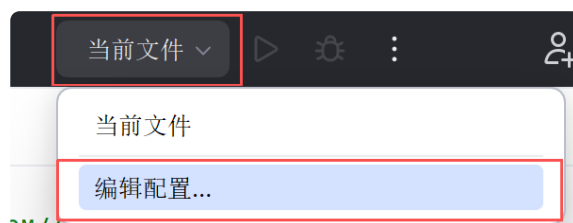
确定

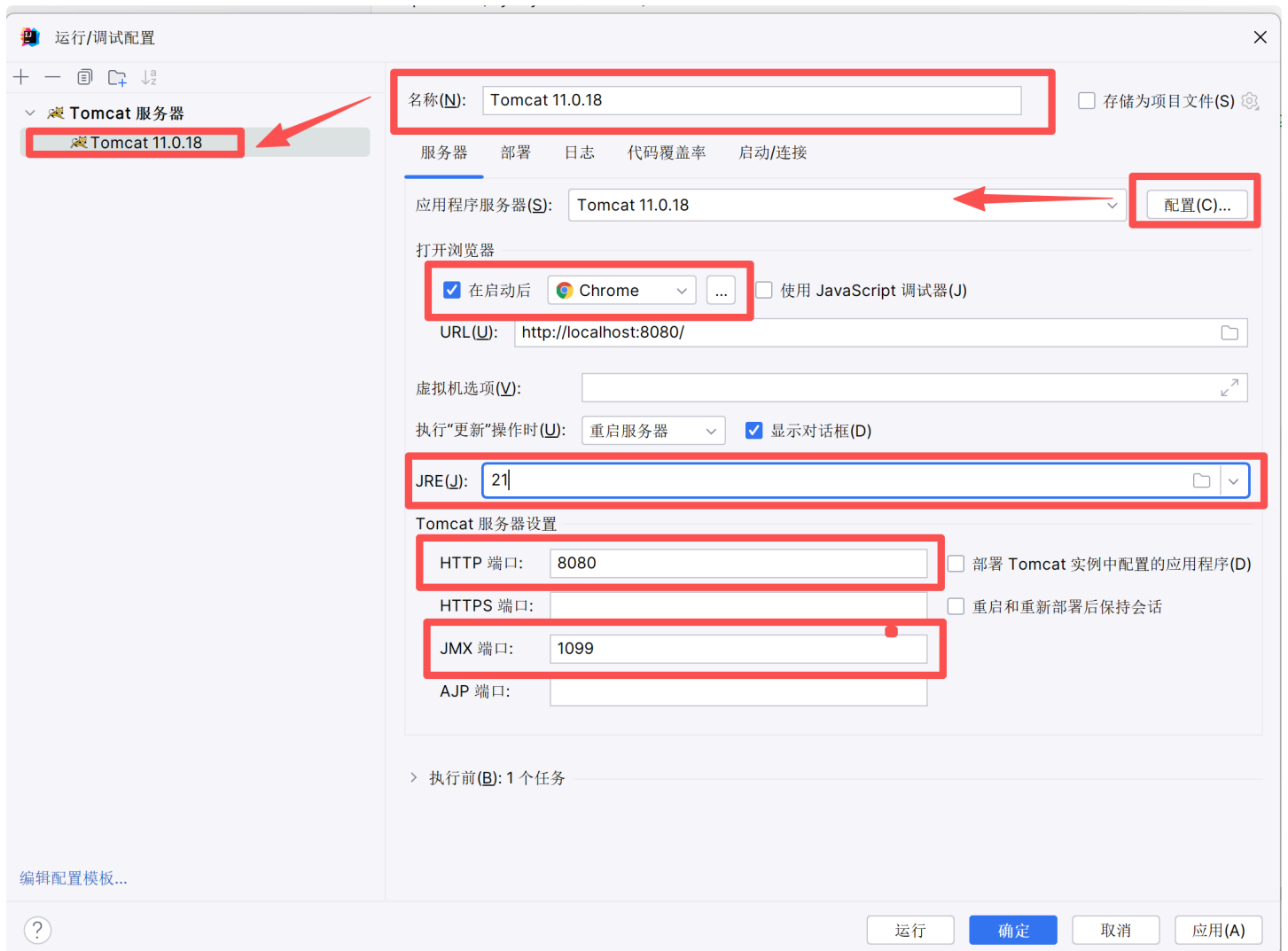
取消

应用(A)

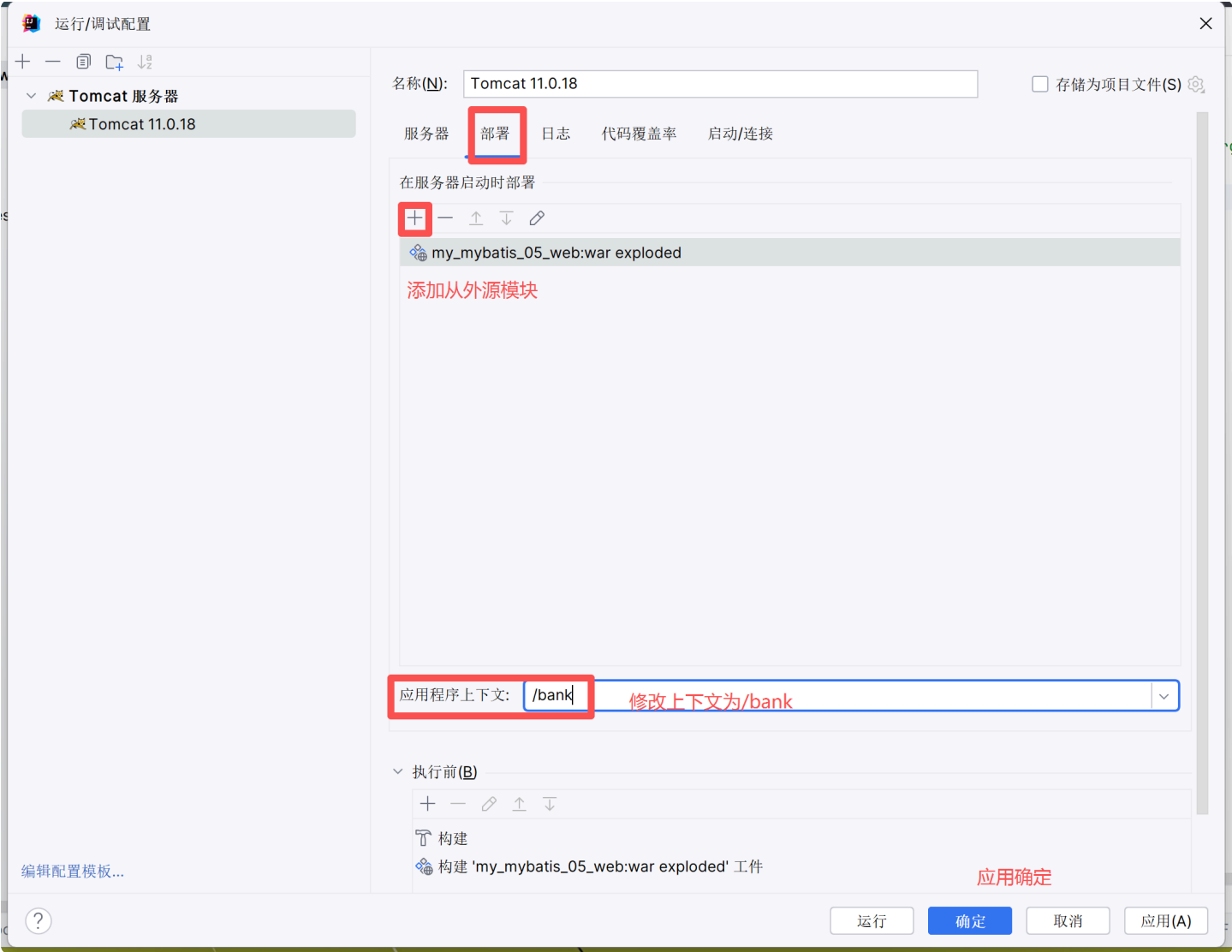


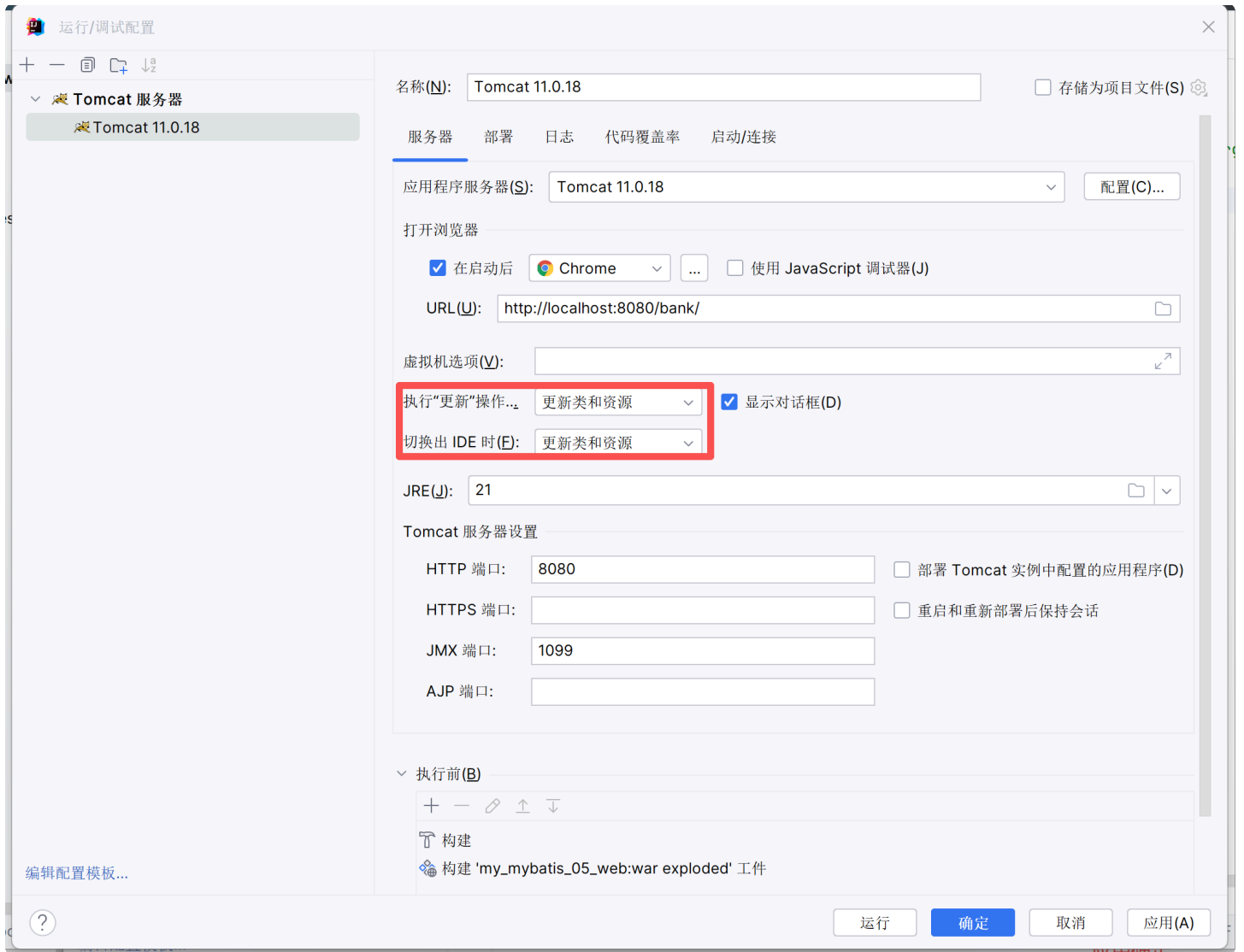
- 配置本地 Tomcat 11.0.18 服务器
 - 点击 IDEA 顶部菜单栏靠右侧的 Current File / Edit Configurations...
 - 点击左上角的 + 号，向下滚动找到并选择 Tomcat Server -> Local。





- 配置到这里先不要点确定关闭窗口，服务器配好了，还要把我们的项目“放”进去，也就是部署。





- 默认创建的maven web应用没有java和resources目录，包括两种解决方案
 - 第一种：自己手动加上。



- 第二种：修改maven-archetype-webapp-1.4.jar中的配置文件

名称	修改日期	类型
_remote.repositories	2026/3/9 17:04	REPOSITORY
maven-archetype-webapp-1.0.jar	2026/3/9 17:04	Executable Ja
maven-archetype-webapp-1.0.jar.sha1	2026/3/9 17:04	SHA1 文件
maven-archetype-webapp-1.0.pom	2026/3/9 17:04	POM 文件
maven-archetype-webapp-1.0.pom.sha1	2026/3/9 17:04	SHA1 文件

路径：C:\Users\jack.m2\repository\org\apache\maven\archetypes\maven-archetype-webapp\1.0

- 在这个目录下，会看到一个文件叫 maven-archetype-webapp-1.0.jar
 - 把它复制到一个空文件夹里，我们在这个副本上操作，以防万一改坏了。
 - 把这个 maven-archetype-webapp-1.0.jar 的后缀名临时改成 .zip，然后解压它。
 - 进入解压后的目录，依次打开：archetype-resources -> src -> main。
 - 会发现这里面只有一个 webapp 文件夹
 - 在这个 main 目录下，手动新建两个空文件夹，分别命名为 java 和 resources。
 - 退回到解压的根目录，依次打开：META-INF -> maven -> 找到 archetype-metadata.xml 文件。



老杜

- 用文本编辑器打开它。
- 找到 `<fileSets>` 标签。在这个标签内部，添加 java 和 resources 的配置，修改后看起来像这样：

```
<fileSets>
  <fileSet filtered="true" packaged="true">
    <directory>src/main/java</directory>
  </fileSet>
  <fileSet filtered="true" packaged="false">
    <directory>src/main/resources</directory>
  </fileSet>

  <fileSet filtered="false" packaged="false">
    <directory>src/main/webapp</directory>
  </fileSet>
</fileSets>
```

- 说明：packaged="true" 表示以后在 java 目录下建类时会自动按照你在建项目时填写的 groupId 生成包结构。
- 保存 archetype-metadata.xml 文件。
- 回到解压出的根目录

是全选里面的所有文件和文件夹，而不是选中外面那个大文件夹

- 右键将这些文件重新压缩成 .zip 格式
- 将新生成的压缩包重命名为 maven-archetype-webapp-1.4.jar

- 把这个修改后的 jar 包，复制回并覆盖掉第一步中 Maven 本地仓库里的那个原版 jar 包。
- web.xml文件的版本较低，可以从tomcat11的样例文件中复制，然后修改
 - 从 Tomcat 10 开始，底层的 Java EE 规范已经全面迁移到了 Jakarta EE。
 - 必须使用全新的 jakartaee 约束。

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
    https://jakarta.ee/xml/ns/jakartaee/web-app_6_0.xsd"
  version="6.0">

  <welcome-file-list>
    <welcome-file>index.html</welcome-file>
  </welcome-file-list>

</web-app>
```

<welcome-file>：写一个 index.html 作为转账页面，配置这个标签后，当用户直接访问 <http://localhost:8080/bank/> 时，Tomcat 就会自动去寻找并展示 index.html，而不需要在地址栏里敲全路径。

如果有index.jsp文件，便删除index.jsp文件，因为我们这个项目不使用JSP。只使用html。

确定pom.xml文件中的打包方式是war包。

- 引入相关依赖
 - 编译器版本修改为：jdk21
 - 引入的依赖包括：mybatis, mysql驱动, junit, logback, servlet。
 - 编写pom.xml文件

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>lzt.lztbatis</groupId>
  <artifactId>my_mybatis_05_web</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>war</packaging>

  <name>my_mybatis_05_web</name>
  <url>http://maven.apache.org</url>

  <properties>
    <maven.compiler.source>21</maven.compiler.source>
    <maven.compiler.target>21</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.mybatis</groupId>
      <artifactId>mybatis</artifactId>
      <version>3.5.19</version>
    </dependency>

    <dependency>
      <groupId>com.mysql</groupId>
```

```

        <artifactId>mysql-connector-j</artifactId>
        <version>8.4.0</version>
    </dependency>

    <dependency>
        <groupId>junit</groupId>
        <artifactId>junit</artifactId>
        <version>4.13.2</version>
        <scope>test</scope>
    </dependency>

    <dependency>
        <groupId>ch.qos.logback</groupId>
        <artifactId>logback-classic</artifactId>
        <version>1.5.6</version>
    </dependency>

    <dependency>
        <groupId>jakarta.servlet</groupId>
        <artifactId>jakarta.servlet-api</artifactId>
        <version>6.0.0</version>
        <scope>provided</scope>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <version>3.11.0</version>
            <configuration>
                <source>21</source>
                <target>21</target>
                <encoding>UTF-8</encoding>
            </configuration>
        </plugin>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-war-plugin</artifactId>
            <version>3.4.0</version>
        </plugin>
    </plugins>
</build>

</project>

```

- 引入相关配置文件，放到resources目录下（全部放到类的根路径下）
 - mybatis-config.xml
 - MyBatis 的核心配置文件

```

● ● ●
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE configuration
    PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-config.dtd">
<configuration>

    <properties resource="jdbc.properties"/>

    <environments default="development">
        <environment id="development">
            <transactionManager type="JDBC"/>

```



```

        <dataSource type="POOLED">
            <property name="driver" value="${jdbc.driver}"/>
            <property name="url" value="${jdbc.url}"/>
            <property name="username" value="${jdbc.username}"/>
            <property name="password" value="${jdbc.password}"/>
        </dataSource>
    </environment>
</environments>

<mappers>
    <mapper resource="AccountMapper.xml"/>
</mappers>

</configuration>

```

- o AccountMapper.xml
 - SQL 映射文件

```

● ● ●
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper namespace="Lzt.Lztbatis.bank.dao.AccountDao">
<!-- 填写相关SQL 语句操作-->
</mapper>

```

- o logback.xml
 - 日志配置文件

```

● ● ●
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
    <appender name="STDOUT" class="ch.qos.logback.core.ConsoleAppender">
        <encoder>
            <pattern>[%thread] %-5level %logger{36} - %msg%n</pattern>
        </encoder>
    </appender>

    <logger name="Lzt.Lztbatis.bank.dao" level="DEBUG"/>

    <root level="INFO">
        <appender-ref ref="STDOUT" />
    </root>
</configuration>

```

- o jdbc.properties
 - 数据库连接信息配置

```

● ● ●
jdbc.driver=com.mysql.cj.jdbc.Driver
jdbc.url=jdbc:mysql://localhost:3306/powernode?useUnicode=true&characterEncoding=utf8&serverTimezone=UTC
jdbc.username=root
jdbc.password=20060329

```

第二步：前端页面index.html

```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8">
  <title>银行账户转账系统</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f4f4f9;
      display: flex;
      justify-content: center;
      align-items: center;
      height: 100vh;
      margin: 0;
    }
    .transfer-container {
      background-color: #fff;
      padding: 30px 40px;
      border-radius: 8px;
      box-shadow: 0 4px 8px rgba(0,0,0,0.1);
      width: 350px;
    }
    h2 {
      text-align: center;
      color: #333;
      margin-bottom: 20px;
    }
    .form-group {
      margin-bottom: 15px;
    }
    .form-group label {
      display: block;
      margin-bottom: 5px;
      color: #666;
      font-size: 14px;
    }
    .form-group input {
      width: 100%;
      padding: 10px;
      border: 1px solid #ccc;
      border-radius: 4px;
      box-sizing: border-box;
    }
    .btn-submit {
      width: 100%;
      padding: 12px;
      background-color: #007bff;
      color: white;
      border: none;
      border-radius: 4px;
      font-size: 16px;
      cursor: pointer;
      transition: background-color 0.3s;
    }
    .btn-submit:hover {
      background-color: #0056b3;
    }
  </style>
</head>
<body>
```

```
<div class="transfer-container">
  <h2>银行账户转账</h2>
  <form action="transfer" method="post">
    <div class="form-group">
      <label for="fromActno">转出账号: </label>
      <input type="text" id="fromActno" name="fromActno" placeholder="请输入转出账号" required>
    </div>

    <div class="form-group">
      <label for="toActno">转入账号: </label>
      <input type="text" id="toActno" name="toActno" placeholder="请输入转入账号" required>
    </div>

    <div class="form-group">
      <label for="money">转账金额: </label>
      <input type="number" id="money" name="money" placeholder="请输入转账金额" step="0.01" min="0.01" required>
    </div>

    <button type="submit" class="btn-submit">确认转账</button>
  </form>
</div>

</body>
</html>
```

银行账户转账

转出账号：

请输入转出账号

转入账号：

请输入转入账号

转账金额：

请输入转账金额

确认转账

第三步：创建基础包结构 (MVC 架构)

- 在src/main/java 目录下
 - lzt.lztbatis.bank.pojo
 - 存放普通的 Java 实体类
 - lzt.lztbatis.bank.dao
 - 数据访问层
 - lzt.lztbatis.bank.dao.impl

- o lzt.lztbatis.bank.service
 - 业务逻辑层
- o lzt.lztbatis.bank.service.impl
- o lzt.lztbatis.bank.web
- o lzt.lztbatis.bank.web.controller
 - 表现层/控制层
- o lzt.lztbatis.bank.utils
 - 存放公共的工具类
 - 将之前编写的SqlSessionUtil工具类拷贝到该包下。
- o lzt.lztbatis.bank.utils.exception

第四步：定义pojo类：Account

- Account.java

```
package lzt.lztbatis.bank.pojo;

/**
 * 银行账户类
 */
public class Account {
    private Long id;
    private String actno;
    private Double balance;

    @Override
    public String toString() {
        return "Account{" +
            "id=" + id +
            ", actno='" + actno + '\'' +
            ", balance=" + balance +
            '}';
    }

    public Account() {
    }

    public Account(Long id, String actno, Double balance) {
        this.id = id;
        this.actno = actno;
        this.balance = balance;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getActno() {
        return actno;
    }

    public void setActno(String actno) {
        this.actno = actno;
    }
}
```

```

    public Double getBalance() {
        return balance;
    }

    public void setBalance(Double balance) {
        this.balance = balance;
    }
}

```

第五步：编写AccountDao接口，以及AccountDaoImpl实现类

分析dao中至少要提供几个方法，才能完成转账：

- 转账前需要查询余额是否充足：selectByActno
- 转账时要更新账户：update
- AccountDao.java

```

package lzt.lztbatis.bank.dao;

import lzt.lztbatis.bank.pojo.Account;

/**
 * 账户数据访问对象
 */
public interface AccountDao {

    /**
     * 根据账号获取账户信息
     * @param actno 账号
     * @return 账户信息
     */
    Account selectByActno(String actno);

    /**
     * 更新账户信息
     * @param act 账户信息
     * @return 1表示更新成功，其他值表示失败
     */
    int update(Account act);
}

```

- AccountDaoImpl.java

```

package lzt.lztbatis.bank.dao.impl;

import lzt.lztbatis.bank.dao.AccountDao;
import lzt.lztbatis.bank.pojo.Account;
import lzt.lztbatis.bank.utils.SqlSessionUtil;
import org.apache.ibatis.session.SqlSession;

public class AccountDaoImpl implements AccountDao {

    @Override
    public Account selectByActno(String actno) {
        SqlSession sqlSession = SqlSessionUtil.openSession();
        return sqlSession.selectOne("account.selectByActno", actno);
    }

    @Override
    public int update(Account act) {

```

```

        sqlSession.update("account.update", act);
    }
}

```

- 编写 SqlSessionUtil 工具类

```

package lzt.lztbatis.bank.utils;

import org.apache.ibatis.io.Resources;
import org.apache.ibatis.session.SqlSession;
import org.apache.ibatis.session.SqlSessionFactory;
import org.apache.ibatis.session.SqlSessionFactoryBuilder;

import java.io.IOException;

/**
 * MyBatis 工具类
 */
public class SqlSessionUtil {

    // SqlSessionFactory 对象一个环境只需要一个即可
    private static SqlSessionFactory sqlSessionFactory;

    // 全局的，一个类中定义一个即可。
    // 使用 ThreadLocal 来保证同一个线程使用的是同一个 SqlSession
    private static ThreadLocal<SqlSession> local = new ThreadLocal<>();

    // 静态代码块，在类加载时执行，且只执行一次
    static {
        try {
            sqlSessionFactory = new SqlSessionFactoryBuilder().build(Resources.getResourceAsStream("mybatis-
config.xml"));
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    /**
     * 获取当前线程的 SqlSession 对象
     * @return SqlSession
     */
    public static SqlSession openSession() {
        SqlSession sqlSession = local.get();
        if (sqlSession == null) {
            sqlSession = sqlSessionFactory.openSession();
            // 将 sqlSession 绑定到当前线程上
            local.set(sqlSession);
        }
        return sqlSession;
    }

    /**
     * 关闭 SqlSession，并从当前线程中移除
     * @param sqlSession 要关闭的 SqlSession
     */
    public static void close(SqlSession sqlSession) {
        if (sqlSession != null) {
            sqlSession.close();
            // 注意：在 Web 环境下（如 Tomcat），线程池的线程是复用的。
            // 如果这里不移除，可能会导致内存泄漏或下一次请求拿到已经关闭的 SqlSession。
            local.remove();
        }
    }
}

```

```
}  
}
```

第六步：编写 SQL 映射文件

- AccountMapper.xml

```
<?xml version="1.0" encoding="UTF-8" ?>  
<!DOCTYPE mapper  
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"  
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">  
  
<mapper namespace="account">  
  
    <select id="selectByActno" resultType="lzt.lztbatis.bank.pojo.Account">  
        select * from t_act where actno = #{actno}  
    </select>  
  
    <update id="update">  
        update t_act set balance = #{balance} where actno = #{actno}  
    </update>  
  
</mapper>
```

第七步：编写AccountService接口以及AccountServiceImpl

- 编写自定义异常类：MoneyNotEnoughException

```
package lzt.lztbatis.bank.exception;  
  
/**  
 * 余额不足异常  
 */  
public class MoneyNotEnoughException extends Exception {  
    public MoneyNotEnoughException() {}  
    public MoneyNotEnoughException(String msg) {  
        super(msg);  
    }  
}
```

- AppException.java

```
package lzt.lztbatis.bank.exception;  
  
/**  
 * 应用异常  
 */  
public class AppException extends Exception {  
  
    public AppException() {}  
  
    public AppException(String msg) {  
        super(msg);  
    }  
  
}
```

- 编写 AccountService 接口

- 对于转账来说，核心动作就是 transfer
- AccountService.java

```
package lzt.lztbatis.bank.service;

import lzt.lztbatis.bank.exception.AppException;
import lzt.lztbatis.bank.exception.MoneyNotEnoughException;

/**
 * 账户业务接口
 */
public interface AccountService {

    /**
     * 账户转账业务
     * @param fromActno 转出账号
     * @param toActno 转入账号
     * @param money 转账金额
     * @throws MoneyNotEnoughException 余额不足时抛出该异常
     * @throws AppException 应用级别异常，如数据库更新失败时抛出
     */
    void transfer(String fromActno, String toActno, double money) throws MoneyNotEnoughException, AppException;
}
```

- 编写 AccountServiceImpl 实现类
 - AccountServiceImpl.java

```
package lzt.lztbatis.bank.service.impl;

import lzt.lztbatis.bank.dao.AccountDao;
import lzt.lztbatis.bank.dao.impl.AccountDaoImpl;
import lzt.lztbatis.bank.exception.AppException;
import lzt.lztbatis.bank.exception.MoneyNotEnoughException;
import lzt.lztbatis.bank.pojo.Account;
import lzt.lztbatis.bank.service.AccountService;
import lzt.lztbatis.bank.utils.SqlSessionUtil;
import org.apache.ibatis.session.SqlSession;

public class AccountServiceImpl implements AccountService {

    private AccountDao accountDao = new AccountDaoImpl();

    @Override
    public void transfer(String fromActno, String toActno, double money) throws MoneyNotEnoughException, AppException {

        // 提前获取一次 session，确保整个业务流程开启了事务
        SqlSession sqlSession = SqlSessionUtil.openSession();

        try {
            // 1. 查询转出账户的信息
            Account fromAct = accountDao.selectByActno(fromActno);
            if (fromAct == null) {
                throw new AppException("转出账户不存在!");
            }

            // 2. 判断余额是否充足
            if (fromAct.getBalance() < money) {
                throw new MoneyNotEnoughException("对不起，余额不足!");
            }
        }
```



```

        // 3. 余额充足, 查询转入账户的信息
        Account toAct = accountDao.selectByActno(toActno);
        if (toAct == null) {
            throw new AppException("转入账户不存在!");
        }

        // 4. 在内存中修改两个账户的余额
        fromAct.setBalance(fromAct.getBalance() - money);
        toAct.setBalance(toAct.getBalance() + money);

        // 5. 调用 DAO 层的 update 方法更新数据库
        int count = accountDao.update(fromAct);
        if (count != 1) {
            throw new AppException("转出账户更新失败!");
        }

        count = accountDao.update(toAct);
        if (count != 1) {
            throw new AppException("转入账户更新失败!");
        }

        sqlSession.commit();

    } catch (Exception e) {
        sqlSession.rollback();
        throw e;
    } finally {
        SqlSessionUtil.close(sqlSession);
    }
}
}

```

第八步：编写AccountController

```

package lzt.lztbatis.bank.web.controller;

import lzt.lztbatis.bank.exception.AppException;
import lzt.lztbatis.bank.exception.MoneyNotEnoughException;
import lzt.lztbatis.bank.service.AccountService;
import lzt.lztbatis.bank.service.impl.AccountServiceImpl;

// 注意: Tomcat 11 环境下, 必须使用 jakarta 命名空间
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.io.PrintWriter;

/**
 * 账户控制器
 */
@WebServlet("/transfer")
public class AccountController extends HttpServlet {

    private AccountService accountService = new AccountServiceImpl();

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        // 1. 设置响应流的内容类型和编码, 防止页面输出中文乱码

```

```

response.setContentType("text/html;charset=UTF-8");
PrintWriter out = response.getWriter();

// 2. 获取前端提交的账户信息
String fromActno = request.getParameter("fromActno");
String toActno = request.getParameter("toActno");

// 修正: 将 Integer.parseInt 改为 Double.parseDouble, 以支持转账金额包含小数
double money = Double.parseDouble(request.getParameter("money"));

// 3. 调用业务方法完成转账
try {
    accountService.transfer(fromActno, toActno, money);
    out.print("<h1>转账成功!!!</h1>");
} catch (MoneyNotEnoughException e) {
    out.print("<h1>转账失败: " + e.getMessage() + "</h1>");
} catch (AppException e) {
    out.print("<h1>转账失败: " + e.getMessage() + "</h1>");
} catch (Exception e) {
    // 兜底异常捕获: 如果发生其他未知异常(如数据库连接失败、空指针等), 方便页面报错排查
    out.print("<h1>系统繁忙, 转账失败: 发生未知错误.</h1>");
    e.printStackTrace();
}
}
}

```

启动服务器, 打开浏览器, 输入地址: <http://localhost:8080/bank>, 测试:

银行账户转账

转出账号:

转入账号:

转账金额:

确认转账

灯家 t_act @powernode (本地MySQL) - 表		
表配置文件	开始事务	单元格编辑器 筛选 &
id # bigint	actno # varchar(255)	balance # decimal(15,2)
1	act001	50000.00
2	act002	0.00

转账成功!!!

表配置文件	开始事务	单元格编辑器
id	actno	balance
# bigint	A ^B C varchar(255)	# decimal(15,2)
1	act001	40000.00
2	act002	10000.00

项目

my_mybatis_05_web

src

main

java

lzt.lztbatis.bank

dao

impl

AccountDaoImpl

AccountDao

exception

AppException

MoneyNotEnoughException

pojo

Account

service

impl

AccountServiceImpl

AccountService

utils

GenerateDaoByJavassist

SqlSessionUtil

web.controller

AccountController

resources

AccountMapper.xml

jdbc.properties

logback.xml

mybatis-config.xml

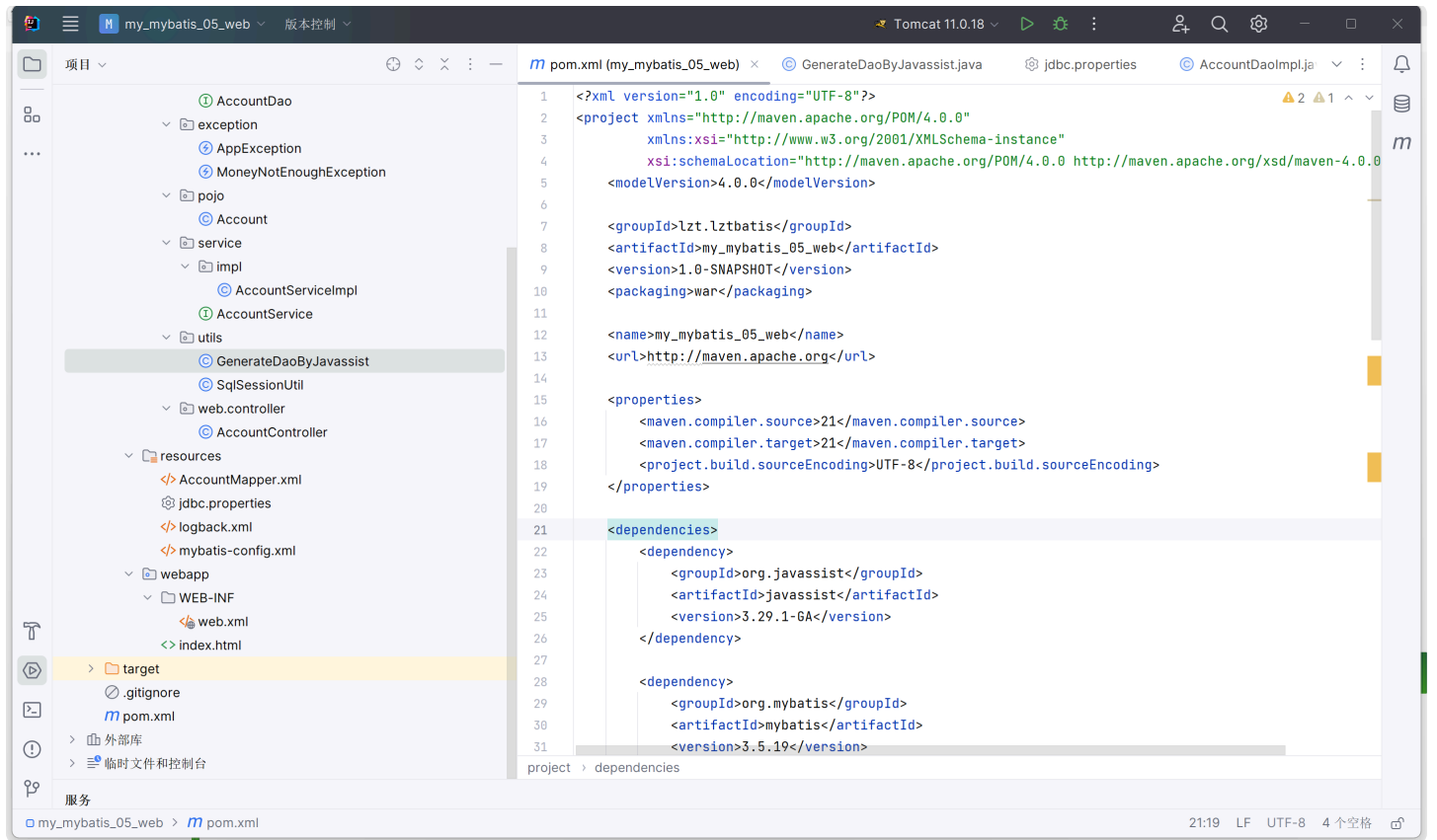
pom.xml (my_mybatis_05_web)

GenerateDaoByJavassist.java

jdbc.properties

AccountDaoImpl.java

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4       xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0
5       <modelVersion>4.0.0</modelVersion>
6
7       <groupId>lzt.lztbatis</groupId>
8       <artifactId>my_mybatis_05_web</artifactId>
9       <version>1.0-SNAPSHOT</version>
10      <packaging>war</packaging>
11
12      <name>my_mybatis_05_web</name>
13      <url>http://maven.apache.org</url>
14
15      <properties>
16        <maven.compiler.source>21</maven.compiler.source>
17        <maven.compiler.target>21</maven.compiler.target>
18        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
19      </properties>
20
21      <dependencies>
22        <dependency>
23          <groupId>org.javassist</groupId>
24          <artifactId>javassist</artifactId>
25          <version>3.29.1-6A</version>
26        </dependency>
27
28        <dependency>
29          <groupId>org.mybatis</groupId>
30          <artifactId>mybatis</artifactId>
31          <version>3.5.19</version>
32        </dependency>
33      </dependencies>
34    </project>
```



MyBatis对象作用域以及事务问题

MyBatis核心对象的作用域

SqlSessionFactoryBuilder

这个类可以被实例化、使用 and 丢弃，一旦创建了 `SqlSessionFactory`，就不再需要它了。因此 `SqlSessionFactoryBuilder` 实例的最佳作用域是方法作用域

作用域（也就是局部方法变量）。你可以重用 `SqlSessionFactoryBuilder` 来创建多个 `SqlSessionFactory` 实例，但最好还是不要一直保留着它，以保证

所有的 XML 解析资源可以被释放给更重要的事情。

SqlSessionFactory

`SqlSessionFactory` 一旦被创建就应该在应用的运行期间一直存在，没有任何理由丢弃它或重新创建另一个实例。使用 `SqlSessionFactory` 的最佳实

践是在应用运行期间不要重复创建多次，多次重建 `SqlSessionFactory` 被视为一种代码“坏习惯”。因此 `SqlSessionFactory` 的最佳作用域是应用作用

域。有很多方法可以做到，最简单的就是使用单例模式或者静态单例模式。

SqlSession

每个线程都应该有它自己的 `SqlSession` 实例。`SqlSession` 的实例不是线程安全的，因此是不能被共享的，所以它的最优的作用域是请求或方法作用

域。绝对不能将 `SqlSession` 实例的引用放在一个类的静态域，甚至一个类的实例变量也不行。也绝不能将 `SqlSession` 实例的引用放在任何类型的

托管作用域中，比如 Servlet 框架中的 HttpSession。如果你现在正在使用一种 Web 框架，考虑将 SqlSession 放在一个和 HTTP 请求相似的作用域

中。换句话说，每次收到 HTTP 请求，就可以打开一个 SqlSession，返回一个响应后，就关闭它。这个关闭操作很重要，为了确保每次都能执行关

闭操作，你应该把这个关闭操作放到 finally 块中。下面的示例就是一个确保 SqlSession 关闭的标准模式：

```
try (SqlSession session = sqlSessionFactory.openSession()) {  
    // 你的应用逻辑代码  
}
```

事务问题

在之前的转账业务中，更新了两个账户，我们需要保证它们的同时成功或同时失败，这个时候就需要使用事务机制，在transfer方法开始执行时开启事

务，直到两个更新都成功之后，再提交事务，我们尝试将transfer方法进行如下修改：

- 如果没有完善的事务控制，发生异常时会导致什么灾难
 - 没有回滚
 - 没有正确关闭连接
- AccountServiceImpl.java 改造

```
package lzt.lztbatis.bank.service.impl;  
  
import lzt.lztbatis.bank.dao.AccountDao;  
import lzt.lztbatis.bank.dao.impl.AccountDaoImpl;  
import lzt.lztbatis.bank.exception.AppException;  
import lzt.lztbatis.bank.exception.MoneyNotEnoughException;  
import lzt.lztbatis.bank.pojo.Account;  
import lzt.lztbatis.bank.service.AccountService;  
import lzt.lztbatis.bank.utils.SqlSessionUtil;  
import org.apache.ibatis.session.SqlSession;  
  
public class AccountServiceImpl implements AccountService {  
  
    private AccountDao accountDao = new AccountDaoImpl();  
  
    @Override  
    public void transfer(String fromActno, String toActno, double money) throws MoneyNotEnoughException, AppException  
    {  
  
        // 1. 开启事务：在这里调用一次 openSession()  
        // 因为 SqlSessionUtil 底层用了 ThreadLocal，所以在这个线程接下来的所有操作中，  
        // DAO 层调用 openSession() 拿到的都会是同一个 SqlSession！  
        SqlSession sqlSession = SqlSessionUtil.openSession();  
  
        try {  
            // 2. 查询转出账户的信息  
            Account fromAct = accountDao.selectByActno(fromActno);  
            if (fromAct == null) {  
                throw new AppException("转出账户不存在！");  
            }  
  
            // 3. 判断余额是否充足  
            if (fromAct.getBalance() < money) {  
                throw new MoneyNotEnoughException("对不起，余额不足！");  
            }  
        }  
    }  
}
```

```

// 4. 查询转入账户的信息
Account toAct = accountDao.selectByActno(toActno);
if (toAct == null) {
    throw new AppException("转入账户不存在!");
}

// 5. 在内存中修改两个账户的余额
fromAct.setBalance(fromAct.getBalance() - money);
toAct.setBalance(toAct.getBalance() + money);

// 6. 调用 DAO 层更新数据库 (此时执行的是扣钱操作)
int count = accountDao.update(fromAct);
if (count != 1) {
    throw new AppException("转出账户更新失败!");
}

// =====
// 【模拟异常测试代码】
// 你可以把下面这两行代码的注释解开, 测试一下发生异常时, 钱会不会被扣掉
String s = null;
s.toString();
// =====

// 7. 调用 DAO 层更新数据库 (此时执行的是加钱操作)
count = accountDao.update(toAct);
if (count != 1) {
    throw new AppException("转入账户更新失败!");
}

// 8. 事务提交
// 只有当所有代码都平稳执行到这里, 没有任何异常时, 才统一提交!
sqlSession.commit();

} catch (Exception e) {
    // 9. 事务回滚
    // 一旦 try 块中发生任何异常 (比如空指针、余额不足等), 立刻进行回滚!
    // 保证 "同时失败", 转出的钱会原封不动退回去
    sqlSession.rollback();

    // 将异常继续向上抛出, 交给 Controller 去处理, 以便在网页上显示错误信息
    if (e instanceof AppException) throw (AppException) e;
    if (e instanceof MoneyNotEnoughException) throw (MoneyNotEnoughException) e;
    throw new AppException("系统繁忙, 转账失败: 未知错误!");
} finally {
    // 10. 释放资源
    // finally 块保证无论成功还是失败, 连接一定会被关闭, 且 ThreadLocal 一定会被清空!
    SqlSessionUtil.close(sqlSession);
}
}
}

```

运行前注意看数据库表中当前的数据:

表配置文件	开始事务	单元格编辑器
id # bigint	actno varchar(255)	balance # decimal(15,2)
1	act001	40000.00
2	act002	10000.00

执行程序：

转账失败：系统繁忙，转账失败：未知错误！

表配置文件	开始事务	单元格编辑器
id # bigint	actno varchar(255)	balance # decimal(15,2)
1	act001	40000.00
2	act002	10000.00

- 事务回滚过程
 - 转账开始：输入 act001 转账给 act002 10000元。
- 扣钱成功：程序非常顺利地走到了第 6 步 `accountDao.update(fromAct)`，此时数据库连接里记录着：act001 扣掉了 10000。
- 突发车祸：程序碰到了未开注释的

```
String s = null;
s.toString();
```

- 当场引发了空指针异常（相当于模拟了服务器突然断电或者死机）。
- 因此，打开数据表发现没有扣钱，act001一分钱不少。
 - 将这两行代码注释掉，发现又可以转账了。

分析当前程序存在的问题

```
package com.powernode.bank.dao.impl;

import com.powernode.bank.dao.AccountDao;
import com.powernode.bank.pojo.Account;
import com.powernode.bank.utils.SqlSessionUtil;
import org.apache.ibatis.session.SqlSession;

public class AccountDaoImpl implements AccountDao {
    @Override
    public Account selectByActno(String actno) {
        SqlSession sqlSession = SqlSessionUtil.openSession();
```

```

        Account act = (Account)sqlSession.selectOne("account.selectByActno", actno);
        return act;
    }

    @Override
    public int update(Account act) {
        SqlSession sqlSession = SqlSessionUtil.openSession();
        int count = sqlSession.update("account.update", act);
        return count;
    }
}

```

我们不难发现，这个dao实现类中的方法代码很固定，基本上就是一行代码，通过SqlSession对象调用insert、delete、update、select等方法，这个类中的方法没有任何业务逻辑，既然是这样，这个类我们能不能动态的生成，以后可以不写这个类吗？答案：可以。

使用javassist生成类

Javassist是一个开源的分析、编辑和创建Java字节码的类库。是由东京工业大学的数学和计算机科学系的 Shigeru Chiba（千叶 滋）所创建的。它已加入了开放源代码JBoss 应用服务器项目，通过使用Javassist对字节码操作为JBoss实现动态"AOP"框架。

Javassist的使用

我们要使用javassist，首先要引入它的依赖

```

<dependency>
  <groupId>org.javassist</groupId>
  <artifactId>javassist</artifactId>
  <version>3.29.1-GA</version>
</dependency>

```

样例代码：

```

package com.powernode.javassist;

import javassist.ClassPool;
import javassist.CtClass;
import javassist.CtMethod;
import javassist.Modifier;

import java.lang.reflect.Method;

public class JavassistTest {
    public static void main(String[] args) throws Exception {
        // 获取类池
        ClassPool pool = ClassPool.getDefault();
        // 创建类
        CtClass ctClass = pool.makeClass("com.powernode.javassist.Test");
        // 创建方法
        // 1.返回值类型 2.方法名 3.形式参数列表 4.所属类
        CtMethod ctMethod = new CtMethod(CtClass voidType, "execute", new CtClass[] {}, ctClass);
        // 设置方法的修饰符列表
        ctMethod.setModifiers(Modifier.PUBLIC);
        // 设置方法体
        ctMethod.setBody("{System.out.println(\"hello world\");}");
        // 给类添加方法
        ctClass.addMethod(ctMethod);
        // 调用方法
        Class<?> aClass = ctClass.toClass();
    }
}

```



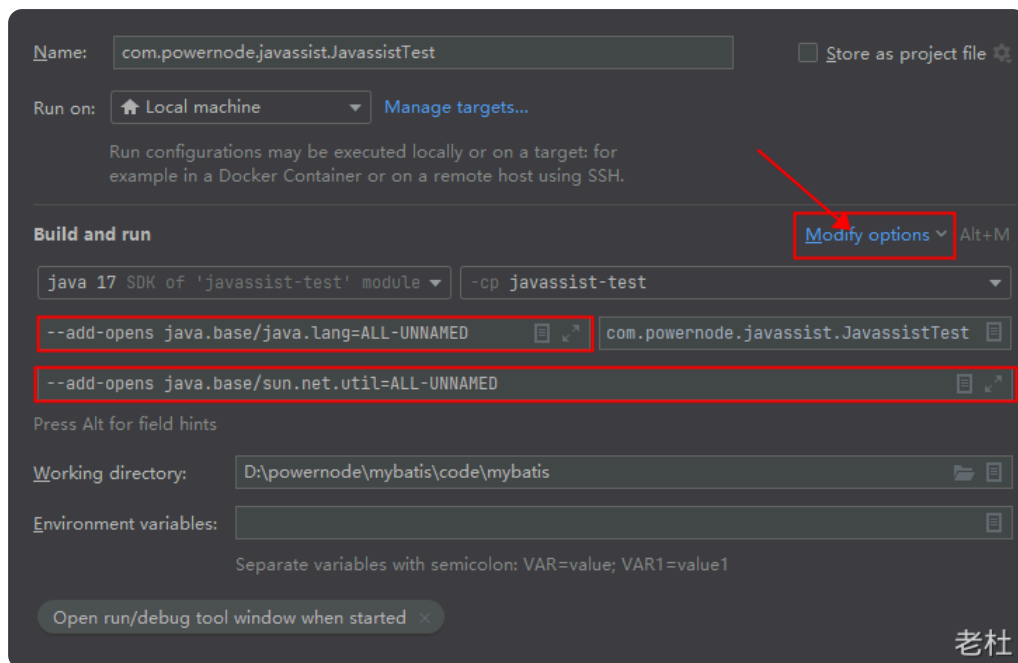
```

    Object o = aClass.newInstance();
    Method method = aClass.getDeclaredMethod("execute");
    method.invoke(o);
}
}

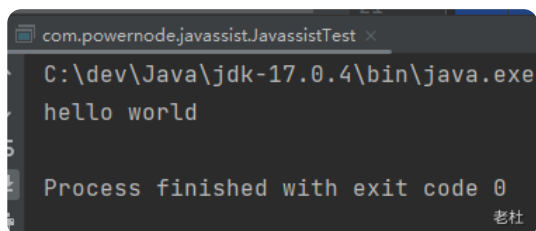
```

运行要注意：加两个参数，要不然会有异常。

- --add-opens java.base/java.lang=ALL-UNNAMED
- --add-opens java.base/sun.net.util=ALL-UNNAMED



运行结果：



使用Javassist生成DaoImpl类

使用Javassist动态生成DaoImpl类

- 在 pom.xml 中引入 Javassist 依赖

```

<dependency>
  <groupId>org.javassist</groupId>
  <artifactId>javassist</artifactId>
  <version>3.29.1-GA</version>
</dependency>

```

- 创建 GenerateDaoByJavassist 工具类
 - 在 lzt.lztbatis.bank.utils 包下新建这个类。

```

package lzt.lztbatis.bank.utils;

```

```

import javassist.CannotCompileException;
import javassist.ClassPool;
import javassist.CtClass;
import javassist.CtMethod;
import org.apache.ibatis.session.SqlSession;

import java.lang.reflect.Constructor;
import java.lang.reflect.Method;
import java.lang.reflect.Modifier;
import java.util.Arrays;

/**
 * 使用javassist库动态生成dao接口的实现类
 */
public class GenerateDaoByJavassist {

    /**
     * 根据dao接口生成dao接口的代理对象
     *
     * @param sqlSession    sql会话
     * @param daoInterface dao接口
     * @return dao接口代理对象
     */
    public static Object getMapper(SqlSession sqlSession, Class daoInterface) {
        ClassPool pool = ClassPool.getDefault();
        // 生成代理类 (例如: Lzt.Lztbatis.bank.dao.impl.AccountDaoImpl)
        CtClass ctClass = pool.makeClass(daoInterface.getPackageName() + ".impl." + daoInterface.getSimpleName() +
"Impl");
        // 接口
        CtClass ctInterface = pool.makeClass(daoInterface.getName());
        // 代理类实现接口
        ctClass.addInterface(ctInterface);
        // 获取所有的方法
        Method[] methods = daoInterface.getDeclaredMethods();
        Arrays.stream(methods).forEach(method -> {
            // 拼接方法的签名
            StringBuilder methodStr = new StringBuilder();
            String returnType = method.getReturnType().getName();
            methodStr.append(returnType);
            methodStr.append(" ");
            String methodName = method.getName();
            methodStr.append(methodName);
            methodStr.append("(");
            Class<?>[] parameterTypes = method.getParameterTypes();
            for (int i = 0; i < parameterTypes.length; i++) {
                methodStr.append(parameterTypes[i].getName());
                methodStr.append(" arg");
                methodStr.append(i);
                if (i != parameterTypes.length - 1) {
                    methodStr.append(", ");
                }
            }
            methodStr.append("){");

            // 获取sqlId (namespace必须是接口的全限定接口名, sqlId必须是接口中方法的方法名。)
            String sqlId = daoInterface.getName() + "." + methodName;
            // 获取SqlCommandType
            String sqlCommandTypeName =
sqlSession.getConfiguration().getMappedStatement(sqlId).getSqlCommandType().name();

            // 注意这里: 替换为你自己项目中的 sqlSessionUtil 全限定名
            if ("SELECT".equals(sqlCommandTypeName)) {
                methodStr.append("org.apache.ibatis.session.SqlSession sqlSession =
Lzt.Lztbatis.bank.utils.SqlSessionUtil.openSession();");
            }
        });
    }
}

```

```

        methodStr.append("Object obj = sqlSession.selectOne(\"\" + sqlId + "\", arg0);");
        methodStr.append("return (\" + returnType.getName() + ")obj;");
    } else if ("UPDATE".equals(sqlCommondTypeName)) {
        methodStr.append("org.apache.ibatis.session.SqlSession sqlSession =
lzt.lztbatis.bank.utils.SqlSessionUtil.openSession();");
        methodStr.append("int count = sqlSession.update(\"\" + sqlId + "\", arg0);");
        methodStr.append("return count;");
    }
    methodStr.append("}");
    System.out.println("动态生成的代码: " + methodStr);
    try {
        // 创建CtMethod对象
        CtMethod ctMethod = CtMethod.make(methodStr.toString(), ctClass);
        ctMethod.setModifiers(Modifier.PUBLIC);
        // 将方法添加到类
        ctClass.addMethod(ctMethod);
    } catch (CannotCompileException e) {
        throw new RuntimeException(e);
    }
}
});
try {
    // 创建代理对象
    Class<?> aClass = ctClass.toClass();
    Constructor<?> defaultCon = aClass.getDeclaredConstructor();
    return defaultCon.newInstance();
} catch (Exception e) {
    throw new RuntimeException(e);
}
}
}
}

```

- 修改AccountMapper.xml文件：namespace必须是dao接口的全限定名称，id必须是dao接口中的方法名：

```

<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper namespace="lzt.lztbatis.bank.dao.AccountDao">

    <select id="selectByActno" resultType="lzt.lztbatis.bank.pojo.Account">
        select * from t_act where actno = #{actno}
    </select>

    <update id="update">
        update t_act set balance = #{balance} where actno = #{actno}
    </update>

</mapper>

```

- 改造 AccountServiceImpl.java
 - 修改service类中获取dao对象的代码：
 - 现在我们不再需要 new AccountDaoImpl() 了。在 Service 开启事务获取 sqlSession 后，直接调用我们写的 Javassist 工具类来生成代理对象。

```

package lzt.lztbatis.bank.service.impl;

import lzt.lztbatis.bank.dao.AccountDao;
import lzt.lztbatis.bank.exception.AppException;
import lzt.lztbatis.bank.exception.MoneyNotEnoughException;
import lzt.lztbatis.bank.pojo.Account;

```

```

import lzt.lztbatis.bank.service.AccountService;
import lzt.lztbatis.bank.utils.GenerateDaoByJavassist;
import lzt.lztbatis.bank.utils.SqlSessionUtil;
import org.apache.ibatis.session.SqlSession;

public class AccountServiceImpl implements AccountService {

    // 删除了 private AccountDao accountDao = new AccountDaoImpl();

    @Override
    public void transfer(String fromActno, String toActno, double money) throws MoneyNotEnoughException, AppException
    {

        SqlSession sqlSession = SqlSessionUtil.openSession();

        // 动态生成 DAO 接口的代理对象
        AccountDao accountDao = (AccountDao) GenerateDaoByJavassist.getMapper(sqlSession, AccountDao.class);

        try {
            Account fromAct = accountDao.selectByActno(fromActno);
            if (fromAct == null) {
                throw new AppException("转出账户不存在!");
            }

            if (fromAct.getBalance() < money) {
                throw new MoneyNotEnoughException("对不起, 余额不足!");
            }

            Account toAct = accountDao.selectByActno(toActno);
            if (toAct == null) {
                throw new AppException("转入账户不存在!");
            }

            fromAct.setBalance(fromAct.getBalance() - money);
            toAct.setBalance(toAct.getBalance() + money);

            int count = accountDao.update(fromAct);
            if (count != 1) {
                throw new AppException("转出账户更新失败!");
            }

            count = accountDao.update(toAct);
            if (count != 1) {
                throw new AppException("转入账户更新失败!");
            }

            sqlSession.commit();

        } catch (Exception e) {
            sqlSession.rollback();
            if (e instanceof AppException) throw (AppException) e;
            if (e instanceof MoneyNotEnoughException) throw (MoneyNotEnoughException) e;
            throw new AppException("系统繁忙, 转账失败: 未知错误!");
        } finally {
            SqlSessionUtil.close(sqlSession);
        }
    }
}

```

- 启动服务器：启动过程中显示，tomcat服务器自动添加了以下的两个运行参数。所以不需要再单独配置。

银行账户转账

转出账号：

act001

转入账号：

act002

转账金额：

10000

确认转账

转账成功!!!

对象 t_act @powernode (本地MySQL) - 表		
表配置文件	开始事务	单元格编辑器 筛选
id # bigint	actno # varchar(255)	balance # decimal(15,2)
1	act001	20000.00
2	act002	30000.00

运行日志

```
D:\apacheTomcat\apache-tomcat-11.0.18\apache-tomcat-11.0.18\bin\catalina.bat run
Using CATALINA_BASE:  "C:\Users\jack\AppData\Local\JetBrains\IntelliJ IDEA 2023.2\tomcat\488ec0a3-e32d-45dd-ba80-6befc0a780cd"
Using CATALINA_HOME:  "D:\apacheTomcat\apache-tomcat-11.0.18\apache-tomcat-11.0.18"
Using CATALINA_TMPDIR: "D:\apacheTomcat\apache-tomcat-11.0.18\apache-tomcat-11.0.18\temp"
Using JRE_HOME:       "D:\Java21"
Using CLASSPATH:      "D:\apacheTomcat\apache-tomcat-11.0.18\apache-tomcat-11.0.18\bin\bootstrap.jar;D:\apacheTomcat\apache-tomcat-11.0.18\apache-tomcat-11.0.18\bin\tomcat-juli.jar"
Using CATALINA_OPTS:  ""
[2026-03-11 10:04:54,882] 工件 my_mybatis_05_web:war exploded: 正在等待服务器连接以启动工件部署...
11-Mar-2026 10:04:55.346 洪℃吨 [main] org.apache.catalina.startup.VersionLoggerListener.log Server. 鏈整姦鎔 | 增鏈:
Apache Tomcat/11.0.18
11-Mar-2026 10:04:55.348 洪℃吨 [main] org.apache.catalina.startup.VersionLoggerListener.log 鏈整姦鎔儿潛寤:
Jan 23 2026 10:22:57 UTC
11-Mar-2026 10:04:55.348 洪℃吨 [main] org.apache.catalina.startup.VersionLoggerListener.log 鏈整姦鎔 | 增鏈:
11.0.18.0
11-Mar-2026 10:04:55.348 洪℃吨 [main] org.apache.catalina.startup.VersionLoggerListener.log 鐘嶺緜緜鞏錫嶠:
Windows 11
```

11-Mar-2026 10:04:55.348	淇℃饨	[main]	org.apache.catalina.startup.VersionLoggerListener.log	OS. 銅塚滄:	10.0
11-Mar-2026 10:04:55.348	淇℃饨	[main]	org.apache.catalina.startup.VersionLoggerListener.log	鍋舵潛:	amd64
11-Mar-2026 10:04:55.349	淇℃饨	[main]	org.apache.catalina.startup.VersionLoggerListener.log	Java 整回鑒檻嘶:	
D:\Java21					
11-Mar-2026 10:04:55.349	淇℃饨	[main]	org.apache.catalina.startup.VersionLoggerListener.log	Java銅氣錠鏈虹增鏈◆:	
21.0.10+8-LTS-217					
11-Mar-2026 10:04:55.349	淇℃饨	[main]	org.apache.catalina.startup.VersionLoggerListener.log	JVM. 渚溪簫◆:	
Oracle Corporation					
11-Mar-2026 10:04:55.349	淇℃饨	[main]	org.apache.catalina.startup.VersionLoggerListener.log	CATALINA_BASE:	
C:\Users\jack\AppData\Local\JetBrains\IntelliJ\Idea2023.2\tomcat\488ec0a3-e32d-45dd-ba80-6befc0a780cd					
11-Mar-2026 10:04:55.349	淇℃饨	[main]	org.apache.catalina.startup.VersionLoggerListener.log	CATALINA_HOME:	
D:\apacheTomcat\apache-tomcat-11.0.18\apache-tomcat-11.0.18					
11-Mar-2026 10:04:55.350	淇℃饨	[main]	org.apache.catalina.startup.VersionLoggerListener.log	鍛戒护琛岍亟鑠帮細	-
Djava.util.logging.config.file=C:\Users\jack\AppData\Local\JetBrains\IntelliJ\Idea2023.2\tomcat\488ec0a3-e32d-45dd-ba80-6befc0a780cd\conf\logging.properties					
11-Mar-2026 10:04:55.350	淇℃饨	[main]	org.apache.catalina.startup.VersionLoggerListener.log	鍛戒护琛岍亟鑠帮細	-
Djava.util.logging.manager=org.apache.juli.ClassLoaderLogManager					
11-Mar-2026 10:04:55.350	淇℃饨	[main]	org.apache.catalina.startup.VersionLoggerListener.log	鍛戒护琛岍亟鑠帮細	-
Dcom.sun.management.jmxremote=					
11-Mar-2026 10:04:55.350	淇℃饨	[main]	org.apache.catalina.startup.VersionLoggerListener.log	鍛戒护琛岍亟鑠帮細	-
Dcom.sun.management.jmxremote.port=1099					
11-Mar-2026 10:04:55.350	淇℃饨	[main]	org.apache.catalina.startup.VersionLoggerListener.log	鍛戒护琛岍亟鑠帮細	-
Dcom.sun.management.jmxremote.ssl=false					
11-Mar-2026 10:04:55.350	淇℃饨	[main]	org.apache.catalina.startup.VersionLoggerListener.log	鍛戒护琛岍亟鑠帮細	-
Dcom.sun.management.jmxremote.password.file=C:\Users\jack\AppData\Local\JetBrains\IntelliJ\Idea2023.2\tomcat\488ec0a3-e32d-45dd-ba80-6befc0a780cd\jmxremote.password					
11-Mar-2026 10:04:55.351	淇℃饨	[main]	org.apache.catalina.startup.VersionLoggerListener.log	鍛戒护琛岍亟鑠帮細	-
Dcom.sun.management.jmxremote.access.file=C:\Users\jack\AppData\Local\JetBrains\IntelliJ\Idea2023.2\tomcat\488ec0a3-e32d-45dd-ba80-6befc0a780cd\jmxremote.access					
11-Mar-2026 10:04:55.351	淇℃饨	[main]	org.apache.catalina.startup.VersionLoggerListener.log	鍛戒护琛岍亟鑠帮細	-
Djava.rmi.server.hostname=127.0.0.1					
11-Mar-2026 10:04:55.351	淇℃饨	[main]	org.apache.catalina.startup.VersionLoggerListener.log	鍛戒护琛岍亟鑠帮細	-
Djdk.tls.ephemeralDHKeySize=2048					
11-Mar-2026 10:04:55.351	淇℃饨	[main]	org.apache.catalina.startup.VersionLoggerListener.log	鍛戒护琛岍亟鑠帮細	--
add-opens=java.base/java.lang=ALL-UNNAMED					
11-Mar-2026 10:04:55.351	淇℃饨	[main]	org.apache.catalina.startup.VersionLoggerListener.log	鍛戒护琛岍亟鑠帮細	--
add-opens=java.base/java.lang.reflect=ALL-UNNAMED					
11-Mar-2026 10:04:55.351	淇℃饨	[main]	org.apache.catalina.startup.VersionLoggerListener.log	鍛戒护琛岍亟鑠帮細	--
add-opens=java.base/java.io=ALL-UNNAMED					
11-Mar-2026 10:04:55.351	淇℃饨	[main]	org.apache.catalina.startup.VersionLoggerListener.log	鍛戒护琛岍亟鑠帮細	--
add-opens=java.base/java.util=ALL-UNNAMED					
11-Mar-2026 10:04:55.352	淇℃饨	[main]	org.apache.catalina.startup.VersionLoggerListener.log	鍛戒护琛岍亟鑠帮細	--
add-opens=java.base/java.util.concurrent=ALL-UNNAMED					
11-Mar-2026 10:04:55.352	淇℃饨	[main]	org.apache.catalina.startup.VersionLoggerListener.log	鍛戒护琛岍亟鑠帮細	--
add-opens=java.rmi/sun.rmi.transport=ALL-UNNAMED					
11-Mar-2026 10:04:55.352	淇℃饨	[main]	org.apache.catalina.startup.VersionLoggerListener.log	鍛戒护琛岍亟鑠帮細	--
enable-native-access=ALL-UNNAMED					
11-Mar-2026 10:04:55.352	淇℃饨	[main]	org.apache.catalina.startup.VersionLoggerListener.log	鍛戒护琛岍亟鑠帮細	-
Dcatalina.base=C:\Users\jack\AppData\Local\JetBrains\IntelliJ\Idea2023.2\tomcat\488ec0a3-e32d-45dd-ba80-6befc0a780cd					
11-Mar-2026 10:04:55.352	淇℃饨	[main]	org.apache.catalina.startup.VersionLoggerListener.log	鍛戒护琛岍亟鑠帮細	-
Dcatalina.home=D:\apacheTomcat\apache-tomcat-11.0.18\apache-tomcat-11.0.18					
11-Mar-2026 10:04:55.352	淇℃饨	[main]	org.apache.catalina.startup.VersionLoggerListener.log	鍛戒护琛岍亟鑠帮細	-
Djava.io.tmpdir=D:\apacheTomcat\apache-tomcat-11.0.18\apache-tomcat-11.0.18\temp					

```

11-Mar-2026 10:04:55.361 淇℃佻 [main] org.apache.catalina.core.AprLifecycleListener.lifecycleEvent 鏼
java.library.path:
[D:\Java21\bin;C:\WINDOWS\Sun\Java\bin;C:\WINDOWS\system32;C:\WINDOWS;D:\Java21\bin;D:\Linux\bin\;C:\Program Files
(x86)\Common Files\Intel\Shared
Libraries\redist\intel64\compiler;C:\WINDOWS\system32;C:\WINDOWS;C:\WINDOWS\System32\Wbem;C:\WINDOWS\System32\WindowsP
owerShell\v1.0\;C:\WINDOWS\System32\OpenSSH\;C:\Program Files\dotnet\;C:\Program Files (x86)\Windows Kits\10\Windows
Performance Toolkit\;D:\MATLAB\R2017b\runtime\win64;D:\MATLAB\R2017b\bin;D:\JAVAidea\maven\apache-maven-
3.9.12\bin;D:\texlive\2025\bin\windows;D:\nodejs\;C:\ProgramData\chocolatey\bin;D:\Rational\common;D:\Git\cmd;D:\curso
r\resources\app\bin;D:\apache-jmeter-5.6.3\apache-jmeter-
5.6.3\bin;D:\Python\Python313\Scripts\;D:\Python\Python313\;C:\Users\jack\AppData\Local\Microsoft\WindowsApps;C:\Users
\jack\.dotnet\tools;D:\JAVAidea\IntelliJ IDEA 2023.2\bin;D:\VS Code\bin;D:\pych;D:\allure-2.36.0\allure-commandline-
2.36.0\allure-2.36.0\bin;C:\Program Files\MySQL\MySQL Server
8.4\bin;D:\Python\Python313\Scripts\;D:\Python\Python313\;C:\Users\jack\AppData\Local\Microsoft\WindowsApps;C:\Users\j
ack\.dotnet\tools;D:\Java\bin;D:\JAVAidea\IntelliJ IDEA 2023.2\bin;D:\VS Code\bin;D:\pycharm\PyCharm
2025.1.2\bin;C:\Users\jack\AppData\Roaming\Programs\Zero
Install;D:\Anaconda3\Scripts;D:\Anaconda3\Library\bin;D:\Anaconda3\Library\usr\bin;D:\Anaconda3\D:\Anaconda3\Library\m
ingw-
w64;C:\Users\jack\AppData\Local\Programs\Ollama;C:\Users\jack\AppData\Roaming\npm;C:\Users\jack\AppData\Local\GitHubDe
sktop\bin;. ]涓捐壘涓查垠鍐测銆露APR鑽澤pache Tomcat鏈回溝辜擄紕璇 旂錦攸回鏼 洊汲x喝澧秭腑鑾峰緱鏈 浣虫 u昃
11-Mar-2026 10:04:55.461 淇℃佻 [main] org.apache.coyote.AbstractProtocol.init 錄漬回錯棧崗璽回鐫�唱榛 ["http-nio-8080"]
11-Mar-2026 10:04:55.472 淇℃佻 [main] org.apache.catalina.startup.Catalina.load 鏈楚炫錯厶湲[273]姁回鍑岀啗濞�寔
11-Mar-2026 10:04:55.506 淇℃佻 [main] org.apache.catalina.core.StandardService.startInternal 妹e 淥錫回婢鏈楚炫
[Catalina]
11-Mar-2026 10:04:55.506 淇℃佻 [main] org.apache.catalina.core.StandardEngine.startInternal 妹e 淥錫回婢 Servlet 寮晨捺
銚欵Apache Tomcat/11.0.18]
11-Mar-2026 10:04:55.516 淇℃佻 [main] org.apache.coyote.AbstractProtocol.start 寮 濮�崗璽回鐫�唱鼻鎢捌"http-nio-8080"]
11-Mar-2026 10:04:55.547 淇℃佻 [main] org.apache.catalina.startup.Catalina.start [74]姁回錫座滑鐫 榛錫回婢
已连接到服务器
[2026-03-11 10:04:55,956] 工件 my_mybatis_05_web:war exploded: 正在部署工件, 请稍候...
11-Mar-2026 10:04:56.579 淇℃佻 [RMI TCP Connection(2)-127.0.0.1] org.apache.jasper.servlet.TldScanner.scanJars 鑷冲奸
鏈变灣涓猪AR琨回堅鎖忒狀浜�LD浡唱敵鏈回真錫厶LD鉅 涓烘回璽板綌錯厶基塈x績瓩晨樺塞梟回蹇羅紅涕 峽鑿棰�凡錄回窖浣啖酒鏼厶呂涓回墨錄
癘LD鑽勳畝鑼碯AR錄偏 鉅 鏼儿堅鎖忒渴閤磋煩枙笮回 假忒堛JAR鑿回互緋 烈錫回婢鍍墜悅鍛孟SP緋栉樺鍍墜悅鉅
[2026-03-11 10:04:56,778] 工件 my_mybatis_05_web:war exploded: 工件已成功部署
[2026-03-11 10:04:56,778] 工件 my_mybatis_05_web:war exploded: 部署已花费 822 毫秒
11-Mar-2026 10:05:05.523 淇℃佻 [Catalina-utility-1] org.apache.catalina.startup.HostConfig.deployDirectory 鐫崙eb 褁旈
敷綿嬪簞闊 | 讲錄扮汨囊 [D:\apacheTomcat\apache-tomcat-11.0.18\apache-tomcat-11.0.18\webapps\manager]
11-Mar-2026 10:05:05.730 淇℃佻 [Catalina-utility-1] org.apache.catalina.startup.HostConfig.deployDirectory Web褁旈敷綿
嬪簞鐫回緋[D:\apacheTomcat\apache-tomcat-11.0.18\apache-tomcat-11.0.18\webapps\manager]鑽勳儀緋插凡鏼 207]姁回鍑�器奮鋹
动态生成的代码: int update(lzt.lztbatis.bank.pojo.Account arg0){org.apache.ibatis.session.SqlSession sqlSession =
lzt.lztbatis.bank.utils.SqlSessionUtil.openSession();int count =
sqlSession.update("lzt.lztbatis.bank.dao.AccountDao.update", arg0);return count;}
动态生成的代码: lzt.lztbatis.bank.pojo.Account selectByActno(java.lang.String arg0)
{org.apache.ibatis.session.SqlSession sqlSession = lzt.lztbatis.bank.utils.SqlSessionUtil.openSession();Object obj =
sqlSession.selectOne("lzt.lztbatis.bank.dao.AccountDao.selectByActno", arg0);return
(lzt.lztbatis.bank.pojo.Account)obj;}
[http-nio-8080-exec-10] DEBUG l.l.b.dao.AccountDao.selectByActno - ==> Preparing: select * from t_act where actno = ?
[http-nio-8080-exec-10] DEBUG l.l.b.dao.AccountDao.selectByActno - ==> Parameters: act001(String)
[http-nio-8080-exec-10] DEBUG l.l.b.dao.AccountDao.selectByActno - <== Total: 1
[http-nio-8080-exec-10] DEBUG l.l.b.dao.AccountDao.selectByActno - ==> Preparing: select * from t_act where actno = ?
[http-nio-8080-exec-10] DEBUG l.l.b.dao.AccountDao.selectByActno - ==> Parameters: act002(String)
[http-nio-8080-exec-10] DEBUG l.l.b.dao.AccountDao.selectByActno - <== Total: 1
[http-nio-8080-exec-10] DEBUG l.l.bank.dao.AccountDao.update - ==> Preparing: update t_act set balance = ? where
actno = ?
[http-nio-8080-exec-10] DEBUG l.l.bank.dao.AccountDao.update - ==> Parameters: 20000.0(Double), act001(String)
[http-nio-8080-exec-10] DEBUG l.l.bank.dao.AccountDao.update - <== Updates: 1
[http-nio-8080-exec-10] DEBUG l.l.bank.dao.AccountDao.update - ==> Preparing: update t_act set balance = ? where
actno = ?
[http-nio-8080-exec-10] DEBUG l.l.bank.dao.AccountDao.update - ==> Parameters: 30000.0(Double), act002(String)
[http-nio-8080-exec-10] DEBUG l.l.bank.dao.AccountDao.update - <== Updates: 1

```

- 关于 Tomcat 的启动参数

--add-opens=java.base/java.lang=ALL-UNNAMED

--add-opens=java.base/java.lang.reflect=ALL-UNNAMED

说明较新版本的 Javassist 配合 Tomcat 自动开放的 java.lang 和 java.lang.reflect 模块，已经完全足够完成底层的字节码操纵。

- Javassist 成功接管了工作

```
动态生成的代码: int update(lzt.lztbatis.bank.pojo.Account arg0){org.apache.ibatis.session.SqlSession sqlSession = lzt.lztbatis.bank.utils.SqlSessionUtil.openSession();
动态生成的代码: lzt.lztbatis.bank.pojo.Account selectByActno(java.lang.String arg0){org.apache.ibatis.session.SqlSession sqlSession = lzt.lztbatis.bank.utils.SqlSessionUtil.openSession();
```

这证明我们写的 GenerateDaoByJavassist.getMapper() 工具类成功在内存中动态组装出了 AccountDaoImpl 的方法体。

- MyBatis 顺利执行了 SQL

```
[http-nio-8080-exec-10] DEBUG l.l.b.dao.AccountDao.selectByActno - ==> Preparing: select * from t_act where actno = ?
[http-nio-8080-exec-10] DEBUG l.l.b.dao.AccountDao.selectByActno - ==> Parameters: act001(String)
[http-nio-8080-exec-10] DEBUG l.l.b.dao.AccountDao.selectByActno - <==      Total: 1
[http-nio-8080-exec-10] DEBUG l.l.b.dao.AccountDao.selectByActno - ==> Preparing: select * from t_act where actno = ?
[http-nio-8080-exec-10] DEBUG l.l.b.dao.AccountDao.selectByActno - ==> Parameters: act002(String)
[http-nio-8080-exec-10] DEBUG l.l.b.dao.AccountDao.selectByActno - <==      Total: 1
[http-nio-8080-exec-10] DEBUG l.l.b.bank.dao.AccountDao.update - ==> Preparing: update t_act set balance = ? where actno = ?
[http-nio-8080-exec-10] DEBUG l.l.b.bank.dao.AccountDao.update - ==> Parameters: 20000.0(Double), act001(String)
[http-nio-8080-exec-10] DEBUG l.l.b.bank.dao.AccountDao.update - <==      Updates: 1
[http-nio-8080-exec-10] DEBUG l.l.b.bank.dao.AccountDao.update - ==> Preparing: update t_act set balance = ? where actno = ?
[http-nio-8080-exec-10] DEBUG l.l.b.bank.dao.AccountDao.update - ==> Parameters: 30000.0(Double), act002(String)
[http-nio-8080-exec-10] DEBUG l.l.b.bank.dao.AccountDao.update - <==      Updates: 1
```

MyBatis中接口代理机制及使用

上一步我们用 Javassist 写的动态生成类的代码，其实 MyBatis 底层早就帮我们封装好了。只需要调用 sqlSession.getMapper()，MyBatis 就会在内存中动态为你生成 DAO 接口的实现类

```
AccountDao accountDao = (AccountDao)sqlSession.getMapper(AccountDao.class);
```

使用以上代码的前提是：AccountMapper.xml文件中的namespace必须和dao接口的全限定名称一致，id必须和dao接口中方法名一致。

上一步已经把 AccountMapper.xml 的 namespace 和 id 都严格按照接口全限定名和方法名配好了

- 修改 AccountServiceImpl.java

```
package lzt.lztbatis.bank.service.impl;

import lzt.lztbatis.bank.dao.AccountDao;
import lzt.lztbatis.bank.exception.AppException;
import lzt.lztbatis.bank.exception.MoneyNotEnoughException;
import lzt.lztbatis.bank.pojo.Account;
import lzt.lztbatis.bank.service.AccountService;
import lzt.lztbatis.bank.utils.SqlSessionUtil;
import org.apache.ibatis.session.SqlSession;

public class AccountServiceImpl implements AccountService {

    // 注释掉之前自己写的实现类
    // private AccountDao accountDao = new AccountDaoImpl();

    // 注释掉上一步手写的 Javassist 动态代理
    // private AccountDao accountDao = (AccountDao)GenerateDaoByJavassist.getMapper(SqlSessionUtil.openSession(),
    AccountDao.class);
```



```

@Override
public void transfer(String fromActno, String toActno, double money) throws MoneyNotEnoughException, AppException
{
    // 1. 获取当前线程绑定的 SqlSession (开启事务)
    SqlSession sqlSession = SqlSessionUtil.openSession();

    // 2. 核心魔法: 使用 MyBatis 内置的接口代理机制获取 DAO 实现类
    AccountDao accountDao = sqlSession.getMapper(AccountDao.class);

    try {
        // 3. 查询转出账户的信息
        Account fromAct = accountDao.selectByActno(fromActno);
        if (fromAct == null) {
            throw new AppException("转出账户不存在!");
        }

        // 4. 判断余额是否充足
        if (fromAct.getBalance() < money) {
            throw new MoneyNotEnoughException("对不起, 余额不足!");
        }

        // 5. 余额充足, 查询转入账户的信息
        Account toAct = accountDao.selectByActno(toActno);
        if (toAct == null) {
            throw new AppException("转入账户不存在!");
        }

        // 6. 在内存中修改两个账户的余额
        fromAct.setBalance(fromAct.getBalance() - money);
        toAct.setBalance(toAct.getBalance() + money);

        // 7. 调用代理对象的方法更新数据库
        int count = accountDao.update(fromAct);
        if (count != 1) {
            throw new AppException("转出账户更新失败!");
        }

        count = accountDao.update(toAct);
        if (count != 1) {
            throw new AppException("转入账户更新失败!");
        }

        // 8. 事务提交
        sqlSession.commit();

    } catch (Exception e) {
        // 9. 事务回滚
        sqlSession.rollback();
        if (e instanceof AppException) throw (AppException) e;
        if (e instanceof MoneyNotEnoughException) throw (MoneyNotEnoughException) e;
        throw new AppException("系统繁忙, 转账失败: 未知错误!");
    } finally {
        // 10. 释放资源并清空 ThreadLocal
        SqlSessionUtil.close(sqlSession);
    }
}
}

```

既然 MyBatis 已经帮我们接管了这部分工作, 你可以安全地做以下清理了:

- 删除 GenerateDaoByJavassist.java 工具类
- 在 pom.xml 中删除 javassist 的依赖

MyBatis 内部自带了实现机制，不需要我们额外引包了

测试前数据：

表配置文件			开始事务		单元格编辑器	筛选
id	actno	balance				
# bigint	# varchar(255)	# decimal(15,2)				
1	act001	20000.00				
2	act002	30000.00				

银行账户转账

转出账号：

act001

转入账号：

act002

转账金额：

5000

确认转账

测试后数据：

转账成功！！！！

对象 t_act @powernode (本地MySQL) - 表					单元格编辑器	筛选
id	actno	balance				
# bigint	# varchar(255)	# decimal(15,2)				
1	act001	15000.00				
2	act002	35000.00				

MyBatis小技巧

- 准备工作
- 数据库：mybatis_demo
- 数据表：t_car

字段	索引	外键	检查	触发器	选项	注释	SQL 预览		
名称			类型	长度	小数点	不是 null	虚拟	键	注释
id			bigint			☒	☐	1	主键
car_num			varchar	255		☐	☐		汽车编号
brand			varchar	255		☐	☐		汽车品牌
guide_price			decimal	10	2	☐	☐		厂家指导价
produce_time			char	10		☐	☐		生产日期
car_type			varchar	255		☐	☐		汽车类型(燃油车/电车/氢能源)

id # bigint	car_num #C varchar(255)	brand #C varchar(255)	guide_price # decimal(10,2)	produce_time #C char(10)	car_type #C varchar(255)
1	100	宝马520Li	41.00	2022-09-01	燃油车
2	101	奔驰E300L	54.00	2022-08-01	电车
3	102	丰田未来	40.30	2014-10-05	氢能源
4	102	丰田未来	40.30	2014-10-05	氢能源
5	102	丰田未来	40.30	2014-10-05	氢能源
6	102	丰田未来	40.30	2014-10-05	氢能源
7	102	丰田未来	40.30	2014-10-05	氢能源
8	102	丰田未来	40.30	2014-10-05	氢能源
9	102	比亚迪汉	30.23	2018-09-10	电车

项目名称：my_mybatis-06-antic

新建项目

新建项目

空项目

生成器

Maven Archetype

Jakarta EE

Spring Initializr

JavaFX

Quarkus

Micronaut

Ktor

Kotlin Multiplatform

Compose for Desktop

HTML 文件

React

Express

Angular CLI

IDE 插件

Android

Vue.js

Vite

名称:

my_mybatis-06-antic

位置:

D:\Java_mybatis_study

项目将在以下位置创建: D:\Java_myb..._study\my_mybatis-06-antic

☐ 创建 Git 仓库

语言:

JavaKotlinGroovyJavaScript+

构建系统:

IntelliJMavenGradle

JDK:

21 Oracle OpenJDK version 21.0.10

☒ 添加示例代码

☒ 使用入门提示生成代码

高级设置

组 ID:

lzt.mybatis

工件 ID:

my_mybatis-06-antic

创建(C)

取消

#{}和\${}

- #{}：先编译sql语句，再给占位符传值，底层是PreparedStatement实现，使用？占位符。可以防止sql注入，比较常用。

- `${}`：先进行sql语句拼接，然后再编译sql语句，底层是Statement实现。存在sql注入现象。只有在需要进行sql语句关键字拼接的情况下才会用到。

需求：根据car_type查询汽车

使用#{}

- 配置 pom.xml
 - 引入依赖

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>lzt.mybatis</groupId>
    <artifactId>my_mybatis-06-antic</artifactId>
    <version>1.0-SNAPSHOT</version>
    <packaging>jar</packaging>

    <dependencies>
        <dependency>
            <groupId>org.mybatis</groupId>
            <artifactId>mybatis</artifactId>
            <version>3.5.19</version>
        </dependency>
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <version>8.4.0</version>
        </dependency>
        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>4.13.2</version>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>ch.qos.logback</groupId>
            <artifactId>logback-classic</artifactId>
            <version>1.5.6</version>
        </dependency>
    </dependencies>

    <properties>
        <maven.compiler.source>21</maven.compiler.source>
        <maven.compiler.target>21</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

</project>
```

- jdbc.properties

```
jdbc.driver=com.mysql.cj.jdbc.Driver
jdbc.url=jdbc:mysql://localhost:3306/mybatis_demo?useUnicode=true&characterEncoding=utf-8&serverTimezone=Asia/Shanghai
jdbc.username=root
jdbc.password=20060329
```

- logback.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
    <appender name="STDOUT" class="ch.qos.logback.core.ConsoleAppender">
        <encoder>
            <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n</pattern>
        </encoder>
    </appender>

    <logger name="Lzt.mybatis.mapper" level="DEBUG"/> <root level="INFO">
        <appender-ref ref="STDOUT" />
    </root>
</configuration>
```

- 配置: utils
 - SqlSessionUtil
 - src/main/java/lzt/mybatis/utils 包下创建

```
package lzt.mybatis.utils;

import org.apache.ibatis.io.Resources;
import org.apache.ibatis.session.SqlSession;
import org.apache.ibatis.session.SqlSessionFactory;
import org.apache.ibatis.session.SqlSessionFactoryBuilder;

/**
 * MyBatis工具类
 *
 */
public class SqlSessionUtil {
    private static SqlSessionFactory sqlSessionFactory;

    /**
     * 类加载时初始化sqlSessionFactory对象
     */
    static {
        try {
            SqlSessionFactoryBuilder sqlSessionFactoryBuilder = new SqlSessionFactoryBuilder();
            sqlSessionFactory = sqlSessionFactoryBuilder.build(Resources.getResourceAsStream("mybatis-config.xml"));
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    private static ThreadLocal<SqlSession> local = new ThreadLocal<>();

    /**
     * 每调用一次openSession()可获取一个新的会话，该会话支持自动提交。
     *
     * @return 新的会话对象
     */
    public static SqlSession openSession() {
        SqlSession sqlSession = local.get();
        if (sqlSession == null) {
            sqlSession = sqlSessionFactory.openSession();
            local.set(sqlSession);
        }
        return sqlSession;
    }

    /**
     * 关闭SqlSession对象
     */
}
```

```

    * @param sqlSession
    */
    public static void close(SqlSession sqlSession){
        if (sqlSession != null) {
            sqlSession.close();
        }
        // 这里非常重要，一定要remove，避免内存泄漏！
        local.remove();
    }
}

```

- 实体类：pojo
 - Car.java
 - 放在 lzt.mybatis.pojo 包下

```

package lzt.mybatis.pojo;

/**
 * 普通实体类：汽车
 */
public class Car {
    private Long id;
    private String carNum;
    private String brand;
    private Double guidePrice;
    private String produceTime;
    private String carType;

    // 1. 无参数构造方法
    public Car() {
    }

    // 2. 全参数构造方法
    public Car(Long id, String carNum, String brand, Double guidePrice, String produceTime, String carType) {
        this.id = id;
        this.carNum = carNum;
        this.brand = brand;
        this.guidePrice = guidePrice;
        this.produceTime = produceTime;
        this.carType = carType;
    }

    // 3. getter 和 setter 方法
    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getCarNum() {
        return carNum;
    }

    public void setCarNum(String carNum) {
        this.carNum = carNum;
    }

    public String getBrand() {
        return brand;
    }
}

```

```

public void setBrand(String brand) {
    this.brand = brand;
}

public Double getGuidePrice() {
    return guidePrice;
}

public void setGuidePrice(Double guidePrice) {
    this.guidePrice = guidePrice;
}

public String getProduceTime() {
    return produceTime;
}

public void setProduceTime(String produceTime) {
    this.produceTime = produceTime;
}

public String getCarType() {
    return carType;
}

public void setCarType(String carType) {
    this.carType = carType;
}

// 4. 重写 toString 方法
@Override
public String toString() {
    return "Car{" +
        "id=" + id +
        ", carNum='" + carNum + '\'' +
        ", brand='" + brand + '\'' +
        ", guidePrice=" + guidePrice +
        ", produceTime='" + produceTime + '\'' +
        ", carType='" + carType + '\'' +
        '}';
}
}

```

- Mapper 接口：CarMapper
 - src/main/java/lzt/mybatis/mapper 包下创建

```

package lzt.mybatis.mapper;
import lzt.mybatis.pojo.Car;
import java.util.List;

/**
 * Car 的sql映射对象
 */
public interface CarMapper {

    /**
     * 根据car_type获取Car
     * @param carType 汽车类型
     * @return 汽车集合
     */
    List<Car> selectByCarType(String carType);
}

```

```
}
```

- 核心配置文件：mybatis-config.xml

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE configuration
    PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-config.dtd">
<configuration>
    <properties resource="jdbc.properties"/>

    <environments default="dev">
        <environment id="dev">
            <transactionManager type="JDBC"/>
            <dataSource type="POOLED">
                <property name="driver" value="${jdbc.driver}"/>
                <property name="url" value="${jdbc.url}"/>
                <property name="username" value="${jdbc.username}"/>
                <property name="password" value="${jdbc.password}"/>
            </dataSource>
        </environment>
    </environments>

    <mappers>
        <mapper resource="CarMapper.xml"/>
    </mappers>
</configuration>
```

- SQL 映射文件：CarMapper.xml
 - 注意namespace必须和接口名一致。id必须和接口中方法名一致。

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper namespace="lzt.mybatis.mapper.CarMapper">

    <select id="selectByCarType" resultType="lzt.mybatis.pojo.Car">
        select
            id, car_num as carNum, brand, guide_price as guidePrice, produce_time as produceTime, car_type as carType
        from
            t_car
        where
            car_type = #{carType}
    </select>
</mapper>
```

- 测试类：CarMapperTest.java
 - 放在 src/test/java/lzt/mybatis/test 包

```
package lzt.mybatis.test;

import lzt.mybatis.mapper.CarMapper;
import lzt.mybatis.pojo.Car;
import lzt.mybatis.utils.SqlSessionUtil;
import org.junit.Test;

import java.util.List;
```



```

/**
 * CarMapper 测试类
 */
public class CarMapperTest {

    @Test
    public void testSelectByCarType(){
        // 获取 Mapper 的代理对象
        CarMapper mapper = SqlSessionUtil.openSession().getMapper(CarMapper.class);

        // 调用方法, 测试 #{ } 传值
        List<Car> cars = mapper.selectByCarType("燃油车");

        // 遍历输出结果
        cars.forEach(car -> System.out.println(car));

        // 记得关闭会话
        SqlSessionUtil.close(SqlSessionUtil.openSession());
    }
}

```

- 执行结果:

```

2026-03-11 14:01:20.178 [main] DEBUG l.m.mapper.CarMapper.selectByCarType - ==> Preparing: select id, car_num as carNum, brand,
guide_price as guidePrice, produce_time as produceTime, car_type as carType from t_car where car_type = ?
2026-03-11 14:01:20.209 [main] DEBUG l.m.mapper.CarMapper.selectByCarType ==> Parameters: 燃油车(String)
2026-03-11 14:01:20.286 [main] DEBUG l.m.mapper.CarMapper.selectByCarType - <== Total: 14
Car{id=1, carNum='100', brand='宝马520Li', guidePrice=41.0, produceTime='2022-09-01', carType='燃油车'}
Car{id=12, carNum='104', brand='开发库测试车', guidePrice=10.0, produceTime='2026-01-01', carType='燃油车'}
Car{id=13, carNum='DEV-001', brand='程序员本地自测车', guidePrice=10.0, produceTime='2026-01-01', carType='燃油车'}
Car{id=14, carNum='DEV-001', brand='程序员本地自测车', guidePrice=10.0, produceTime='2026-01-01', carType='燃油车'}
Car{id=15, carNum='DEV-001', brand='程序员本地自测车', guidePrice=10.0, produceTime='2026-01-01', carType='燃油车'}
Car{id=16, carNum='DEV-001', brand='程序员本地自测车', guidePrice=10.0, produceTime='2026-01-01', carType='燃油车'}
Car{id=19, carNum='MANAGED-999', brand='甩手掌柜测试车', guidePrice=99.9, produceTime='2026-10-10', carType='燃油车'}
Car{id=20, carNum='MANAGED-999', brand='MANAGED测试车', guidePrice=99.9, produceTime='2026-10-10', carType='燃油车'}
Car{id=21, carNum='DEV-001', brand='程序员本地自测车', guidePrice=10.0, produceTime='2026-01-01', carType='燃油车'}
Car{id=24, carNum='MANAGED-999', brand='MANAGED-999', guidePrice=99.9, produceTime='2026-10-10', carType='燃油车'}
Car{id=25, carNum='UNPOOLED-01', brand='非连接池测试车1', guidePrice=10.0, produceTime='2026-04-01', carType='燃油车'}
Car{id=26, carNum='UNPOOLED-02', brand='非连接池测试车2', guidePrice=20.0, produceTime='2026-04-02', carType='燃油车'}
Car{id=27, carNum='UNPOOLED-01', brand='非连接池测试车1', guidePrice=10.0, produceTime='2026-04-01', carType='燃油车'}
Car{id=28, carNum='UNPOOLED-02', brand='非连接池测试车2', guidePrice=20.0, produceTime='2026-04-02', carType='燃油车'}

```

通过执行可以清楚的看到, sql语句中是带有 ? 的, 这个 ? 就是大家在JDBC中所学的占位符, 专门用来接收值的。

把“燃油车”以String类型的值, 传递给 ?

这就是 #{ }, 它会先进行sql语句的预编译, 然后再给占位符传值。

使用\${ }

同样的需求, 我们使用 \${ } 来完成

CarMapper.xml文件修改如下:

```

<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper namespace="lzt.mybatis.mapper.CarMapper">
    <select id="selectByCarType" resultType="lzt.mybatis.pojo.Car">
        select
            id, car_num as carNum, brand, guide_price as guidePrice, produce_time as produceTime, car_type as carType

```

```

        from
            t_car
        where
            car_type = ${carType}
    </select>
</mapper>

```

再次运行测试程序：

```

D:\Java21\bin\java.exe ...
2026-03-11 14:07:00.310 [main] DEBUG l.m.mapper.CarMapper.selectByCarType - ==> Preparing: select id, car_num as carNum, brand,
    guide_price as guidePrice, produce_time as produceTime, car_type as carType from t_car where car_type = 燃油车
2026-03-11 14:07:00.338 [main] DEBUG l.m.mapper.CarMapper.selectByCarType - ==> Parameters:

org.apache.ibatis.exceptions.PersistenceException:
### Error querying database.  Cause: java.sql.SQLException: Unknown column '燃油车' in 'where clause'
### The error may exist in CarMapper.xml
### The error may involve defaultParameterMap
### The error occurred while setting parameters
### SQL: select          id, car_num as carNum, brand, guide_price as guidePrice, produce_time as produceTime, car_type as
    carType          from          t_car          where          car_type = 燃油车
### Cause: java.sql.SQLException: Unknown column '燃油车' in 'where clause'

```

出现异常了，这是为什么呢？看看生成的sql语句：

```

2026-03-11 14:07:00.310 [main] DEBUG l.m.mapper.CarMapper.selectByCarType - ==> Preparing: select id, car_num as carNum, brand,
    guide_price as guidePrice, produce_time as produceTime, car_type as carType from t_car where car_type = 燃油车
2026-03-11 14:07:00.338 [main] DEBUG l.m.mapper.CarMapper.selectByCarType - ==> Parameters:

```

很显然，`${}` 是先进行sql语句的拼接，然后再编译，出现语法错误是正常的，因为 燃油车 是一个字符串，在sql语句中应该添加单引号

修改：

```

<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper namespace="lzt.mybatis.mapper.CarMapper">

    <select id="selectByCarType" resultType="lzt.mybatis.pojo.Car">
        select
            id, car_num as carNum, brand, guide_price as guidePrice, produce_time as produceTime, car_type as carType
        from
            t_car
        where
            car_type = '${carType}'
    </select>

</mapper>

```

再执行测试程序：

```
D:\Java21\bin\java.exe ...
2026-03-11 14:11:33.080 [main] DEBUG l.m.mapper.CarMapper.selectByCarType - ==> Preparing: select id, car_num as carNum, brand,
  guide_price as guidePrice, produce_time as produceTime, car_type as carType from t_car where car_type = '燃油车'
2026-03-11 14:11:33.115 [main] DEBUG l.m.mapper.CarMapper.selectByCarType - ==> Parameters:
2026-03-11 14:11:33.160 [main] DEBUG l.m.mapper.CarMapper.selectByCarType - <==          Total: 14
Car{id=1, carNum='100', brand='宝马520Li', guidePrice=41.0, produceTime='2022-09-01', carType='燃油车'}
Car{id=12, carNum='104', brand='开发库测试车', guidePrice=10.0, produceTime='2026-01-01', carType='燃油车'}
Car{id=13, carNum='DEV-001', brand='程序员本地自测车', guidePrice=10.0, produceTime='2026-01-01', carType='燃油车'}
Car{id=14, carNum='DEV-001', brand='程序员本地自测车', guidePrice=10.0, produceTime='2026-01-01', carType='燃油车'}
Car{id=15, carNum='DEV-001', brand='程序员本地自测车', guidePrice=10.0, produceTime='2026-01-01', carType='燃油车'}
Car{id=16, carNum='DEV-001', brand='程序员本地自测车', guidePrice=10.0, produceTime='2026-01-01', carType='燃油车'}
Car{id=19, carNum='MANAGED-999', brand='甩手掌柜测试车', guidePrice=99.9, produceTime='2026-10-10', carType='燃油车'}
Car{id=20, carNum='MANAGED-999', brand='MANAGED测试车', guidePrice=99.9, produceTime='2026-10-10', carType='燃油车'}
Car{id=21, carNum='DEV-001', brand='程序员本地自测车', guidePrice=10.0, produceTime='2026-01-01', carType='燃油车'}
Car{id=24, carNum='MANAGED-999', brand='MANAGED测试车', guidePrice=99.9, produceTime='2026-10-10', carType='燃油车'}
Car{id=25, carNum='UNPOOLED-01', brand='非连接池测试车1', guidePrice=10.0, produceTime='2026-04-01', carType='燃油车'}
Car{id=26, carNum='UNPOOLED-02', brand='非连接池测试车2', guidePrice=20.0, produceTime='2026-04-02', carType='燃油车'}
Car{id=27, carNum='UNPOOLED-01', brand='非连接池测试车1', guidePrice=10.0, produceTime='2026-04-01', carType='燃油车'}
Car{id=28, carNum='UNPOOLED-02', brand='非连接池测试车2', guidePrice=20.0, produceTime='2026-04-02', carType='燃油车'}
```

通过以上测试，可以看出，对于以上这种需求来说，还是建议使用#{ } 的方式。

原则：能用#{ } 就不用\${ }

什么情况下必须使用\${ }

当需要进行sql语句关键字拼接的时候。必须使用\${ }

需求：通过向sql语句中注入asc或desc关键字，来完成数据的升序或降序排列。

- 先使用#{ }尝试：

修改 CarMapper.java 接口

追加 selectAll 方法：

```
package lzt.mybatis.mapper;

import lzt.mybatis.pojo.Car;
import java.util.List;

/**
 * Car的sql映射对象
 */
public interface CarMapper {

    /**
     * 根据car_type获取Car
     * @param carType 汽车类型
     * @return 汽车集合
     */
    List<Car> selectByCarType(String carType);

    /**
     * 查询所有的Car并按指定规则排序
     * @param ascOrDesc asc或desc
     * @return 汽车集合
     */
    List<Car> selectAll(String ascOrDesc);

}
```

修改 CarMapper.xml 映射文件

在 `<mapper>` 标签内追加如下的 `<select>` 标签。



```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper namespace="Lzt.mybatis.mapper.CarMapper">

    <select id="selectByCarType" resultType="Lzt.mybatis.pojo.Car">
        select
            id, car_num as carNum, brand, guide_price as guidePrice, produce_time as produceTime, car_type as carType
        from
            t_car
        where
            car_type = '${carType}'
    </select>

    <select id="selectAll" resultType="Lzt.mybatis.pojo.Car">
        select
            id, car_num as carNum, brand, guide_price as guidePrice, produce_time as produceTime, car_type as carType
        from
            t_car
        order by carNum #{ascOrDesc}
    </select>

</mapper>
```

修改 CarMapperTest.java 测试类



```
package lzt.mybatis.test;

import lzt.mybatis.mapper.CarMapper;
import lzt.mybatis.pojo.Car;
import lzt.mybatis.utils.SqlSessionUtil;
import org.junit.Test;

import java.util.List;

public class CarMapperTest {

    // ... 之前的 testSelectByCarType 方法 ...

    @Test
    public void testSelectAll(){
        CarMapper mapper = SqlSessionUtil.openSession().getMapper(CarMapper.class);
        // 尝试传入 desc 进行降序排列
        List<Car> cars = mapper.selectAll("desc");
        cars.forEach(car -> System.out.println(car));

        // 关闭会话
        SqlSessionUtil.close(SqlSessionUtil.openSession());
    }
}
```

运行：

```
D:\Java21\bin\java.exe ...
```

```
2026-03-11 14:19:06.652 [main] DEBUG l.mybatis.mapper.CarMapper.selectAll - ==> Preparing: select id, car_num as carNum, brand, guide_price as guidePrice, produce_time as produceTime, car_type as carType from t_car order by carNum ?
```

```
2026-03-11 14:19:06.689 [main] DEBUG l.mybatis.mapper.CarMapper.selectAll - ==> Parameters: desc(String)
```

```
org.apache.ibatis.exceptions.PersistenceException:
```

```
### Error querying database. Cause: java.sql.SQLException: You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near ''desc'' at line 5
```

```
### The error may exist in CarMapper.xml
```

```
### The error may involve defaultParameterMap
```

```
### The error occurred while setting parameters
```

```
### SQL: select id, car_num as carNum, brand, guide_price as guidePrice, produce_time as produceTime, car_type as carType from t_car order by carNum ?
```

```
### Cause: java.sql.SQLException: You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near ''desc'' at line 5
```

```
at org.apache.ibatis.exceptions.ExceptionFactory.wrapException(ExceptionFactory.java:30)
at org.apache.ibatis.session.defaults.DefaultSqlSession.selectList(DefaultSqlSession.java:156)
at org.apache.ibatis.session.defaults.DefaultSqlSession.selectList(DefaultSqlSession.java:147)
at org.apache.ibatis.session.defaults.DefaultSqlSession.selectList(DefaultSqlSession.java:142)
at org.apache.ibatis.binding.MapperMethod.executeForMany(MapperMethod.java:147)
at org.apache.ibatis.binding.MapperMethod.execute(MapperMethod.java:80)
at org.apache.ibatis.binding.MapperProxy$PlainMethodInvoker.invoke(MapperProxy.java:141)
at org.apache.ibatis.binding.MapperProxy.invoke(MapperProxy.java:86)
```

报错的原因是sql语句不合法，因为采用这种方式传值，最终sql语句会是这样：

```
select id,car_num as carNum,brand,guide_price as guidePrice,produce_time as produceTime,car_type as carType from t_car order by carNum 'desc'
```

desc是一个关键字，不能带单引号的，所以在进行sql语句关键字拼接的时候，必须使用\${}

- 使用\${} 改造

```
<select id="selectAll" resultType="lzt.mybatis.pojo.Car">
  select
    id, car_num as carNum, brand, guide_price as guidePrice, produce_time as produceTime, car_type as carType
  from
    t_car
  order by carNum ${ascOrDesc}
</select>
```

再次执行测试程序：

```
2026-03-11 14:52:28.804 [main] DEBUG l.mybatis.mapper.CarMapper.selectAll - ==> Preparing: select id, car_num as carNum, brand, guide_price as guidePrice, produce_time as produceTime, car_type as carType from t_car order by carNum desc
2026-03-11 14:52:28.835 [main] DEBUG l.mybatis.mapper.CarMapper.selectAll - ==> Parameters:
2026-03-11 14:52:28.890 [main] DEBUG l.mybatis.mapper.CarMapper.selectAll - <==      Total: 27
Car{id=26, carNum='UNPOOLED-02', brand='非连接池测试车2', guidePrice=20.0, produceTime='2026-04-02', carType='燃油车'}
Car{id=28, carNum='UNPOOLED-02', brand='非连接池测试车2', guidePrice=20.0, produceTime='2026-04-02', carType='燃油车'}
Car{id=25, carNum='UNPOOLED-01', brand='非连接池测试车1', guidePrice=10.0, produceTime='2026-04-01', carType='燃油车'}
Car{id=27, carNum='UNPOOLED-01', brand='非连接池测试车1', guidePrice=10.0, produceTime='2026-04-01', carType='燃油车'}
Car{id=17, carNum='TEST-002', brand='测试组找Bug专用车', guidePrice=20.0, produceTime='2026-02-02', carType='电车'}
Car{id=22, carNum='TEST-002', brand='测试组找Bug专用车', guidePrice=20.0, produceTime='2026-02-02', carType='电车'}
Car{id=18, carNum='PROD-003', brand='真实用户购买的量产车', guidePrice=30.0, produceTime='2026-03-03', carType='氢能源'}
Car{id=23, carNum='PROD-003', brand='真实用户购买的量产车', guidePrice=30.0, produceTime='2026-03-03', carType='氢能源'}
Car{id=19, carNum='MANAGED-999', brand='甩手掌柜测试车', guidePrice=99.9, produceTime='2026-10-10', carType='燃油车'}
```

拼接表名

业务背景：实际开发中，有的表数据量非常庞大，可能会采用分表方式进行存储，比如每天生成一张表，表的名字与日期挂钩，例如：2022年8月1日

生成的表：t_user20220108。2000年1月1日生成的表：t_user20000101。此时前端在进行查询的时候会提交一个具体的日期，比如前端提交的日期

为：2000年1月1日，那么后端就会根据这个日期动态拼接表名为：t_user20000101。有了这个表名之后，将表名拼接到sql语句当中，返回查询结果。

那么大家思考一下，拼接表名到sql语句当中应该使用#{ } 还是 \${ } 呢？

1. 使用#{ }会是这样：select * from 't_car'
 2. 使用\${ }会是这样：select * from t_car
- 修改 CarMapper.java 接口
 - 在接口中增加 selectAllByTableName 方法：

```
package lzt.mybatis.mapper;

import lzt.mybatis.pojo.Car;
import java.util.List;

public interface CarMapper {
    // ... 前面的 selectByCarType 和 selectALL 方法 ...

    /**
     * 根据表名查询所有的Car (演示 ${ } 动态拼接表名)
     * @param tableName 表名
     * @return 汽车集合
     */
    List<Car> selectAllByTableName(String tableName);
}
```

- 修改 CarMapper.xml 映射文件
 - 在 XML 文件中追加查询标签。

```
<select id="selectAllByTableName" resultType="lzt.mybatis.pojo.Car">
    select
        id, car_num as carNum, brand, guide_price as guidePrice, produce_time as produceTime, car_type as carType
    from
        ${tableName}
</select>
```

- 修改 CarMapperTest.java 测试类
 - 追加测试方法

```

@Test
public void testSelectAllByTableName(){
    CarMapper mapper = SqlSessionUtil.openSession().getMapper(CarMapper.class);

    // 动态传入表名 t_car
    List<Car> cars = mapper.selectAllByTableName("t_car");
    cars.forEach(car -> System.out.println(car));

    // 释放资源
    SqlSessionUtil.close(SqlSessionUtil.openSession());
}

```

- 执行结果:

```

D:\java\lzt\lzt\java\src>
2026-03-11 15:00:39.840 [main] DEBUG l.m.m.CarMapper.selectAllByTableName - ==> Preparing: select id, car_num as carNum, brand,
    guide_price as guidePrice, produce_time as produceTime, car_type as carType from t_car
2026-03-11 15:00:39.879 [main] DEBUG l.m.m.CarMapper.selectAllByTableName - ==> Parameters:
2026-03-11 15:00:39.919 [main] DEBUG l.m.m.CarMapper.selectAllByTableName - <==      Total: 27
Car{id=1, carNum='100', brand='宝马520Li', guidePrice=41.0, produceTime='2022-09-01', carType='燃油车'}
Car{id=2, carNum='101', brand='奔驰E300L', guidePrice=54.0, produceTime='2022-08-01', carType='电车'}
Car{id=3, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=4, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=5, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=6, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=7, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=8, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=9, carNum='102', brand='比亚迪汉', guidePrice=30.23, produceTime='2018-09-10', carType='电车'}
Car{id=10, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=12, carNum='104', brand='开发库测试车', guidePrice=10.0, produceTime='2026-01-01', carType='燃油车'}
Car{id=13, carNum='DEV-001', brand='程序员本地自测车', guidePrice=10.0, produceTime='2026-01-01', carType='燃油车'}
Car{id=14, carNum='DEV-001', brand='程序员本地自测车', guidePrice=10.0, produceTime='2026-01-01', carType='燃油车'}

```

批量删除

业务背景：一次删除多条记录。

需求：删除 ID 从 12 到 28 的车辆

- 对应的 SQL 语句

```
DELETE FROM t_car WHERE id IN (12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28);
```

或者 or; 假设删除id 1, 2, 3

```
delete from t_user where id = 1 or id = 2 or id = 3;
```

- 如果使用mybatis处理, 应该使用#{ } 还是 \${ }
 - 使用#{ } : delete from t_user where id in('1,2,3') 执行错误: 1292 - Truncated incorrect DOUBLE value: '1,2,3'
 - 使用\${ } : delete from t_user where id in(1, 2, 3)

假设现在使用in的方式处理, 前端传过来的字符串: 12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28

- 修改 CarMapper.java 接口
 - 追加 deleteBatch 方法

```
package lzt.mybatis.mapper;
```



```
import lzt.mybatis.pojo.Car;
import java.util.List;

public interface CarMapper {
    // ... 前面的方法 ...

    /**
     * 根据id批量删除车辆 (演示 ${} 在 in 语句中的应用)
     * @param ids 逗号分隔的多个id字符串
     * @return 受影响的行数
     */
    int deleteBatch(String ids);
}
```

- 修改 CarMapper.xml 映射文件
 - 追加 `<delete>` 标签, 严格使用 `${}`:

```
<delete id="deleteBatch">
    delete from t_car where id in(${ids})
</delete>
```

- 修改 CarMapperTest.java 测试类

```
@Test
public void testDeleteBatch(){
    CarMapper mapper = SqlSessionUtil.openSession().getMapper(CarMapper.class);

    // 传入从 12 到 28 逗号分隔的字符串
    String idsToDelete = "12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28";
    int count = mapper.deleteBatch(idsToDelete);

    System.out.println("删除了几条记录: " + count);

    // 执行了增删改操作, 一定要记得 commit 提交事务!
    SqlSessionUtil.openSession().commit();
    // 释放资源
    SqlSessionUtil.close(SqlSessionUtil.openSession());
}
```

执行结果:

```
2026-03-11 15:13:47.936 [main] DEBUG l.m.mapper.CarMapper.deleteBatch - ==> Preparing: delete from t_car where id in(12,13,14,15,
16,17,18,19,20,21,22,23,24,25,26,27,28)
2026-03-11 15:13:47.974 [main] DEBUG l.m.mapper.CarMapper.deleteBatch - ==> Parameters:
2026-03-11 15:13:48.015 [main] DEBUG l.m.mapper.CarMapper.deleteBatch - <== Updates: 17
删除了几条记录: 17
```

测试前数据:

对象

t_act @powernode (本地MySQL) - 表

t_car @mybatis_demo (本地MySQL) -...

t_car @mybatis_demo (本地MySQL) -...

表配置文件

开始事务

单元格编辑器

筛选 & 排序

列

数据分析

导入

导出

数据生成

创建 BI 工作区

id # bigint	car_num varchar(255)	brand varchar(255)	guide_price decimal(10,2)	produce_time char(10)	car_type varchar(255)
5	102	丰田未来	40.30	2014-10-05	氢能源
6	102	丰田未来	40.30	2014-10-05	氢能源
7	102	丰田未来	40.30	2014-10-05	氢能源
8	102	丰田未来	40.30	2014-10-05	氢能源
9	102	比亚迪汉	30.23	2018-09-10	电车
10	102	丰田未来	40.30	2014-10-05	氢能源
12	104	开发库测试车	10.00	2026-01-01	燃油车
13	DEV-001	程序员本地自测车	10.00	2026-01-01	燃油车
14	DEV-001	程序员本地自测车	10.00	2026-01-01	燃油车
15	DEV-001	程序员本地自测车	10.00	2026-01-01	燃油车
16	DEV-001	程序员本地自测车	10.00	2026-01-01	燃油车
17	TEST-002	测试组找Bug专用车	20.00	2026-02-02	电车
18	PROD-003	真实用户购买的量	30.00	2026-03-03	氢能源
19	MANAGED-999	甩手掌柜测试车	99.90	2026-10-10	燃油车
20	MANAGED-999	MANAGED测试车	99.90	2026-10-10	燃油车
21	DEV-001	程序员本地自测车	10.00	2026-01-01	燃油车
22	TEST-002	测试组找Bug专用车	20.00	2026-02-02	电车
23	PROD-003	真实用户购买的量	30.00	2026-03-03	氢能源
24	MANAGED-999	MANAGED测试车	99.90	2026-10-10	燃油车
25	UNPOOLED-01	非连接池测试车1	10.00	2026-04-01	燃油车
26	UNPOOLED-02	非连接池测试车2	20.00	2026-04-02	燃油车
27	UNPOOLED-01	非连接池测试车1	10.00	2026-04-01	燃油车
28	UNPOOLED-02	非连接池测试车2	20.00	2026-04-02	燃油车

文本

十六进制

图像

网页

28

+ - ✓ ✕ ↺ ■

1

⏪ ⏩ ⚙

📊 📄

测试后的数据：

对象

t_car @mybatis_demo (本地MySQL) -...

表配置文件

开始事务

单元格编辑器

筛选 & 排序

列

数据分析

导入

导出

数据生成

创建 BI 工作区

id # bigint	car_num varchar(255)	brand varchar(255)	guide_price decimal(10,2)	produce_time char(10)	car_type varchar(255)
1	100	宝马520Li	41.00	2022-09-01	燃油车
2	101	奔驰E300L	54.00	2022-08-01	电车
3	102	丰田未来	40.30	2014-10-05	氢能源
4	102	丰田未来	40.30	2014-10-05	氢能源
5	102	丰田未来	40.30	2014-10-05	氢能源
6	102	丰田未来	40.30	2014-10-05	氢能源
7	102	丰田未来	40.30	2014-10-05	氢能源
8	102	丰田未来	40.30	2014-10-05	氢能源
9	102	比亚迪汉	30.23	2018-09-10	电车
10	102	丰田未来	40.30	2014-10-05	氢能源

模糊查询

需求：查询丰田系列的汽车。【只要品牌brand中含有丰田两个字的都查询出来。】

使用\${}

- 修改 CarMapper.java 接口
 - 追加 selectLikeByBrand 方法

```
package lzt.mybatis.mapper;
```

```
import lzt.mybatis.pojo.Car;
import java.util.List;

public interface CarMapper {
    // ... 前面写过的方法保持不变 ...

    /**
     * 根据品牌进行模糊查询
     * @param brand 品牌关键字
     * @return 汽车集合
     */
    List<Car> selectLikeByBrand(String brand);
}
```

- 修改 CarMapper.xml 映射文件
 - 追加 `<select>` 标签:



```
<select id="selectLikeByBrand" resultType="lzt.mybatis.pojo.Car">
    select
        id, car_num as carNum, brand, guide_price as guidePrice, produce_time as produceTime, car_type as carType
    from
        t_car
    where
        brand like '%${brand}%'
</select>
```

- 修改 CarMapperTest.java 测试类



```
@Test
public void testSelectLikeByBrand(){
    CarMapper mapper = SqlSessionUtil.openSession().getMapper(CarMapper.class);

    // 传入关键字进行模糊查询
    List<Car> cars = mapper.selectLikeByBrand("丰田");
    cars.forEach(car -> System.out.println(car));

    // 记得释放资源
    SqlSessionUtil.close(SqlSessionUtil.openSession());
}
```

执行代码:

```
0. java -cp bin\java.exe ...
```

```
2026-03-11 15:23:01.825 [main] DEBUG l.m.m.CarMapper.selectLikeByBrand - ==> Preparing: select id, car_num as carNum, brand,
    guide_price as guidePrice, produce_time as produceTime, car_type as carType from t_car where brand like '%丰田%'
2026-03-11 15:23:01.856 [main] DEBUG l.m.m.CarMapper.selectLikeByBrand - ==> Parameters:
2026-03-11 15:23:01.940 [main] DEBUG l.m.m.CarMapper.selectLikeByBrand - <== Total: 7
Car{id=3, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=4, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=5, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=6, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=7, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=8, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=10, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
```

使用#{}

- 第一种：concat函数
 - concat 是 MySQL 数据库自带的字符串拼接函数。
 - 它的作用就是把括号里的多个字符串无缝拼接成一个完整的字符串。
 - 追加 CarMapper.java 接口

```
/**
 * 根据品牌模糊查询 (使用 concat 函数 + #{})
 * @param brand 品牌
 * @return 汽车集合
 */
List<Car> selectLikeByBrandByConcat(String brand);
```

追加 CarMapper.xml 映射文件

```
<select id="selectLikeByBrandByConcat" resultType="lzt.mybatis.pojo.Car">
    select
        id, car_num as carNum, brand, guide_price as guidePrice, produce_time as produceTime, car_type as carType
    from
        t_car
    where
        brand like concat('%', #{brand}, '%')
</select>
```

- 追加 CarMapperTest.java 测试类

```
@Test
public void testSelectLikeByBrandByConcat(){
    CarMapper mapper = SqlSessionUtil.openSession().getMapper(CarMapper.class);

    List<Car> cars = mapper.selectLikeByBrandByConcat("丰田");
    cars.forEach(car -> System.out.println(car));

    SqlSessionUtil.close(SqlSessionUtil.openSession());
}
```

```
D:\Java\bin\java.exe ...
2026-03-11 15:29:41.910 [main] DEBUG l.m.m.C.selectLikeByBrandByConcat - ==> Preparing: select id, car_num as carNum, brand,
    guide_price as guidePrice, produce_time as produceTime, car_type as carType from t_car where brand like concat('%', ?, '%')
2026-03-11 15:29:41.945 [main] DEBUG l.m.m.C.selectLikeByBrandByConcat - ==> Parameters: 丰田(String)
2026-03-11 15:29:42.015 [main] DEBUG l.m.m.C.selectLikeByBrandByConcat - <==          Total: 7
Car{id=3, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=4, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=5, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=6, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=7, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=8, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=10, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
```

- 第二种：双引号方式
 - 追加 CarMapper.java 接口

```
/**
 * 根据品牌模糊查询 (使用 双引号拼接 + #{})
 * @param brand 品牌
 * @return 汽车集合
 */
List<Car> selectLikeByBrandByQuote(String brand);
```

- 追加 CarMapper.xml 映射文件

```
<select id="selectLikeByBrandByQuote" resultType="Lzt.mybatis.pojo.Car">
    select
        id, car_num as carNum, brand, guide_price as guidePrice, produce_time as produceTime, car_type as carType
    from
        t_car
    where
        brand like "%#{brand}%"
</select>
```

- 追加 CarMapperTest.java 测试类

```
@Test
public void testSelectLikeByBrandByQuote(){
    CarMapper mapper = SqlSessionUtil.openSession().getMapper(CarMapper.class);

    List<Car> cars = mapper.selectLikeByBrandByQuote("丰田");
    cars.forEach(car -> System.out.println(car));

    SqlSessionUtil.close(SqlSessionUtil.openSession());
}
```

```
D:\Java21\bin\java.exe ...
2026-03-12 10:36:31.500 [main] DEBUG l.m.m.C.selectLikeByBrandByQuote - ==> Preparing: select id, car_num as carNum, brand,
    guide_price as guidePrice, produce_time as produceTime, car_type as carType from t_car where brand like "%?%"
2026-03-12 10:36:31.536 [main] DEBUG l.m.m.C.selectLikeByBrandByQuote - ==> Parameters: 丰田(String)
2026-03-12 10:36:31.603 [main] DEBUG l.m.m.C.selectLikeByBrandByQuote - <== Total: 7
Car{id=3, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=4, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=5, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=6, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=7, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=8, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=10, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
```

typeAliases

我们来观察一下CarMapper.xml中的配置信息：

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper namespace="Lzt.mybatis.mapper.CarMapper">

    <select id="selectByCarType" resultType="Lzt.mybatis.pojo.Car">
        select
```

```

        id, car_num as carNum, brand, guide_price as guidePrice, produce_time as produceTime, car_type as carType
    from
        t_car
    where
        car_type = '${carType}'
</select>

<select id="selectAll" resultType="lzt.mybatis.pojo.Car">
    select
        id, car_num as carNum, brand, guide_price as guidePrice, produce_time as produceTime, car_type as carType
    from
        t_car
    order by carNum ${ascOrDesc}
</select>

```

resultType属性用来指定查询结果集的封装类型，这个名字太长，可以起别名吗？可以。

在mybatis-config.xml文件中使用typeAliases标签来起别名，包括两种方式：

第一种方式：typeAlias

```

<typeAliases>
    <typeAlias type="lzt.mybatis.pojo.Car" alias="Car"/>
</typeAliases>

```

- 首先要注意typeAliases标签的放置位置，如果不清楚的话，可以看看错误提示信息。
- typeAliases标签中的typeAlias可以写多个。
- typeAlias：
 - type属性：指定给哪个类起别名
 - alias属性：别名。
 - alias属性不是必须的，如果缺省的话，type属性指定的类型名的简类名作为别名。
 - alias是大小写不敏感的。也就是说假设alias="Car"，再用的时候，可以CAR，也可以car，也可以Car，都行。
- 在CarMapper.xml 文件使用
 - resultType 使用别名 Car

```

<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper namespace="lzt.mybatis.mapper.CarMapper">

    <select id="selectByCarType" resultType="Car">
        select
            id, car_num as carNum, brand, guide_price as guidePrice, produce_time as produceTime, car_type as carType
        from
            t_car
        where
            car_type = '${carType}'
    </select>

    <select id="selectAll" resultType="Car">
        select
            id, car_num as carNum, brand, guide_price as guidePrice, produce_time as produceTime, car_type as carType
        from
            t_car
        order by carNum ${ascOrDesc}
    </select>

```

```
</select>

</mapper>
```

第二种方式：package

如果一个包下的类太多，每个类都要起别名，会导致typeAlias标签配置较多，所以mybatis用提供package的配置方式，只需要指定包名，该包下的所有类都自动起别名，别名就是简类名。并且别名不区分大小写。

使用场景：如果 pojo 包下有 50 个实体类（Car、User、Order、Product...），写 50 个 `<typeAlias>` 标签会很麻烦、浪费时间；因此，只需要告诉 MyBatis 你的实体类都在哪个包下，MyBatis 会自动去扫描这个包里的所有类，并**把它们的首字母变小写作为默认别名**。

```
● ● ●
<typeAliases>
  <package name="lzt.mybatis.pojo"/>
</typeAliases>
```

package也可以配置多个的。

使用了包扫描后，`CarMapper.xml` 中的 `resultType="Car"` 或 `resultType="car"` 依然完美生效，不需要任何修改！

mappers

它是 MyBatis 核心配置文件（mybatis-config.xml）和 SQL 映射文件（CarMapper.xml）之间的“桥梁”。

SQL映射文件的配置方式包括四种：

- resource：从类路径中加载
- url：从指定的全限定资源路径中加载
- class：使用映射器接口实现类的完全限定类名
- package：将包内的映射器接口实现全部注册为映射器

resource

这种方式是从类路径中加载配置文件，所以这种方式要求SQL映射文件必须放在resources目录下或其子目录下。它的路径是以 `/` 分隔的

缺点：每一个 XML 文件都需要单独配置一行，文件多了难以维护。

语法：

```
● ● ●
<mapper resource="相对类路径下的 XML 文件位置"/>
```

例子：

```
● ● ●
<mappers>
  <mapper resource="org/mybatis/builder/AuthorMapper.xml"/>
  <mapper resource="org/mybatis/builder/BlogMapper.xml"/>
  <mapper resource="org/mybatis/builder/PostMapper.xml"/>
  <mapper resource="CarMapper.xml"/>
</mappers>
```

url

这种方式显然使用了绝对路径的方式，这种配置对SQL映射文件存放的位置没有要求，随意。

```
● ● ●
<mappers>
  <mapper url="file:///var/mappers/AuthorMapper.xml"/>
  <mapper url="file:///var/mappers/BlogMapper.xml"/>
  <mapper url="file:///var/mappers/PostMapper.xml"/>
</mappers>
```

class

如果使用这种方式必须满足以下条件：

- SQL映射文件和mapper接口放在同一个目录下。
- SQL映射文件的名字也必须和mapper接口名一致。

它不再指向 XML 文件，而是直接指向你的 Java 接口

语法：

```
● ● ●
<mapper class="Mapper接口的全限定类名"/>
```

```
● ● ●
<!-- 使用映射器接口实现类的完全限定类名 -->
<mappers>
  <mapper class="org.mybatis.builder.AuthorMapper"/>
  <mapper class="org.mybatis.builder.BlogMapper"/>
  <mapper class="org.mybatis.builder.PostMapper"/>
</mappers>
```

将CarMapper.xml文件移动到和mapper接口同一个目录下：

- 在resources目录下新建：com/powernode/mybatis/mapper【这里千万要注意：**不能这样新建 com.powernode.mybatis.dao**】
- 将CarMapper.xml文件移动到mapper目录下
- 修改mybatis-config.xml文件

```
● ● ●
<mappers>
  <mapper class="com.powernode.mybatis.mapper.CarMapper"/>
</mappers>
```

package

如果class较多，可以使用这种package的方式，但前提条件和上一种方式一样。

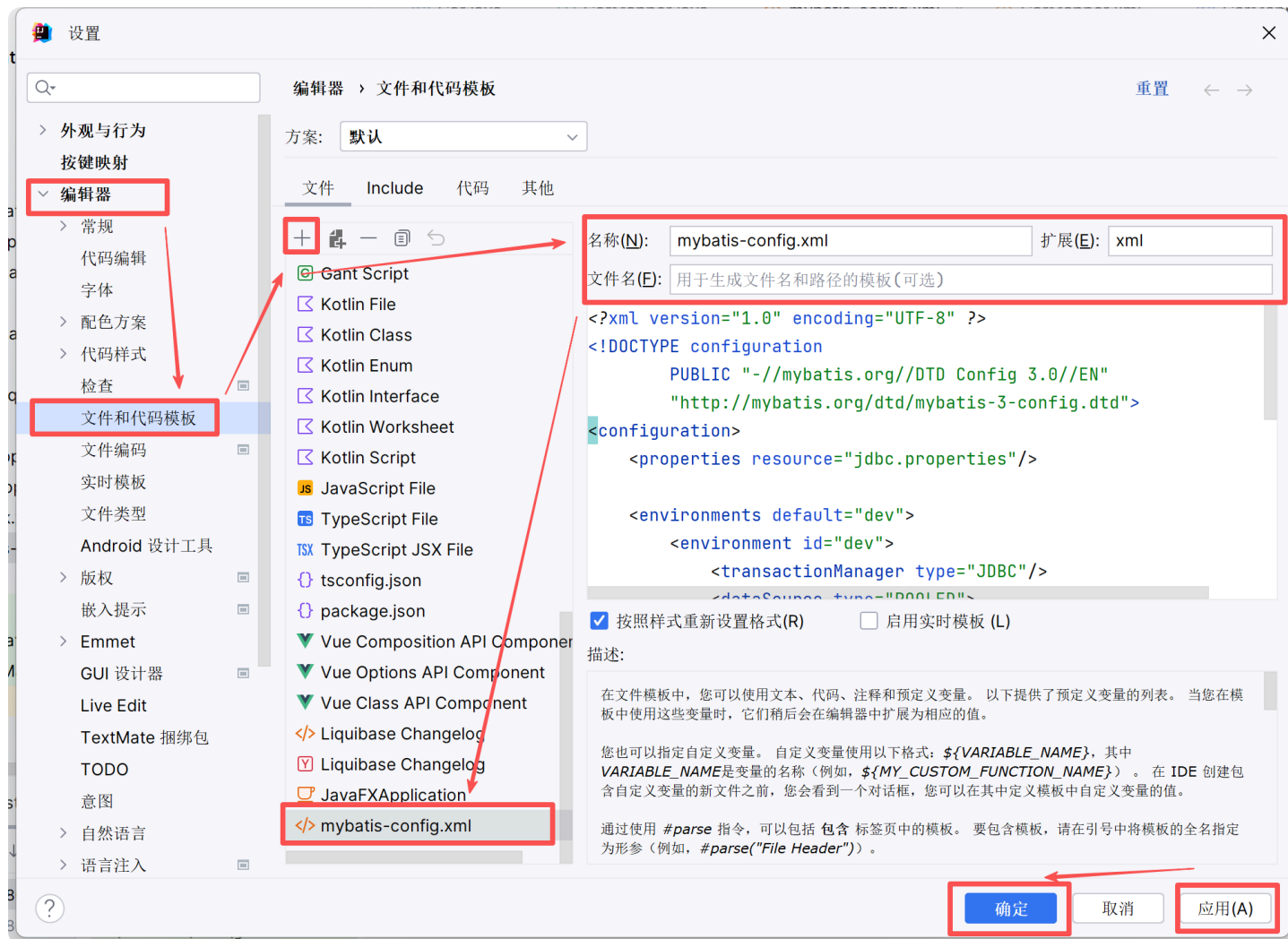
语法：

```
● ● ●
<package name="接口所在的包名"/>
```

```
<!-- 将包内的映射器接口实现全部注册为映射器 -->
<mappers>
  <package name="com.powernode.mybatis.mapper"/>
</mappers>
```

idea配置文件模板

mybatis-config.xml和SqlMapper.xml文件可以在IDEA中提前创建好模板，以后通过模板创建配置文件。



插入数据时获取自动生成的主键

前提是：主键是自动生成的。
业务背景：一个用户有多个角色。

t_user用户表	
id	name
1	小明
2	小花

t_role角色表		
id	name	user_id
1	项目经理	1
2	技术总监	1
3	Java软件工程师	1
4	CEO	2
5	组长	2
6	测试员	2

老杜

插入一条新的记录之后，自动生成了主键，而这个主键需要在其他表中使用时。

插入一个用户数据的同时需要给该用户分配角色：需要将生成的用户的id插入到角色表的user_id字段上。

第一种方式：可以先插入用户数据，再写一条查询语句获取id，然后再插入user_id字段。【比较麻烦】

第二种方式：mybatis提供了一种方式更加便捷。

- 修改 CarMapper.java 接口

```
package lzt.mybatis.mapper;

import lzt.mybatis.pojo.Car;
import java.util.List;

public interface CarMapper {
    // ... 前面的查询、删除方法保持不变 ...

    /**
     * 新增Car，并获取自动生成的主键
     * @param car 汽车对象
     * @return 受影响的行数（注意：返回的不是主键ID！）
     */
    int insertUseGeneratedKeys(Car car);
}
```

- 修改 CarMapper.xml 映射文件

```
<insert id="insertUseGeneratedKeys" useGeneratedKeys="true" keyProperty="id">
    insert into t_car(id, car_num, brand, guide_price, produce_time, car_type)
    values(null, #{carNum}, #{brand}, #{guidePrice}, #{produceTime}, #{carType})
</insert>
```

- 修改 CarMapperTest.java 测试类

```
@Test
public void testInsertUseGeneratedKeys(){
    CarMapper mapper = SqlSessionUtil.openSession().getMapper(CarMapper.class);

    // 1. 创建一个全新的 Car 对象，此时它的 id 是 null
    Car car = new Car();
```

```

car.setCarNum("5262");
car.setBrand("BYD汉");
car.setGuidePrice(30.3);
car.setProduceTime("2020-10-11");
car.setCarType("新能源");

System.out.println("插入前, car的id是: " + car.getId()); // 这里打印肯定是 null

// 2. 执行插入操作
mapper.insertUseGeneratedKeys(car);

// 3. 提交事务 (增删改必写!)
SqlSessionUtil.openSession().commit();

// 4. 见证奇迹的时刻: MyBatis 已经悄悄把数据库生成的新 ID 塞进了这个 car 对象里!
System.out.println("插入后, 自动生成的主键id是: " + car.getId());

// 5. 释放资源
SqlSessionUtil.close(SqlSessionUtil.openSession());
}

```

测试前的数据

表配置文件 开始事务 单元格编辑器 筛选 & 排序 列 数据分析 导入 导出 数据生						
id # bigint	car_num varchar(255)	brand varchar(255)	guide_price # decimal(10,2)	produce_time char(10)	car_type varchar(255)	
1	100	宝马520Li	41.00	2022-09-01	燃油车	
2	101	奔驰E300L	54.00	2022-08-01	电车	
3	102	丰田未来	40.30	2014-10-05	氢能源	
4	102	丰田未来	40.30	2014-10-05	氢能源	
5	102	丰田未来	40.30	2014-10-05	氢能源	
6	102	丰田未来	40.30	2014-10-05	氢能源	
7	102	丰田未来	40.30	2014-10-05	氢能源	
8	102	丰田未来	40.30	2014-10-05	氢能源	
9	102	比亚迪汉	30.23	2018-09-10	电车	
10	102	丰田未来	40.30	2014-10-05	氢能源	

D:\Java21\bin\java.exe ...

插入前, car的id是: null

2026-03-12 11:40:41.415 [main] DEBUG l.m.m.C.insertUseGeneratedKeys - ==> Preparing: insert into t_car(id, car_num, brand, guide_price, produce_time, car_type) values(null, ?, ?, ?, ?, ?)

2026-03-12 11:40:41.458 [main] DEBUG l.m.m.C.insertUseGeneratedKeys - ==> Parameters: 5262(String), BYD汉(String), 30.3(Double), 2020-10-11(String), 新能源(String)

2026-03-12 11:40:41.467 [main] DEBUG l.m.m.C.insertUseGeneratedKeys - <== Updates: 1

插入后, 自动生成的主键id是: 29

表配置文件 开始事务 单元格编辑器 筛选 & 排序 列 数据分析 导入 导出 数据						
id # bigint	car_num varchar(255)	brand varchar(255)	guide_price # decimal(10,2)	produce_time char(10)	car_type varchar(255)	
1	100	宝马520Li	41.00	2022-09-01	燃油车	
2	101	奔驰E300L	54.00	2022-08-01	电车	
3	102	丰田未来	40.30	2014-10-05	氢能源	
4	102	丰田未来	40.30	2014-10-05	氢能源	
5	102	丰田未来	40.30	2014-10-05	氢能源	
6	102	丰田未来	40.30	2014-10-05	氢能源	
7	102	丰田未来	40.30	2014-10-05	氢能源	
8	102	丰田未来	40.30	2014-10-05	氢能源	
9	102	比亚迪汉	30.23	2018-09-10	电车	
10	102	丰田未来	40.30	2014-10-05	氢能源	
29	5262	BYD汉	30.30	2020-10-11	新能源	

MyBatis参数处理

准备工作

项目名：my_mybatis-07-param

数据库：powernode

数据表：t_student

- 准备数据库和表结构的 SQL 脚本

-- 1. 创建数据库并使用

```
CREATE DATABASE IF NOT EXISTS powernode DEFAULT CHARSET utf8mb4;
USE powernode;
```

-- 2. 删除已存在的表（如果存在）

```
DROP TABLE IF EXISTS t_student;
```

-- 3. 创建 t_student 表

```
CREATE TABLE t_student (
  id bigint NOT NULL AUTO_INCREMENT COMMENT '主键',
  name varchar(255) DEFAULT NULL COMMENT '姓名',
  age int DEFAULT NULL COMMENT '年龄',
  height double DEFAULT NULL COMMENT '身高',
  birth date DEFAULT NULL COMMENT '生日',
  sex char(1) DEFAULT NULL COMMENT '性别',
  PRIMARY KEY (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

-- 4. 插入测试数据

```
INSERT INTO t_student (id, name, age, height, birth, sex) VALUES (2, '张三', 20, 1.81, '2022-08-16', '男');
INSERT INTO t_student (id, name, age, height, birth, sex) VALUES (3, '张三', 20, 1.81, '2022-08-16', '女');
```

```
COMMIT;
```

对象 t_student @powernode (本地MySQL)...						
表配置文件 开始事务 单元格编辑器 筛选 & 排序 列 数据分析 导						
id # bigint	name A B C varchar(255)	age # int	height # double	birth 🕒 date	sex A B C char(1)	
2	张三	20	1.81	2022-08-16	男	
3	张三	20	1.81	2022-08-16	女	

对象 t_student @powernode (本地MySQL)... t_student @powernode (本地MySQL)...								
保存 添加字段 插入字段 删除字段 主键 上移 下移								
字段	索引	外键	检查	触发器	选项	注释	SQL 预览	
名称	类型		长度	小数点	不是 null	虚拟	键	注释
id	bigint				☒	☐	🔑 1	主键
name	varchar		255		☐	☐		姓名
age	int				☐	☐		年龄
height	double				☐	☐		身高
birth	date				☐	☐		生日
sex	char		1		☐	☐		性别

- 创建项目：

mybatis-07-param

版本控制

当前文件

项目

mybatis-07-param

idea

compiler.xml

encodings.xml

jarRepositories.xml

misc.xml

workspace.xml

src

main

java

lzt.mybatis

resources

test

java

lzt.mybatis

gitignore

pom.xml

外部库

临时文件和控制台

pom.xml (mybatis-07-param)

<?xml version="1.0" encoding="UTF-8"?>

<project xmlns="http://maven.apache.org/POM/4.0.0"

xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd"

<modelVersion>4.0.0</modelVersion>

<groupId>lzt.mybatis</groupId>

<artifactId>mybatis-07-param</artifactId>

<version>1.0-SNAPSHOT</version>

<properties>

<maven.compiler.source>21</maven.compiler.source>

<maven.compiler.target>21</maven.compiler.target>

<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>

</properties>

</project>

配置相关基础文件

- POJO 类

```

package lzt.mybatis.pojo;
import java.util.Date;

/**
 * 学生类（实体类/POJO）
 * 用来封装数据库 t_student 表中的数据
 */
public class Student {
    private Long id;
    private String name;
    private Integer age;

```

```
private Double height;
private Character sex;
private Date birth;

public Student() {
}

public Student(Long id, String name, Integer age, Double height, Character sex, Date birth) {
    this.id = id;
    this.name = name;
    this.age = age;
    this.height = height;
    this.sex = sex;
    this.birth = birth;
}

public Long getId() {
    return id;
}

public void setId(Long id) {
    this.id = id;
}

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

public Integer getAge() {
    return age;
}

public void setAge(Integer age) {
    this.age = age;
}

public Double getHeight() {
    return height;
}

public void setHeight(Double height) {
    this.height = height;
}

public Character getSex() {
    return sex;
}

public void setSex(Character sex) {
    this.sex = sex;
}

public Date getBirth() {
    return birth;
}

public void setBirth(Date birth) {
    this.birth = birth;
}
```

@Override

```

public String toString() {
    return "Student{" +
        "id=" + id +
        ", name='" + name + '\'' +
        ", age=" + age +
        ", height=" + height +
        ", sex=" + sex +
        ", birth=" + birth +
        '}';
}
}

```

- pom.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>lzt.mybatis</groupId>
    <artifactId>my_mybatis-07-param</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.source>21</maven.compiler.source>
        <maven.compiler.target>21</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.mybatis</groupId>
            <artifactId>mybatis</artifactId>
            <version>3.5.19</version>
        </dependency>
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <version>8.4.0</version>
        </dependency>
        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>4.13.2</version>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>ch.qos.logback</groupId>
            <artifactId>logback-classic</artifactId>
            <version>1.5.6</version>
        </dependency>
    </dependencies>

    <build>
        <resources>
            <resource>
                <directory>src/main/java</directory>
                <includes>
                    <include>/**/*.xml</include>
                    <include>/**/*.properties</include>
                </includes>
            </resource>

```

```

        <resource>
            <directory>src/main/resources</directory>
            <includes>
                <include>**/*.xml</include>
                <include>**/*.properties</include>
            </includes>
        </resource>
    </resources>
</build>
</project>

```

- jdbc.properties

```

jdbc.driver=com.mysql.cj.jdbc.Driver
jdbc.url=jdbc:mysql://localhost:3306/powernode?useUnicode=true&characterEncoding=utf8&serverTimezone=Asia/Shanghai
jdbc.username=root
jdbc.password=20060329

```

- mybatis-config.xml

```

<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE configuration
    PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-config.dtd">
<configuration>

    <properties resource="jdbc.properties"/>

    <settings>
        <setting name="mapUnderscoreToCamelCase" value="true"/>
        <setting name="LogImpl" value="SLF4J"/>
    </settings>

    <typeAliases>
        <package name="lzt.mybatis.pojo"/>
    </typeAliases>

    <environments default="development">
        <environment id="development">
            <transactionManager type="JDBC"/>
            <dataSource type="POOLED">
                <property name="driver" value="${jdbc.driver}"/>
                <property name="url" value="${jdbc.url}"/>
                <property name="username" value="${jdbc.username}"/>
                <property name="password" value="${jdbc.password}"/>
            </dataSource>
        </environment>
    </environments>

    <mappers>
        <mapper resource="lzt/mybatis/mapper/StudentMapper.xml"/>
    </mappers>

</configuration>

```

- logback.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<configuration>
    <appender name="STDOUT" class="ch.qos.logback.core.ConsoleAppender">

```

```

        <encoder class="ch.qos.logback.classic.encoder.PatternLayoutEncoder">
            <pattern>[%thread] %-5level %logger{50} - %msg%n</pattern>
        </encoder>
    </appender>

    <logger name="Lzt.mybatis.mapper" level="DEBUG"/>

    <root level="INFO">
        <appender-ref ref="STDOUT" />
    </root>
</configuration>

```

- SqlSessionUtil.java

```

package lzt.mybatis.utils;

import org.apache.ibatis.io.Resources;
import org.apache.ibatis.session.SqlSession;
import org.apache.ibatis.session.SqlSessionFactory;
import org.apache.ibatis.session.SqlSessionFactoryBuilder;

import java.io.IOException;

/**
 * MyBatis 的 SqlSession 工具类
 */
public class SqlSessionUtil {

    // 工具类的构造方法一般私有化，防止别人 new 对象
    private SqlSessionUtil() {}

    private static SqlSessionFactory sqlSessionFactory;

    // 全局唯一的一个 ThreadLocal 对象
    private static ThreadLocal<SqlSession> local = new ThreadLocal<>();

    // 静态代码块，类加载时只执行一次，保证全局只有一个 SqlSessionFactory
    static {
        try {
            sqlSessionFactory = new SqlSessionFactoryBuilder().build(Resources.getResourceAsStream("mybatis-
config.xml"));
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    /**
     * 获取 SqlSession 对象
     * @return SqlSession
     */
    public static SqlSession openSession() {
        SqlSession sqlSession = local.get();
        if (sqlSession == null) {
            // 如果当前线程没有绑定 SqlSession，就新建一个，并绑定到当前线程
            sqlSession = sqlSessionFactory.openSession();
            local.set(sqlSession);
        }
        return sqlSession;
    }

    /**
     * 关闭 SqlSession 对象，并从当前线程中移除
     * 一定要移除！不然在线程池环境下会导致内存泄漏或者拿到别人的连接！

```



```

    */
    public static void close(SqlSession sqlSession) {
        if (sqlSession != null) {
            sqlSession.close();
            // 务必解绑
            local.remove();
        }
    }
}

```

单个简单类型参数

在 MyBatis 的底层源码中，对简单类型有明确的界定。主要包括以下几种：

- 基本数据类型：byte, short, int, long, float, double, char, boolean
- 包装类：Byte, Short, Integer, Long, Float, Double, Character, Boolean
- 字符串类型：java.lang.String
- 日期/时间类型：java.util.Date, java.sql.Date, java.sql.Time, java.sql.Timestamp
- 其他：java.math.BigDecimal, java.math.BigInteger 等

只要是 Java 自带的、那些最基础的、用来表示单一值的类型，MyBatis 基本都把它们当做“简单类型”来处理。

需求：根据name查、根据id查、根据birth查、根据sex查

- Mapper 接口
 - StudentMapper.java

```

package lzt.mybatis.mapper;
import lzt.mybatis.pojo.Student;
import java.util.Date;
import java.util.List;

/**
 * 学生数据Sql映射器
 * 演示：单个简单类型参数的多种情况
 */
public interface StudentMapper {
    /**
     * 根据name 查询 (String类型)
     * @param name
     * @return
     */
    List<Student> selectByName(String name);

    /**
     * 根据id 查询 (Long类型包装类)
     * @param id
     * @return
     */
    Student selectById(Long id);

    /**
     * 根据birth 查询 (Date类型)
     * @param birth
     * @return
     */
    List<Student> selectByBirth(Date birth);

    /**

```

```

    * 根据sex查询 (Character类型)
    * @param sex
    * @return
    */
    List<Student> selectBySex(Character sex);
}

```

- Mapper 映射文件
 - StudentMapper.xml

```

<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper namespace="lzt.mybatis.mapper.StudentMapper">

    <select id="selectByName" resultType="student">
        select * from t_student where name = #{name}
    </select>

    <select id="selectById" resultType="student">
        select * from t_student where id = #{id}
    </select>

    <select id="selectByBirth" resultType="student">
        select * from t_student where birth = #{birth}
    </select>

    <select id="selectBySex" resultType="student">
        select * from t_student where sex = #{sex}
    </select>

</mapper>

```

- 测试类
 - StudentMapperTest.java

```

package lzt.mybatis.test;

import lzt.mybatis.mapper.StudentMapper;
import lzt.mybatis.pojo.Student;
import lzt.mybatis.utils.SqlSessionUtil;
import org.junit.Test;

import java.text.ParseException;
import java.text.SimpleDateFormat;
import java.util.Date;
import java.util.List;

public class StudentMapperTest {

    // 获取 Mapper 接口的代理对象
    StudentMapper mapper = SqlSessionUtil.openSession().getMapper(StudentMapper.class);

    @Test
    public void testSelectByName(){
        List<Student> students = mapper.selectByName("张三");
        students.forEach(student -> System.out.println(student));
    }
}

```

```

@Test
public void testSelectById(){
    Student student = mapper.selectById(2L);
    System.out.println(student);
}

@Test
public void testSelectByBirth(){
    try {
        // String 转换成 Date
        Date birth = new SimpleDateFormat("yyyy-MM-dd").parse("2022-08-16");
        List<Student> students = mapper.selectByBirth(birth);
        students.forEach(student -> System.out.println(student));
    } catch (ParseException e) {
        throw new RuntimeException(e);
    }
}

@Test
public void testSelectBySex(){
    List<Student> students = mapper.selectBySex('男');
    students.forEach(student -> System.out.println(student));
}
}

```

通过测试得知，简单类型对于mybatis来说都是可以自动类型识别的：

```

D:\Java21\bin\java.exe ...
[main] DEBUG lzt.mybatis.mapper.StudentMapper.selectByName - ==> Preparing: select * from t_student where name = ?
[main] DEBUG lzt.mybatis.mapper.StudentMapper.selectByName - ==> Parameters: 张三(String)
[main] DEBUG lzt.mybatis.mapper.StudentMapper.selectByName - <==          Total: 2
Student{id=2, name='张三', age=20, height=1.81, sex=男, birth=Tue Aug 16 00:00:00 CST 2022}
Student{id=3, name='张三', age=20, height=1.81, sex=女, birth=Tue Aug 16 00:00:00 CST 2022}
[main] DEBUG lzt.mybatis.mapper.StudentMapper.selectByBirth - ==> Preparing: select * from t_student where birth = ?
[main] DEBUG lzt.mybatis.mapper.StudentMapper.selectByBirth - ==> Parameters: 2022-08-16 00:00:00.0(Timestamp)
[main] DEBUG lzt.mybatis.mapper.StudentMapper.selectByBirth - <==          Total: 2
Student{id=2, name='张三', age=20, height=1.81, sex=男, birth=Tue Aug 16 00:00:00 CST 2022}
Student{id=3, name='张三', age=20, height=1.81, sex=女, birth=Tue Aug 16 00:00:00 CST 2022}
[main] DEBUG lzt.mybatis.mapper.StudentMapper.selectBySex - ==> Preparing: select * from t_student where sex = ?
[main] DEBUG lzt.mybatis.mapper.StudentMapper.selectBySex - ==> Parameters: 男(String)
[main] DEBUG lzt.mybatis.mapper.StudentMapper.selectBySex - <==          Total: 1
Student{id=2, name='张三', age=20, height=1.81, sex=男, birth=Tue Aug 16 00:00:00 CST 2022}
[main] DEBUG lzt.mybatis.mapper.StudentMapper.selectById - ==> Preparing: select * from t_student where id = ?
[main] DEBUG lzt.mybatis.mapper.StudentMapper.selectById - ==> Parameters: 2(Long)
[main] DEBUG lzt.mybatis.mapper.StudentMapper.selectById - <==          Total: 1
Student{id=2, name='张三', age=20, height=1.81, sex=男, birth=Tue Aug 16 00:00:00 CST 2022}

```

也就是说对于mybatis来说，它是可以自动推断出ps.setXxx()方法的。ps.setString()还是ps.setInt()。它可以自动推断。

其实SQL映射文件中的配置比较完整的写法是：

```

<select id="selectByName" resultType="student" parameterType="java.lang.String">
    select * from t_student where name = #{name, javaType=String, jdbcType=VARCHAR}
</select>

```

其中sql语句中的javaType, jdbcType, 以及select标签中的parameterType属性，都是用来帮助mybatis进行类型确定的。不过这些配置多数是可以省略的。因为mybatis它有强大的自动类型推断机制。

- javaType：可以省略
- jdbcType：可以省略

- parameterType：可以省略

如果参数只有一个的话，#{ } 里面的内容就随便写了。对于 \${ } 来说，注意加单引号。

Map参数

需求：根据name和age查询

- Mapper 接口

```
package lzt.mybatis.mapper;

import lzt.mybatis.pojo.Student;

import java.util.Date;
import java.util.List;
import java.util.Map; // 导包

/**
 * 学生数据SQL映射器
 */
public interface StudentMapper {

    // ... （前面写的单个参数的方法保留，这里省略不写了）...

    /**
     * 根据name和age查询（演示 Map 传参）
     * @param paramMap 封装了查询条件的 Map 集合
     * @return 返回查询到的学生列表
     */
    List<Student> selectByParamMap(Map<String, Object> paramMap);
}
```

- Mapper 映射文件

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper namespace="lzt.mybatis.mapper.StudentMapper">

    <select id="selectByParamMap" resultType="student">
        select * from t_student where name = #{nameKey} and age = #{ageKey}
    </select>

</mapper>
```

- StudentMapperTest.java

```
package lzt.mybatis.test;

import lzt.mybatis.mapper.StudentMapper;
import lzt.mybatis.pojo.Student;
import lzt.mybatis.utils.SqlSessionUtil;
import org.junit.Test;

import java.text.ParseException;
```

```

import java.text.SimpleDateFormat;
import java.util.Date;
import java.util.HashMap; // 导包
import java.util.List;
import java.util.Map;     // 导包

public class StudentMapperTest {

    StudentMapper mapper = SqlSessionUtil.openSession().getMapper(StudentMapper.class);

    // ... (前面写的单个参数的测试方法保留) ...

    @Test
    public void testSelectByParamMap(){
        // 1. 准备 Map 集合, 把参数封装进去
        Map<String, Object> paramMap = new HashMap<>();

        // 老杜再次强调: 这里的 "nameKey" 和 "ageKey", 就是 XML 里面 #{ } 要抓取的标识符!
        paramMap.put("nameKey", "张三");
        paramMap.put("ageKey", 20);

        // 2. 调用接口方法, 把 Map 这个大包裹传过去
        List<Student> students = mapper.selectByParamMap(paramMap);

        // 3. 打印结果
        students.forEach(student -> System.out.println(student));
    }
}

```

```

D:\Java21\bin\java.exe ...
[main] DEBUG lzt.mybatis.mapper.StudentMapper.selectByParamMap - ==> Preparing: select * from t_student where name = ? and age = ?
[main] DEBUG lzt.mybatis.mapper.StudentMapper.selectByParamMap - ==> Parameters: 张三(String), 20(Integer)
[main] DEBUG lzt.mybatis.mapper.StudentMapper.selectByParamMap - <==      Total: 2
Student{id=2, name='张三', age=20, height=1.81, sex=男, birth=Tue Aug 16 00:00:00 CST 2022}
Student{id=3, name='张三', age=20, height=1.81, sex=女, birth=Tue Aug 16 00:00:00 CST 2022}

```

测试运行正常。

这种方式是手动封装Map集合，将每个条件以key和value的形式存放到集合中。然后在使用的时候通过#{map集合的key}来取值。

实体类参数

需求：插入一条Student数据

- StudentMapper接口

```

/**
 * 保存学生数据
 * @param student
 * @return
 */
int insert(Student student);

```

- StudentMapper.xml

```

<insert id="insert">
    insert into t_student values(null,#{name},#{age},#{height},#{birth},#{sex})
</insert>

```

- StudentMapperTest.testInsert

```
@Test
public void testInsert(){
    Student student = new Student();
    student.setName("李四");
    student.setAge(30);
    student.setHeight(1.70);
    student.setSex('男');
    student.setBirth(new Date());
    int count = mapper.insert(student);
    SqlSessionUtil.openSession().commit();
}
```

```
D:\Java21\bin\java.exe ...
[main] DEBUG lzt.mybatis.mapper.StudentMapper.insert - ==> Preparing: insert into t_student values(?, ?, ?, ?, ?)
[main] DEBUG lzt.mybatis.mapper.StudentMapper.insert - ==> Parameters: 李四(String), 30(Integer), 1.7(Double), 2026-03-12 20:47:08.58
(Timestamp), 男(String)
[main] DEBUG lzt.mybatis.mapper.StudentMapper.insert - <== Updates: 1

进程已结束，退出代码为 0
```

id	name	age	height	birth	sex
# bigint	varchar(255)	# int	# double	date	char(1)
2	张三	20	1.81	2022-08-16	男
3	张三	20	1.81	2022-08-16	女
4	李四	30	1.7	2026-03-12	男

运行正常，数据库中成功添加一条数据。

这里需要注意的是：#{ } 里面写的是属性名字。这个属性名其本质上是：set/get方法名去掉set/get之后的名字。

多参数

需求：通过name和sex查询

- StudentMapper接口

```
/**
 * 根据name和sex查询
 * @param name
 * @param sex
 * @return
 */
List<Student> selectByNameAndSex(String name, Character sex);
```

- StudentMapper.xml

```
<select id="selectByNameAndSex" resultType="student">
    select * from t_student where name = #{name} and sex = #{sex}
</select>
```

- StudentMapperTest.testSelectByNameAndSex

```
● ● ●
@Test
public void testSelectByNameAndSex(){
    List<Student> students = mapper.selectByNameAndSex("张三", '女');
    students.forEach(student -> System.out.println(student));
}
```

执行结果：

```
org.apache.ibatis.exceptions.PersistenceException:
### Error querying database.  Cause: org.apache.ibatis.binding.BindingException: Parameter 'name' not found. Available parameters
are [arg1, arg0, param1, param2]
### Cause: org.apache.ibatis.binding.BindingException: Parameter 'name' not found. Available parameters are [arg1, arg0, param1,
param2]

at org.apache.ibatis.exceptions.ExceptionFactory.wrapException(ExceptionFactory.java:30)
at org.apache.ibatis.session.defaults.DefaultSqlSession.selectList(DefaultSqlSession.java:156)
at org.apache.ibatis.session.defaults.DefaultSqlSession.selectList(DefaultSqlSession.java:147)
at org.apache.ibatis.session.defaults.DefaultSqlSession.selectList(DefaultSqlSession.java:142)
at org.apache.ibatis.binding.MapperMethod.executeForMany(MapperMethod.java:147)
```

异常信息描述了：name参数找不到，可用的参数包括[arg1, arg0, param1, param2]

修改StudentMapper.xml配置文件：尝试使用[arg1, arg0, param1, param2]去参数

```
● ● ●
<select id="selectByNameAndSex" resultType="student">
    <!--select * from t_student where name = #{name} and sex = #{sex}-->
    select * from t_student where name = #{arg0} and sex = #{arg1}
</select>
```

运行结果：

```
D:\Java21\bin\java.exe ...
[main] DEBUG l.mybatis.mapper.StudentMapper.selectByNameAndSex - ==> Preparing: select * from t_student where name = ? and sex = ?
[main] DEBUG l.mybatis.mapper.StudentMapper.selectByNameAndSex - ==> Parameters: 张三(String), 女(String)
[main] DEBUG l.mybatis.mapper.StudentMapper.selectByNameAndSex - <==      Total: 1
Student{id=3, name='张三', age=20, height=1.81, sex=女, birth=Tue Aug 16 00:00:00 CST 2022}
```

再次尝试修改StudentMapper.xml文件

```
● ● ●
<select id="selectByNameAndSex" resultType="student">
    <!--select * from t_student where name = #{name} and sex = #{sex}-->
    <!--select * from t_student where name = #{arg0} and sex = #{arg1}-->
    <!--select * from t_student where name = #{param1} and sex = #{param2}-->
    select * from t_student where name = #{arg0} and sex = #{param2}
</select>
```

✔ 测试 已通过: 1/共 1 个测试 - /UI 笔秒

```
D:\Java21\bin\java.exe ...
[main] DEBUG l.mybatis.mapper.StudentMapper.selectByNameAndSex - ==> Preparing: select * from t_student where name = ? and sex = ?
[main] DEBUG l.mybatis.mapper.StudentMapper.selectByNameAndSex - ==> Parameters: 张三(String), 女(String)
[main] DEBUG l.mybatis.mapper.StudentMapper.selectByNameAndSex - <==      Total: 1
Student{id=3, name='张三', age=20, height=1.81, sex=女, birth=Tue Aug 16 00:00:00 CST 2022}
```

通过测试可以看到：

- arg0 是第一个参数
- param1是第一个参数
- arg1 是第二个参数

- param2是第二个参数

实现原理：实际上在mybatis底层会创建一个map集合，以arg0/param1为key，以方法上的参数为value，例如以下代码：

```
Map<String, Object> map = new HashMap<>();
map.put("arg0", name);
map.put("arg1", sex);
map.put("param1", name);
map.put("param2", sex);

// 所以可以这样取值: #{arg0} #{arg1} #{param1} #{param2}
// 其本质就是#{map集合的key}
```

注意：使用mybatis3.4.2之前的版本时：要用#{0}和#{1}这种形式。

@Param注解（命名参数）

可以不用arg0 arg1 param1 param2吗？这个map集合的key我们自定义可以吗？当然可以。使用@Param注解即可。这样可以增强可读性。

需求：根据name和age查询

- StudentMapper接口
- 导包

```
import org.apache.ibatis.annotations.Param;
```

```
/**
 * 根据name和age查询
 * @param name
 * @param age
 * @return
 */
List<Student> selectByNameAndAge(@Param(value="name") String name, @Param("age") int age);
```

- StudentMapper.xml

```
<select id="selectByNameAndAge" resultType="student">
  select * from t_student where name = #{name} and age = #{age}
</select>
```

- StudentMapperTest.testSelectByNameAndAge

```
@Test
public void testSelectByNameAndAge(){
    List<Student> stus = mapper.selectByNameAndAge("张三", 20);
    stus.forEach(student -> System.out.println(student));
}
```

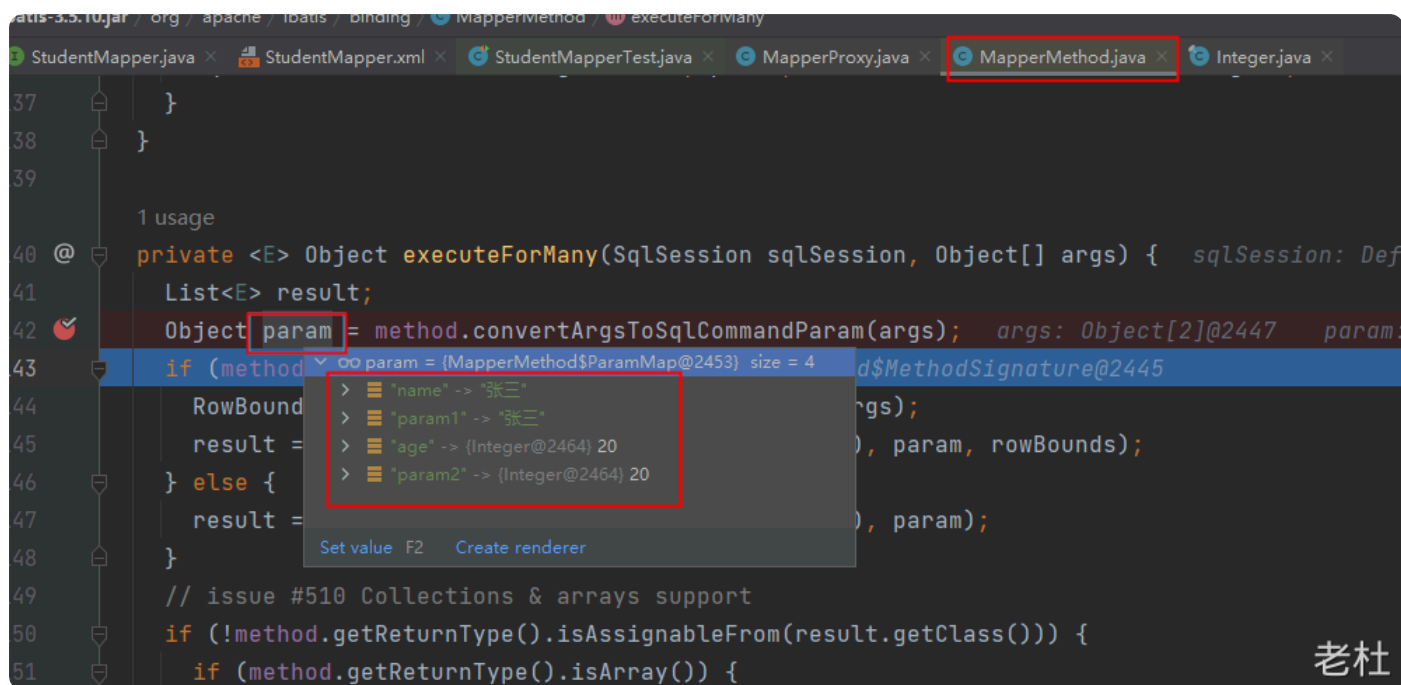


```
[main] DEBUG l.mybatis.mapper.StudentMapper.selectByNameAndAge - ==> Preparing: select * from t_student where name = ? and age = ?
[main] DEBUG l.mybatis.mapper.StudentMapper.selectByNameAndAge - ==> Parameters: 张三(String), 20(Integer)
[main] DEBUG l.mybatis.mapper.StudentMapper.selectByNameAndAge - <==      Total: 2
Student{id=2, name='张三', age=20, height=1.81, sex=男, birth=Tue Aug 16 00:00:00 CST 2022}
Student{id=3, name='张三', age=20, height=1.81, sex=女, birth=Tue Aug 16 00:00:00 CST 2022}
```

通过测试，一切正常。

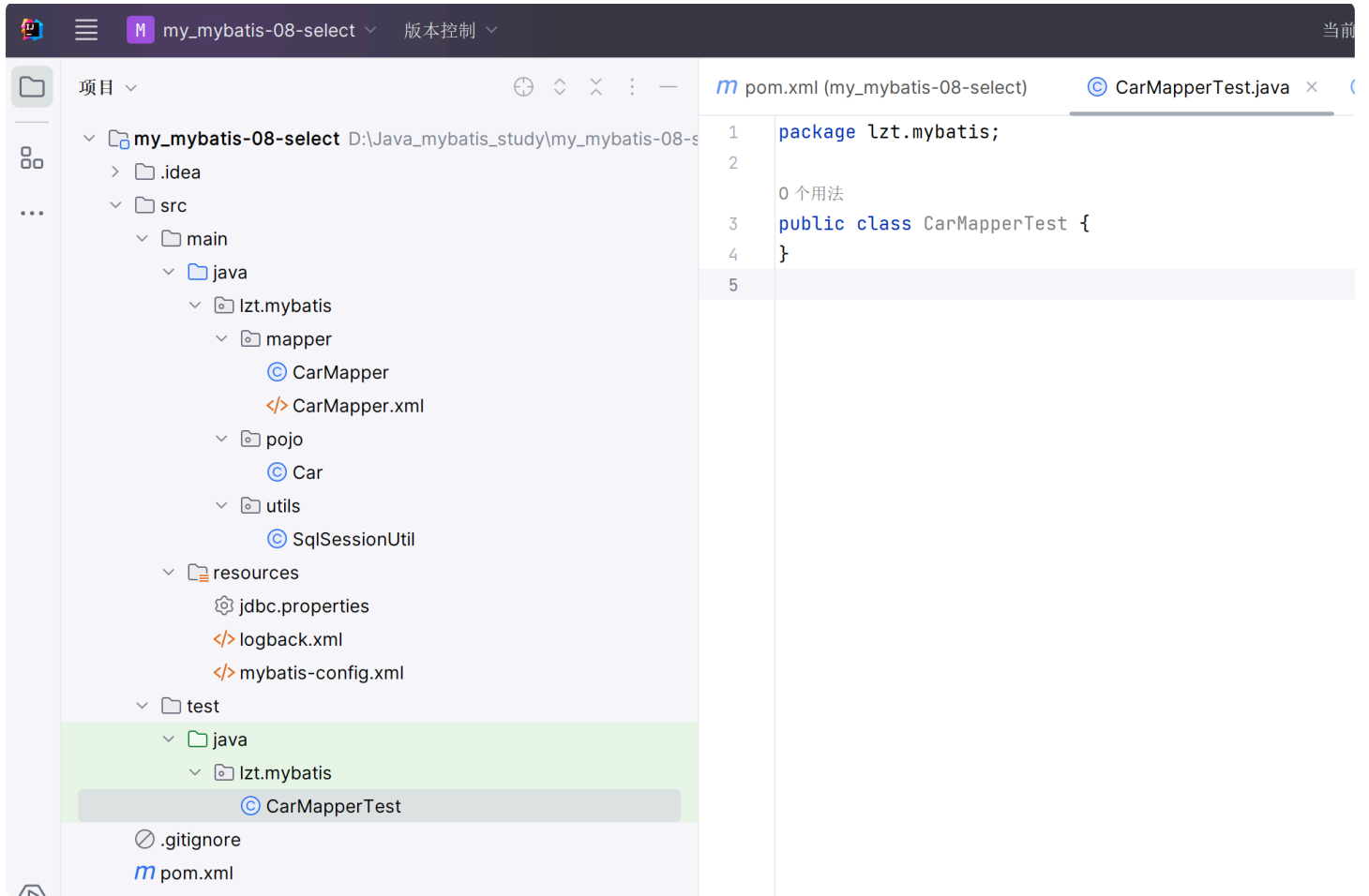
核心：@Param("这里填写的其实就是map集合的key")

@Param源码分析



MyBatis查询语句专题

- 模块名：my_mybatis-08-select
- 数据库：mybatis_demo
- 数据表：t_car
- MySQL密码：20060329
- 打包方式：jar
- 引入依赖：mysql驱动依赖、mybatis依赖、logback依赖、junit依赖。
- 引入配置文件：jdbc.properties、mybatis-config.xml、logback.xml
- 创建pojo类：Car
- 创建Mapper接口：CarMapper
- 创建Mapper接口对应的映射文件：lzt/mybatis/mapper/CarMapper.xml
- 创建单元测试：CarMapperTest
- 拷贝工具类：SqlSessionUtil



返回Car

当查询的结果，有对应的实体类，并且查询结果只有一条时：

- Mapper 接口：CarMapper.java

```
package lzt.mybatis.mapper;

import lzt.mybatis.pojo.Car;

/**
 * Car SQL 映射器接口
 * 重点关注：当查询结果只有一条时，方法的返回值类型直接写对应的 POJO 类。
 */
public interface CarMapper {

    /**
     * 根据id主键查询：结果最多只有一条
     * @param id 汽车的主键
     * @return 封装好的 Car 对象
     */
    Car selectById(Long id);
}
```

- Mapper 映射文件：CarMapper.xml

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper
PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
```

```

"http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper namespace="lzt.mybatis.mapper.CarMapper">

    <select id="selectById" resultType="Car">
        select
            id,
            car_num as carNum,
            brand,
            guide_price as guidePrice,
            produce_time as produceTime,
            car_type as carType
        from t_car
        where id = #{id}
    </select>

</mapper>

```

- 测试类：CarMapperTest.java

```

package lzt.mybatis.test;

import lzt.mybatis.mapper.CarMapper;
import lzt.mybatis.pojo.Car;
import lzt.mybatis.utils.SqlSessionUtil;
import org.apache.ibatis.session.SqlSession;
import org.junit.Test;

public class CarMapperTest {

    @Test
    public void testSelectById(){
        // 1. 获取 SqlSession 对象
        SqlSession sqlSession = SqlSessionUtil.openSession();

        // 2. 获取 Mapper 接口的代理对象
        CarMapper mapper = sqlSession.getMapper(CarMapper.class);

        // 3. 执行查询。根据你的数据库数据，我们查询 id 为 1 的记录
        Car car = mapper.selectById(1L);

        // 4. 打印结果
        System.out.println(car);

        SqlSessionUtil.close(sqlSession);
    }
}

```

执行结果：

```

D:\Java21\bin\java.exe ...
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectById - ==> Preparing: select id, car_num as carNum, brand, guide_price as
guidePrice, produce_time as produceTime, car_type as carType from t_car where id = ?
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectById - ==> Parameters: 1(Long)
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectById - <==      Total: 1
Car{id=1, carNum='100', brand='宝马520Li', guidePrice=41.0, produceTime='2022-09-01', carType='燃油车'}

```

查询结果是一条的话可以使用List集合接收吗？当然可以。

- CarMapper.selectByIdToList

```

import java.util.List;

/**
 * 根据id主键查询: 结果最多只有一条, 可以放到List集合中吗?
 * @return
 */
List<Car> selectByIdToList(Long id);

```

- CarMapper.xml

```

<select id="selectByIdToList" resultType="Car">
    select id,car_num carNum,brand,guide_price guidePrice,produce_time produceTime,car_type carType from t_car where id
    = #{id}
</select>

```

- CarMapperTest.testSelectByIdToList

```

import java.util.List;

@Test
public void testSelectByIdToList(){
    CarMapper mapper = SqlSessionUtil.openSession().getMapper(CarMapper.class);
    List<Car> cars = mapper.selectByIdToList(35L);
    System.out.println(cars);
}

```

```

D:\Java21\bin\java.exe ...
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectByIdToList - ==> Preparing: select id,car_num carNum,brand,guide_price guidePrice,
    produce_time produceTime,car_type carType from t_car where id = ?
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectByIdToList - ==> Parameters: 3(Long)
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectByIdToList - <==          Total: 1
[Car{id=3, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}]

```

返回List<Car>

当查询的记录条数是多条的时候, 必须使用集合接收。如果使用单个实体类接收会出现异常。

- CarMapper.selectAll

```

/**
 * 查询所有的Car
 * @return
 */
List<Car> selectAll();

```

- CarMapper.xml

```

<select id="selectAll" resultType="Car">
    select id,car_num carNum,brand,guide_price guidePrice,produce_time produceTime,car_type carType from t_car
</select>

```

- CarMapperTest.testSelectAll



```

@Test
public void testSelectAll(){
    CarMapper mapper = SqlSessionUtil.openSession().getMapper(CarMapper.class);
    List<Car> cars = mapper.selectAll();
    cars.forEach(car -> System.out.println(car));
}

```

D:\Java21\bin\java.exe ...

```

[main] DEBUG lzt.mybatis.mapper.CarMapper.selectAll - ==> Preparing: select id,car_num carNum,brand,guide_price guidePrice,
produce_time produceTime,car_type carType from t_car
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectAll - ==> Parameters:
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectAll - <==          Total: 11
Car{id=1, carNum='100', brand='宝马520Li', guidePrice=41.0, produceTime='2022-09-01', carType='燃油车'}
Car{id=2, carNum='101', brand='奔驰E300L', guidePrice=54.0, produceTime='2022-08-01', carType='电车'}
Car{id=3, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=4, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=5, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=6, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=7, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=8, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=9, carNum='102', brand='比亚迪汉', guidePrice=30.23, produceTime='2018-09-10', carType='电车'}
Car{id=10, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}
Car{id=29, carNum='5262', brand='BYD汉', guidePrice=30.3, produceTime='2020-10-11', carType='新能源'}

```

如果返回多条记录，采用单个实体类接收会怎样？

- CarMapper



```

/**
 * 查询多条记录，采用单个实体类接收会怎样？
 * @return
 */
Car selectAll2();

```

- CarMapper.xml



```

<select id="selectAll2" resultType="Car">
    select id,car_num carNum,brand,guide_price guidePrice,produce_time produceTime,car_type carType from t_car
</select>

```

- CarMapperTest.testSelectAll2



```

@Test
public void testSelectAll2(){
    CarMapper mapper = SqlSessionUtil.openSession().getMapper(CarMapper.class);
    Car car = mapper.selectAll2();
    System.out.println(car);
}

```

```
D:\Java21\bin\java.exe ...
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectAll2 - ==> Preparing: select id,car_num carNum,brand,guide_price guidePrice,
produce_time produceTime,car_type carType from t_car
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectAll2 - ==> Parameters:
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectAll2 - <==          Total: 11

org.apache.ibatis.exceptions_TOO_MANY_RESULTS: Expected one result (or null) to be returned by selectOne(), but found: 11

    at org.apache.ibatis.session.defaults.DefaultSqlSession.selectOne(DefaultSqlSession.java:81)
    at org.apache.ibatis.binding.MapperMethod.execute(MapperMethod.java:87)
    at org.apache.ibatis.binding.MapperProxy$PlainMethodInvoker.invoke(MapperProxy.java:141)
    at org.apache.ibatis.binding.MapperProxy.invoke(MapperProxy.java:86)
    at jdk.proxy2/jdk.proxy2.$Proxy9.selectAll2(Unknown Source)
    at lzt.mybatis.CarMapperTest.testSelectAll2(CarMapperTest.java:45) <25 个内部行>
```

返回Map

当返回的数据，没有合适的实体类对应的话，可以采用Map集合接收。字段名做key，字段值做value。

查询如果可以保证只有一条数据，则返回一个Map集合即可。

Map集合	
key	value
id	1
carNum	1001
brand	奔驰E300L
guidePrice	50.5
produceTime	2010/11/11
carType	燃油车

- CarMapper.selectByIdRetMap

```
import java.util.Map;
/**
 * 通过id查询一条记录，返回Map集合
 * @param id
 * @return
 */
Map<String, Object> selectByIdRetMap(Long id);
```

- CarMapper.xml

```
<select id="selectByIdRetMap" resultType="map">
    select id,car_num carNum,brand,guide_price guidePrice,produce_time produceTime,car_type carType from t_car where id
    = #{id}
</select>
```

- resultMap="map", 这是因为mybatis内置了很多别名。【参见mybatis开发手册】



```
import java.util.Map;

@Test
public void testSelectByIdRetMap(){
    CarMapper mapper = SqlSessionUtil.openSession().getMapper(CarMapper.class);
    Map<String, Object> car = mapper.selectByIdRetMap(35L);
    System.out.println(car);
}
```

执行结果：

```
D:\Java21\bin\java.exe ...
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectByIdRetMap - ==> Preparing: select id,car_num carNum,brand,guide_price guidePrice,
produce_time produceTime,car_type carType from t_car where id = ?
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectByIdRetMap - ==> Parameters: 2(Long)
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectByIdRetMap - <== Total: 1
{carType=电车, carNum=101, guidePrice=54.00, produceTime=2022-08-01, id=2, brand=奔驰E300L}
```

当然，如果返回一个Map集合，可以将Map集合放到List集合中吗？当然可以，这里就不再测试了。

反过来，如果返回的不是一条记录，是多条记录的话，只采用单个Map集合接收，这样同样会出现之前的异常：

TooManyResultsException

返回List<Map>

查询结果条数大于等于1条数据，则可以返回一个存储Map集合的List集合。List<Map> 等同于List<Car>

List<Map>

map1	
key	value
id	1
carNum	1001
brand	奔驰E300L
guidePrice	50.5
produceTime	2010/11/11
carType	燃油车

map2	
key	value
id	2
carNum	1002
brand	丰田霸道
guidePrice	30
produceTime	2010/11/11
carType	燃油车

map3	
key	value
id	3
carNum	1003
brand	凯美瑞
guidePrice	20
produceTime	2010/11/11
carType	燃油车

老杜

- CarMapper接口

```
/**
 * 查询所有的Car，返回一个List集合。List集合中存储的是Map集合。
 * @return
 */
List<Map<String,Object>> selectAllRetListMap();
```

- CarMapper.xml

```
<select id="selectAllRetListMap" resultType="map">
    select id,car_num carNum,brand,guide_price guidePrice,produce_time produceTime,car_type carType from t_car
</select>
```

- CarMapperTest.testSelectAllRetListMap

```
@Test
public void testSelectAllRetListMap(){
    CarMapper mapper = SqlSessionUtil.openSession().getMapper(CarMapper.class);
    List<Map<String,Object>> cars = mapper.selectAllRetListMap();
    System.out.println(cars);
}
```


执行结果：

```

[main] DEBUG lzt.mybatis.mapper.CarMapper.selectAllRetListMap - ==> Preparing: select id,car_num carNum,brand,guide_price
guidePrice,produce_time produceTime,car_type carType from t_car
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectAllRetListMap - ==> Parameters:
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectAllRetListMap - <==      Total: 11
[{carType=燃油车, carNum=100, guidePrice=41.00, produceTime=2022-09-01, id=1, brand=宝马520Li}, {carType=电车, carNum=101,
guidePrice=54.00, produceTime=2022-08-01, id=2, brand=奔驰E300L}, {carType=氢能源, carNum=102, guidePrice=40.30,
produceTime=2014-10-05, id=3, brand=丰田未来}, {carType=氢能源, carNum=102, guidePrice=40.30, produceTime=2014-10-05, id=4, brand=丰田
未来}, {carType=氢能源, carNum=102, guidePrice=40.30, produceTime=2014-10-05, id=5, brand=丰田未来}, {carType=氢能源, carNum=102,
guidePrice=40.30, produceTime=2014-10-05, id=6, brand=丰田未来}, {carType=氢能源, carNum=102, guidePrice=40.30, produceTime=2014-10-05, id=7, brand=丰田未来}, {carType=氢能源, carNum=102, guidePrice=40.30, produceTime=2014-10-05, id=8, brand=丰田
未来}, {carType=电车, carNum=102, guidePrice=30.23, produceTime=2018-09-10, id=9, brand=比亚迪汉}, {carType=氢能源, carNum=102,
guidePrice=40.30, produceTime=2014-10-05, id=10, brand=丰田未来}, {carType=新能源, carNum=5262, guidePrice=30.30,
produceTime=2020-10-11, id=29, brand=BYD汉}]

```

返回Map<String,Map>

拿Car的id做key，以后取出对应的Map集合时更方便。

Map<String,Map>

1	<table><tr><th colspan="2">map1</th></tr><tr><th>key</th><th>value</th></tr><tr><td>id</td><td>1</td></tr><tr><td>carNum</td><td>1001</td></tr><tr><td>brand</td><td>奔驰E300L</td></tr><tr><td>guidePrice</td><td>50.5</td></tr><tr><td>produceTime</td><td>2010/11/11</td></tr><tr><td>carType</td><td>燃油车</td></tr></table>	map1		key	value	id	1	carNum	1001	brand	奔驰E300L	guidePrice	50.5	produceTime	2010/11/11	carType	燃油车
map1																	
key	value																
id	1																
carNum	1001																
brand	奔驰E300L																
guidePrice	50.5																
produceTime	2010/11/11																
carType	燃油车																
2	<table><tr><th colspan="2">map2</th></tr><tr><th>key</th><th>value</th></tr><tr><td>id</td><td>2</td></tr><tr><td>carNum</td><td>1002</td></tr><tr><td>brand</td><td>丰田霸道</td></tr><tr><td>guidePrice</td><td>30</td></tr><tr><td>produceTime</td><td>2010/11/11</td></tr><tr><td>carType</td><td>燃油车</td></tr></table>	map2		key	value	id	2	carNum	1002	brand	丰田霸道	guidePrice	30	produceTime	2010/11/11	carType	燃油车
map2																	
key	value																
id	2																
carNum	1002																
brand	丰田霸道																
guidePrice	30																
produceTime	2010/11/11																
carType	燃油车																
3	<table><tr><th colspan="2">map3</th></tr><tr><th>key</th><th>value</th></tr><tr><td>id</td><td>3</td></tr><tr><td>carNum</td><td>1003</td></tr><tr><td>brand</td><td>凯美瑞</td></tr><tr><td>guidePrice</td><td>20</td></tr><tr><td>produceTime</td><td>2010/11/11</td></tr><tr><td>carType</td><td>燃油车</td></tr></table>	map3		key	value	id	3	carNum	1003	brand	凯美瑞	guidePrice	20	produceTime	2010/11/11	carType	燃油车
map3																	
key	value																
id	3																
carNum	1003																
brand	凯美瑞																
guidePrice	20																
produceTime	2010/11/11																
carType	燃油车																

老杜

- CarMapper接口



```
import org.apache.ibatis.annotations.MapKey;

/**
 * 获取所有的Car，返回一个Map集合。
 * Map集合的key是Car的id。
 * Map集合的value是对应Car。
 * @return
 */
@MapKey("id")
Map<Long,Map<String,Object>> selectAllRetMap();
```

- CarMapper.xml



```
<select id="selectAllRetMap" resultType="map">
    select id,car_num carNum,brand,guide_price guidePrice,produce_time produceTime,car_type carType from t_car
</select>
```

- CarMapperTest.testSelectAllRetMap



```
@Test
public void testSelectAllRetMap(){
    CarMapper mapper = SqlSessionUtil.openSession().getMapper(CarMapper.class);
    Map<Long,Map<String,Object>> cars = mapper.selectAllRetMap();
    System.out.println(cars);
}
```

执行结果：

D:\Java21\bin\java.exe ...

```
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectAllRetMap - ==> Preparing: select id,car_num carNum,brand,guide_price guidePrice,
produce_time produceTime,car_type carType from t_car
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectAllRetMap - ==> Parameters:
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectAllRetMap - <== Total: 11
{1={carType=燃油车, carNum=100, guidePrice=41.00, produceTime=2022-09-01, id=1, brand=宝马520Li}, 2={carType=电车, carNum=101,
guidePrice=54.00, produceTime=2022-08-01, id=2, brand=奔驰E300L}, 3={carType=氢能源, carNum=102, guidePrice=40.30,
produceTime=2014-10-05, id=3, brand=丰田未来}, 4={carType=氢能源, carNum=102, guidePrice=40.30, produceTime=2014-10-05, id=4, brand=丰
田未来}, 5={carType=氢能源, carNum=102, guidePrice=40.30, produceTime=2014-10-05, id=5, brand=丰田未来}, 6={carType=氢能源, carNum=102,
guidePrice=40.30, produceTime=2014-10-05, id=6, brand=丰田未来}, 7={carType=氢能源, carNum=102, guidePrice=40.30,
produceTime=2014-10-05, id=7, brand=丰田未来}, 8={carType=氢能源, carNum=102, guidePrice=40.30, produceTime=2014-10-05, id=8, brand=丰
田未来}, 9={carType=电车, carNum=102, guidePrice=30.23, produceTime=2018-09-10, id=9, brand=比亚迪汉}, 10={carType=氢能源, carNum=102,
guidePrice=40.30, produceTime=2014-10-05, id=10, brand=丰田未来}, 29={carType=新能源, carNum=5262, guidePrice=30.30,
produceTime=2020-10-11, id=29, brand=BYD汉}}
```

resultMap结果映射

resultMap（结果映射）是 MyBatis 中用于将 SQL 查询结果 映射到 Java 对象 的机制。
简单来说，就是告诉 MyBatis：数据库表的字段 如何对应 Java 类的属性。

查询结果的列名和java对象的属性名对应不上怎么办？

- 第一种方式：as 给列起别名
- 第二种方式：使用resultMap进行结果映射
- 第三种方式：是否开启驼峰命名自动映射（配置settings）

使用resultMap进行结果映射

- CarMapper接口

```
/**
 * 查询所有Car，使用resultMap进行结果映射
 * @return
 */
List<Car> selectAllByResultMap();
```

- CarMapper.xml

```
<!--
    resultMap:
        id: 这个结果映射的标识，作为select标签的resultMap属性的值。
        type: 结果集要映射的类。可以使用别名。
-->
<resultMap id="carResultMap" type="car">
    <!-- 对象的唯一标识，官方解释是：为了提高mybatis的性能。建议写上。 -->
    <id property="id" column="id"/>
    <result property="carNum" column="car_num"/>
    <!-- 当属性名和数据库列名一致时，可以省略。但建议都写上。 -->
    <!-- javaType用来指定属性类型。jdbcType用来指定列类型。一般可以省略。 -->
    <result property="brand" column="brand" javaType="string" jdbcType="VARCHAR"/>
    <result property="guidePrice" column="guide_price"/>
    <result property="produceTime" column="produce_time"/>
    <result property="carType" column="car_type"/>
</resultMap>

<!-- resultMap属性的值必须和resultMap标签中id属性值一致。 -->
<select id="selectAllByResultMap" resultMap="carResultMap">
    select * from t_car
</select>
```

- CarMapperTest.testSelectAllByResultMap

```
@Test
public void testSelectAllByResultMap(){
    CarMapper carMapper = SqlSessionUtil.openSession().getMapper(CarMapper.class);
    List<Car> cars = carMapper.selectAllByResultMap();
    System.out.println(cars);
}
```

```
D:\Java21\bin\java.exe ...
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectAllByResultMap - ==> Preparing: select * from t_car
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectAllByResultMap - ==> Parameters:
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectAllByResultMap - <==      Total: 11
[Car{id=1, carNum='100', brand='宝马520Li', guidePrice=41.0, produceTime='2022-09-01', carType='燃油车'}, Car{id=2, carNum='101',
brand='奔驰E300L', guidePrice=54.0, produceTime='2022-08-01', carType='电车'}, Car{id=3, carNum='102', brand='丰田未来', guidePrice=40.
3, produceTime='2014-10-05', carType='氢能源'}, Car{id=4, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05',
carType='氢能源'}, Car{id=5, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}, Car{id=6,
carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}, Car{id=7, carNum='102', brand='丰田未来',
guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}, Car{id=8, carNum='102', brand='丰田未来', guidePrice=40.3,
produceTime='2014-10-05', carType='氢能源'}, Car{id=9, carNum='102', brand='比亚迪汉', guidePrice=30.23, produceTime='2018-09-10',
carType='电车'}, Car{id=10, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}, Car{id=29,
carNum='5262', brand='BYD汉', guidePrice=30.3, produceTime='2020-10-11', carType='新能源'}]
```

执行结果正常。

是否开启驼峰命名自动映射

使用这种方式的前提是：属性名遵循Java的命名规范，数据库表的列名遵循SQL的命名规范。

Java命名规范：首字母小写，后面每个单词首字母大写，遵循驼峰命名方式。

SQL命名规范：全部小写，单词之间采用下划线分割。

比如以下的对应关系：

实体类中的属性名	数据库表的列名
carNum	car_num
carType	car_type
produceTime	produce_time

如何启用该功能，在mybatis-config.xml文件中进行配置：

- 修改mybatis-config.xml `<settings>` 标签

```
<!-- 放在properties 标签后面-->
<settings>
  <setting name="mapUnderscoreToCamelCase" value="true"/>
</settings>
```

- CarMapper接口

```
/**
 * 查询所有Car，启用驼峰命名自动映射
 * @return
 */
List<Car> selectAllByMapUnderscoreToCamelCase();
```

- CarMapper.xml

```
<select id="selectAllByMapUnderscoreToCamelCase" resultType="Car">
  select * from t_car
</select>
```

- CarMapperTest.testSelectAllByMapUnderscoreToCamelCase

```
@Test
public void testSelectAllByMapUnderscoreToCamelCase(){
  CarMapper carMapper = SqlSessionUtil.openSession().getMapper(CarMapper.class);
  List<Car> cars = carMapper.selectAllByMapUnderscoreToCamelCase();
  System.out.println(cars);
}
```

```
D:\Java21\bin\java.exe ...
```

```
[main] DEBUG l.m.m.C.selectAllByMapUnderscoreToCamelCase - ==> Preparing: select * from t_car
[main] DEBUG l.m.m.C.selectAllByMapUnderscoreToCamelCase - ==> Parameters:
[main] DEBUG l.m.m.C.selectAllByMapUnderscoreToCamelCase - <==          Total: 11
[Car{id=1, carNum='100', brand='宝马520Li', guidePrice=41.0, produceTime='2022-09-01', carType='燃油车'}, Car{id=2, carNum='101',
brand='奔驰E300L', guidePrice=54.0, produceTime='2022-08-01', carType='电车'}, Car{id=3, carNum='102', brand='丰田未来', guidePrice=40
.3, produceTime='2014-10-05', carType='氢能源'}, Car{id=4, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05',
carType='氢能源'}, Car{id=5, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}, Car{id=6,
carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}, Car{id=7, carNum='102', brand='丰田未来',
guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}, Car{id=8, carNum='102', brand='丰田未来', guidePrice=40.3,
produceTime='2014-10-05', carType='氢能源'}, Car{id=9, carNum='102', brand='比亚迪汉', guidePrice=30.23, produceTime='2018-09-10',
carType='电车'}, Car{id=10, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}, Car{id=29,
carNum='5262', brand='BYD汉', guidePrice=30.3, produceTime='2020-10-11', carType='新能源'}]
```

返回总记录条数

需求：查询总记录条数

- CarMapper接口

```
/**
 * 获取总记录条数
 * @return
 */
Long selectTotal();
```

- CarMapper.xml

```
<!--Long是别名，可参考mybatis开发手册。-->
<select id="selectTotal" resultType="Long">
    select count(*) from t_car
</select>
```

- CarMapperTest.testSelectTotal

```
@Test
public void testSelectTotal(){
    CarMapper carMapper = SqlSessionUtil.openSession().getMapper(CarMapper.class);
    Long total = carMapper.selectTotal();
    System.out.println(total);
}
```

```
D:\Java21\bin\java.exe ...
```

```
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectTotal - ==> Preparing: select count(*) from t_car
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectTotal - ==> Parameters:
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectTotal - <==          Total: 1
11
```

动态SQL

有的业务场景，也需要SQL语句进行动态拼接，例如：

- 批量删除

全选 ☒ 删除

☒ 张三

☒ 李四

☒ 王五

☒ 赵六

老杜

- sql语句

```
delete from t_car where id in(1,2,3,4,5,6,.....这里的值是动态的，根据用户选择的id不同，值是不同的);
```

- 多条件查询

全部结果 > 品牌: Apple × 运行内存: 18GB × 机身内存: 1TB × "手机"

CPU型号:	骁龙8+ Gen 1	骁龙8 Gen 1	A15	Apple A系列	骁龙4系列	功能机	
屏幕尺寸:	7英寸以上	6.8-7.0英寸	6.6-6.79英寸	6.3-6.59英寸	6.0-6.29英寸	6.0英寸以下	未上市
特征特质:	OLED直屏	屏幕指纹	5G				
电池容量:	5500mAh以上	5001-5500mAh	4500-5000mAh	4000-4499mAh	3000-3999mAh		
屏幕材质:	OLED折叠屏	OLED直屏	LTPS LCD	LCD	未上市		
高级选项:	充电功率 ▾	后摄主像素 ▾	屏幕分辨率 ▾	机身色系 ▾	三防标准 ▾	其他分类 ▾	

商品精选 广告

HUAWEI

畅享20e

5000mAh大电池
6+128GB大内存
搭载鸿蒙系统

¥969.00

综合 ↓ 销量 ↓ 评论数 ↓ 新品 ↓ 价格 ↑

配送至 北京东城区 ▾ ☐ 京东物流 ☐ 货到付款 ☐ 仅显示有货 ☐ 京东国际



- SQL语句

```
select * from t_car where brand like '丰田%' and guide_price > 30 and .....
```

创建模块: my_mybatis-09-dynamic-sql

打包方式: jar

引入依赖: mysql驱动依赖、mybatis依赖、junit依赖、logback依赖

pojo: Car

mapper接口: mapper.CarMapper

引入配置文件: mybatis-config.xml、jdbc.properties、logback.xml

mapper配置文件: mapper/CarMapper.xml

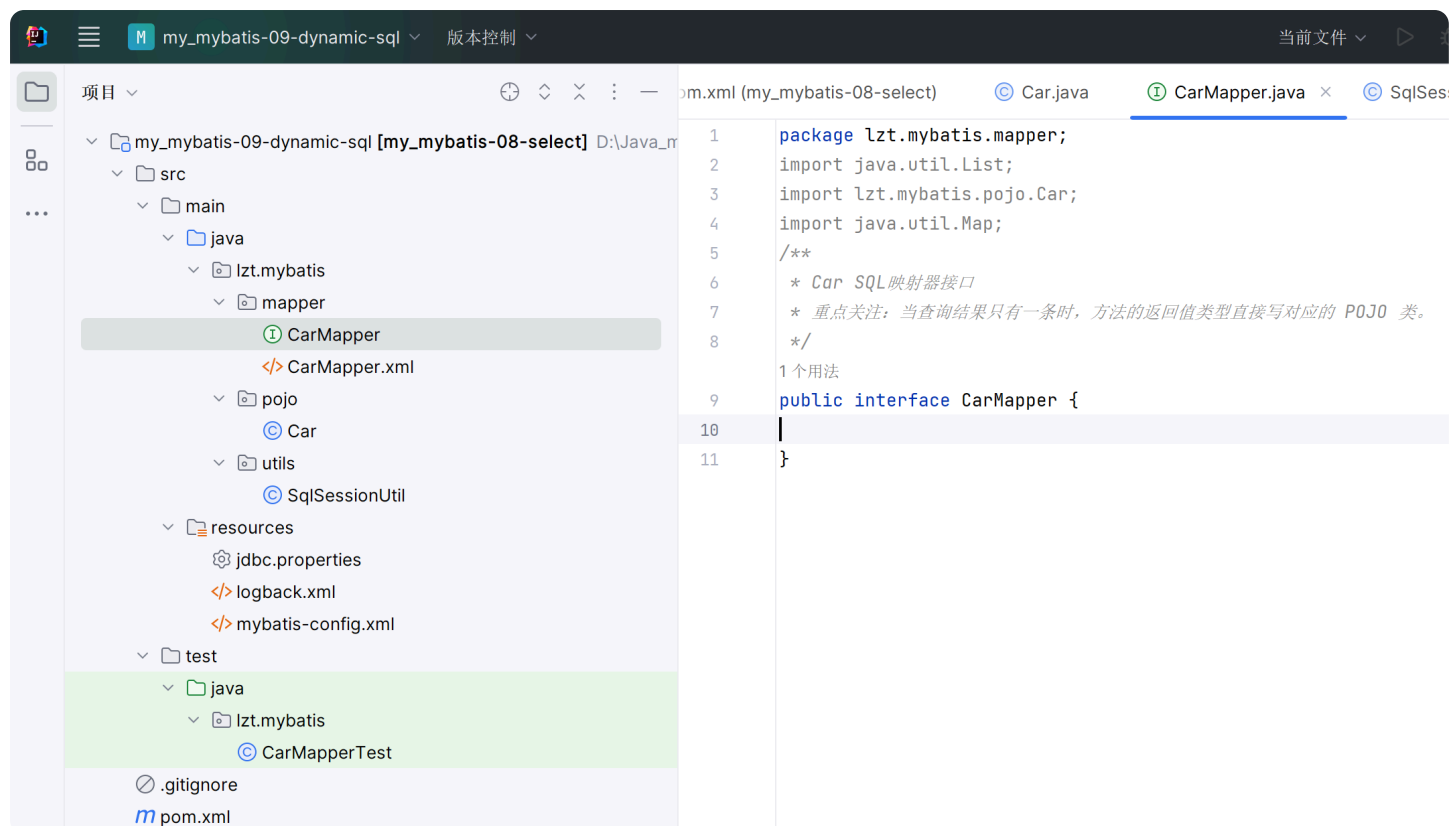
编写测试类: test.CarMapperTest

拷贝工具类: SqlSessionUtil

数据库: mybatis_demo

数据表: t_car

MySQL密码：20060329



if标签

需求：多条件查询。

可能的条件包括：品牌（brand）、指导价格（guide_price）、汽车类型（car_type）

- CarMapper.java

```
package lzt.mybatis.mapper;
import java.util.List;
import lzt.mybatis.pojo.Car;
import org.apache.ibatis.annotations.Param;

/**
 * Car SQL 映射器接口
 */
public interface CarMapper {
    /**
     * 根据多条件查询Car
     * @param brand
     * @param guidePrice
     * @param carType
     * @return
     */
    List<Car> selectByMultiCondition(@Param("brand") String brand, @Param("guidePrice") Double guidePrice,
    @Param("carType") String carType);
}
```

- CarMapper.xml

```
<?xml version="1.0" encoding="UTF-8" ?>
```

```

<!DOCTYPE mapper
  PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
  "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper namespace="lzt.mybatis.mapper.CarMapper">

  <select id="selectByMultiCondition" resultType="car">
    select * from t_car where
    <if test="brand != null and brand != ''">
      brand like #{brand}%"
    </if>
    <if test="guidePrice != null and guidePrice != ''">
      and guide_price >= #{guidePrice}
    </if>
    <if test="carType != null and carType != ''">
      and car_type = #{carType}
    </if>
  </select>

</mapper>

```

- CarMapperTest.testSelectByMultiCondition

```

package lzt.mybatis;
import lzt.mybatis.mapper.CarMapper;
import lzt.mybatis.pojo.Car;
import lzt.mybatis.utils.SqlSessionUtil;
import org.junit.Test;
import java.util.List;

public class CarMapperTest {
    @Test
    public void testSelectByMultiCondition(){
        CarMapper mapper = SqlSessionUtil.openSession().getMapper(CarMapper.class);
        List<Car> cars = mapper.selectByMultiCondition("宝马", 41.0, "燃油车");
        System.out.println(cars);
    }
}

```

执行结果：

```

D:\Java21\bin\java.exe ...
[main] DEBUG l.mybatis.mapper.CarMapper.selectByMultiCondition - ==> Preparing: select * from t_car where brand like ?"%" and
guide_price >= ? and car_type = ?
[main] DEBUG l.mybatis.mapper.CarMapper.selectByMultiCondition - ==> Parameters: 宝马(String), 41.0(Double), 燃油车(String)
[main] DEBUG l.mybatis.mapper.CarMapper.selectByMultiCondition - <==      Total: 1
[Car{id=1, carNum='100', brand='宝马520Li', guidePrice=41.0, produceTime='2022-09-01', carType='燃油车'}]

```

如果第一个条件为空，剩下两个条件不为空，会是怎样呢？

```

List<Car> cars = mapper.selectByMultiCondition("", 41.0, "燃油车");

```



```
D:\Java21\bin\java.exe ...
[main] DEBUG l.mybatis.mapper.CarMapper.selectByMultiCondition - ==> Preparing: select * from t_car where and guide_price >= ? and
car_type = ?
[main] DEBUG l.mybatis.mapper.CarMapper.selectByMultiCondition - ==> Parameters: 41.0(Double), 燃油车(String)

org.apache.ibatis.exceptions.PersistenceException:
### Error querying database. Cause: java.sql.SQLException: You have an error in your SQL syntax; check the manual that
corresponds to your MySQL server version for the right syntax to use near 'and guide_price >= 41.0

and car_type = '燃油车' at line 4
```

报错了，SQL语法有问题，where后面出现了and。这该怎么解决呢？

可以where后面添加一个恒成立的条件。

- 修改 CarMapper.xml

```
<select id="selectByMultiCondition" resultType="car">
    select * from t_car where 0 = 0
    <if test="brand != null and brand != ''">
        brand like #{brand}%"
    </if>
    <if test="guidePrice != null and guidePrice != ''">
        and guide_price >= #{guidePrice}
    </if>
    <if test="carType != null and carType != ''">
        and car_type = #{carType}
    </if>
</select>
```

执行结果：

```
D:\Java21\bin\java.exe ...
[main] DEBUG l.mybatis.mapper.CarMapper.selectByMultiCondition - ==> Preparing: select * from t_car where 0 = 0 and guide_price >=
? and car_type = ?
[main] DEBUG l.mybatis.mapper.CarMapper.selectByMultiCondition - ==> Parameters: 41.0(Double), 燃油车(String)
[main] DEBUG l.mybatis.mapper.CarMapper.selectByMultiCondition - <== Total: 1
[Car{id=1, carNum='100', brand='宝马520Li', guidePrice=41.0, produceTime='2022-09-01', carType='燃油车'}]
```

如果三个条件都是空，有影响吗？

```
List<Car> cars = mapper.selectByMultiCondition("", null, "");
```

执行结果：

```
D:\Java21\bin\java.exe ...
[main] DEBUG l.mybatis.mapper.CarMapper.selectByMultiCondition - ==> Preparing: select * from t_car where 0 = 0
[main] DEBUG l.mybatis.mapper.CarMapper.selectByMultiCondition - ==> Parameters:
[main] DEBUG l.mybatis.mapper.CarMapper.selectByMultiCondition - <== Total: 11
[Car{id=1, carNum='100', brand='宝马520Li', guidePrice=41.0, produceTime='2022-09-01', carType='燃油车'}, Car{id=2, carNum='101',
brand='奔驰E300L', guidePrice=54.0, produceTime='2022-08-01', carType='电车'}, Car{id=3, carNum='102', brand='丰田未来', guidePrice=40
.3, produceTime='2014-10-05', carType='氢能源'}, Car{id=4, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05',
carType='氢能源'}, Car{id=5, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}, Car{id=6,
carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}, Car{id=7, carNum='102', brand='丰田未来',
guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}, Car{id=8, carNum='102', brand='丰田未来', guidePrice=40.3,
produceTime='2014-10-05', carType='氢能源'}, Car{id=9, carNum='102', brand='比亚迪汉', guidePrice=30.23, produceTime='2018-09-10',
carType='电车'}, Car{id=10, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}, Car{id=29,
carNum='5262', brand='BYD汉', guidePrice=30.3, produceTime='2020-10-11', carType='新能源'}]
```

所有数据都查询出来

三个条件都不为空呢？

```
List<Car> cars = mapper.selectByMultiCondition("宝马", 41.0, "燃油车");
```

执行结果：

```
D:\Java21\bin\java.exe ...
[main] DEBUG l.mybatis.mapper.CarMapper.selectByMultiCondition - ==> Preparing: select * from t_car where 0 = 0 brand like ?"%" and
guide_price >= ? and car_type = ?
[main] DEBUG l.mybatis.mapper.CarMapper.selectByMultiCondition - ==> Parameters: 宝马(String), 41.0(Double), 燃油车(String)

org.apache.ibatis.exceptions.PersistenceException:
### Error querying database. Cause: java.sql.SQLException: You have an error in your SQL syntax; check the manual that
corresponds to your MySQL server version for the right syntax to use near 'brand like '宝马'%"

        and guide_price >= 41.0
' at line 3
### The error may exist in lzt/mybatis/mapper/CarMapper.xml
### The error may involve defaultParameterMap
### The error occurred while setting parameters
### SQL: select * from t_car where 0 = 0 brand like ?"%" and guide_price >= ?
and car_type = ?
```

where标签

where标签的作用：让where子句更加动态智能。

- 所有条件都为空时，where标签保证不会生成where子句。
- 自动去除某些条件前面多余的and或or。

继续使用if标签中的需求。

- CarMapper.selectByMultiConditionWithWhere

```
/**
 * 根据多条件查询Car，使用where标签
 * @param brand
 * @param guidePrice
 * @param carType
 * @return
 */
List<Car> selectByMultiConditionWithWhere(@Param("brand") String brand, @Param("guidePrice") Double guidePrice,
@Param("carType") String carType);
```

- CarMapper.xml

```
<select id="selectByMultiConditionWithWhere" resultType="car">
  select * from t_car
  <where>
    <if test="brand != null and brand != ''">
      and brand like #{brand}%"
    </if>
    <if test="guidePrice != null and guidePrice != ''">
      and guide_price >= #{guidePrice}
    </if>
    <if test="carType != null and carType != ''">
```

```

        and car_type = #{carType}
    </if>
</where>
</select>

```

- CarMapperTest.testSelectByMultiConditionWithWhere

```

@Test
public void testSelectByMultiConditionWithWhere(){
    CarMapper mapper = SqlSessionUtil.openSession().getMapper(CarMapper.class);
    List<Car> cars = mapper.selectByMultiConditionWithWhere("宝马", 41.0, "燃油车");
    System.out.println(cars);
}

```

运行结果：

```

D:\Java21\bin\java.exe ...
[main] DEBUG l.m.m.CarMapper.selectByMultiConditionWithWhere - ==> Preparing: select * from t_car WHERE brand like ?"%" and
guide_price >= ? and car_type = ?
[main] DEBUG l.m.m.CarMapper.selectByMultiConditionWithWhere - ==> Parameters: 宝马(String), 41.0(Double), 燃油车(String)
[main] DEBUG l.m.m.CarMapper.selectByMultiConditionWithWhere - <== Total: 1
[Car{id=1, carNum='100', brand='宝马520Li', guidePrice=41.0, produceTime='2022-09-01', carType='燃油车'}]

```

如果所有条件都是空呢？

- CarMapperTest.testSelectByMultiConditionWithWhere

```

List<Car> cars = mapper.selectByMultiConditionWithWhere("", null, "");

```

```

D:\Java21\bin\java.exe ...
[main] DEBUG l.m.m.CarMapper.selectByMultiConditionWithWhere - ==> Preparing: select * from t_car
[main] DEBUG l.m.m.CarMapper.selectByMultiConditionWithWhere - ==> Parameters:
[main] DEBUG l.m.m.CarMapper.selectByMultiConditionWithWhere - <== Total: 11
[Car{id=1, carNum='100', brand='宝马520Li', guidePrice=41.0, produceTime='2022-09-01', carType='燃油车'}, Car{id=2, carNum='101',
brand='奔驰E300L', guidePrice=54.0, produceTime='2022-08-01', carType='电车'}, Car{id=3, carNum='102', brand='丰田未来', guidePrice=40
.3, produceTime='2014-10-05', carType='氢能源'}, Car{id=4, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05',
carType='氢能源'}, Car{id=5, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}, Car{id=6,
carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}, Car{id=7, carNum='102', brand='丰田未来',
guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}, Car{id=8, carNum='102', brand='丰田未来', guidePrice=40.3,
produceTime='2014-10-05', carType='氢能源'}, Car{id=9, carNum='102', brand='比亚迪汉', guidePrice=30.23, produceTime='2018-09-10',
carType='电车'}, Car{id=10, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}, Car{id=29,
carNum='5262', brand='BYD汉', guidePrice=30.3, produceTime='2020-10-11', carType='新能源'}]

```

它可以自动去掉前面多余的and，那可以自动去掉前面多余的or吗？

- CarMapperTest.testSelectByMultiConditionWithWhere

```

List<Car> cars = mapper.selectByMultiConditionWithWhere("宝马", 41.0, "燃油车");

```

- CarMapper.xml

```

<select id="selectByMultiConditionWithWhere" resultType="car">
    select * from t_car
    <where>
        <if test="brand != null and brand != ''">
            or brand like #{brand}%"
        </if>
        <if test="guidePrice != null and guidePrice != ''">
            and guide_price >= #{guidePrice}
        </if>
        <if test="carType != null and carType != ''">
            and car_type = #{carType}
        </if>
    </where>
</select>

```

运行结果：

```

D:\Java21\bin\java.exe ...
[main] DEBUG l.m.m.CarMapper.selectByMultiConditionWithWhere - ==> Preparing: select * from t_car WHERE brand like ?%" and
guide_price >= ? and car_type = ?
[main] DEBUG l.m.m.CarMapper.selectByMultiConditionWithWhere - ==> Parameters: 宝马(String), 41.0(Double), 燃油车(String)
[main] DEBUG l.m.m.CarMapper.selectByMultiConditionWithWhere - <==      Total: 1
[Car{id=1, carNum='100', brand='宝马520Li', guidePrice=41.0, produceTime='2022-09-01', carType='燃油车'}]

```

它可以自动去掉前面多余的and，那可以自动去掉后面多余的and吗？

- CarMapper.xml

```

<select id="selectByMultiConditionWithWhere" resultType="car">
    select * from t_car
    <where>
        <if test="brand != null and brand != ''">
            brand like #{brand}%"
        </if>
        <if test="guidePrice != null and guidePrice != ''">
            guide_price >= #{guidePrice} and
        </if>
        <if test="carType != null and carType != ''">
            car_type = #{carType}
        </if>
    </where>
</select>

```

- CarMapperTest.testSelectByMultiConditionWithWhere

```

// 让最后一个条件为空
List<Car> cars = mapper.selectByMultiConditionWithWhere("宝马", 41.0, "");

```

运行结果：

```
D:\Java21\bin\java.exe ...
[main] DEBUG l.m.m.CarMapper.selectByMultiConditionWithWhere - ==> Preparing: select * from t_car WHERE brand like ?"%" and
guide_price >= ? and
[main] DEBUG l.m.m.CarMapper.selectByMultiConditionWithWhere - ==> Parameters: 丰田(String), 20.0(Double)

org.apache.ibatis.exceptions.PersistenceException:
### Error querying database. Cause: java.sql.SQLException: You have an error in your SQL syntax; check the manual that
corresponds to your MySQL server version for the right syntax to use near '' at line 5
### The error may exist in lzt/mybatis/mapper/CarMapper.xml
### The error may involve defaultParameterMap
### The error occurred while setting parameters
### SQL: select * from t_car WHERE brand like ?"%" and guide_price >= ? and
### Cause: java.sql.SQLException: You have an error in your SQL syntax; check the manual that corresponds to your MySQL
server version for the right syntax to use near '' at line 5
```

很显然，后面多余的and是会被去除的。

trim标签

- trim标签的属性：
 - prefix：在trim标签中的语句前添加内容
 - suffix：在trim标签中的语句后添加内容
 - prefixOverrides：前缀覆盖掉（去掉）
 - suffixOverrides：后缀覆盖掉（去掉）
- CarMapper接口

```
/**
 * 根据多条件查询Car，使用trim标签
 * @param brand
 * @param guidePrice
 * @param carType
 * @return
 */
List<Car> selectByMultiConditionWithTrim(@Param("brand") String brand, @Param("guidePrice") Double guidePrice,
@Param("carType") String carType);
```

- CarMapper.xml

```
<select id="selectByMultiConditionWithTrim" resultType="car">
  select * from t_car
  <trim prefix="where" suffixOverrides="and/or">
    <if test="brand != null and brand != ''">
      brand like #{brand}"%" and
    </if>
    <if test="guidePrice != null and guidePrice != ''">
      guide_price >= #{guidePrice} and
    </if>
    <if test="carType != null and carType != ''">
      car_type = #{carType}
    </if>
  </trim>
</select>
```

CarMapperTest.testSelectByMultiConditionWithTrim



@Test

```
public void testSelectByMultiConditionWithTrim(){
    CarMapper mapper = SqlSessionUtil.openSession().getMapper(CarMapper.class);
    List<Car> cars = mapper.selectByMultiConditionWithTrim("宝马", 41.0, "");
    System.out.println(cars);
}
```

D:\Java21\bin\java.exe ...

```
[main] DEBUG l.m.m.CarMapper.selectByMultiConditionWithTrim - ==> Preparing: select * from t_car where brand like ?"%" and
guide_price >= ?
[main] DEBUG l.m.m.CarMapper.selectByMultiConditionWithTrim - ==> Parameters: 宝马(String), 41.0(Double)
[main] DEBUG l.m.m.CarMapper.selectByMultiConditionWithTrim - <== Total: 1
[Car{id=1, carNum='100', brand='宝马520Li', guidePrice=41.0, produceTime='2022-09-01', carType='燃油车'}]
```

如果所有条件为空，where会被加上吗？



```
List<Car> cars = mapper.selectByMultiConditionWithTrim("", null, "");
```

D:\Java21\bin\java.exe ...

```
[main] DEBUG l.m.m.CarMapper.selectByMultiConditionWithTrim - ==> Preparing: select * from t_car
[main] DEBUG l.m.m.CarMapper.selectByMultiConditionWithTrim - ==> Parameters:
[main] DEBUG l.m.m.CarMapper.selectByMultiConditionWithTrim - <== Total: 11
[Car{id=1, carNum='100', brand='宝马520Li', guidePrice=41.0, produceTime='2022-09-01', carType='燃油车'}, Car{id=2, carNum='101',
brand='奔驰E300L', guidePrice=54.0, produceTime='2022-08-01', carType='电车'}, Car{id=3, carNum='102', brand='丰田未来', guidePrice=40
.3, produceTime='2014-10-05', carType='氢能源'}, Car{id=4, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05',
carType='氢能源'}, Car{id=5, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}, Car{id=6,
carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}, Car{id=7, carNum='102', brand='丰田未来',
guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}, Car{id=8, carNum='102', brand='丰田未来', guidePrice=40.3,
produceTime='2014-10-05', carType='氢能源'}, Car{id=9, carNum='102', brand='比亚迪汉', guidePrice=30.23, produceTime='2018-09-10',
carType='电车'}, Car{id=10, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}, Car{id=29,
carNum='5262', brand='BYD汉', guidePrice=30.3, produceTime='2020-10-11', carType='新能源'}]
```

set标签

主要使用在update语句当中，用来生成set关键字，同时去掉最后多余的“，”

比如我们只更新提交的不为空的字段，如果提交的数据是空或者“”，那么这个字段我们将不更新。

- CarMapper接口



```
/**
 * 更新信息，使用set标签
 * @param car
 * @return
 */
int updateWithSet(Car car);
```

- CarMapper.xml



```
<update id="updateWithSet">
  update t_car
  <set>
    <if test="carNum != null and carNum != ''">car_num = #{carNum},</if>
    <if test="brand != null and brand != ''">brand = #{brand},</if>
    <if test="guidePrice != null and guidePrice != ''">guide_price = #{guidePrice},</if>
    <if test="produceTime != null and produceTime != ''">produce_time = #{produceTime},</if>
    <if test="carType != null and carType != ''">car_type = #{carType},</if>
  </set>
  where id = #{id}
</update>
```

- CarMapperTest.testUpdateWithSet



```
@Test
public void testUpdateWithSet(){
    CarMapper mapper = SqlSessionUtil.openSession().getMapper(CarMapper.class);
    Car car = new Car(3L, "102", "丰田未来", 40.30, "", null);
    int count = mapper.updateWithSet(car);
    System.out.println(count);
    SqlSessionUtil.openSession().commit();
}
```

执行结果：

```
D:\Java21\bin\java.exe ...
[main] DEBUG lzt.mybatis.mapper.CarMapper.updateWithSet - ==> Preparing: update t_car SET car_num = ?, brand = ?, guide_price = ?
where id = ?
[main] DEBUG lzt.mybatis.mapper.CarMapper.updateWithSet - ==> Parameters: 102(String), 丰田未来(String), 40.3(Double), 3(Long)
[main] DEBUG lzt.mybatis.mapper.CarMapper.updateWithSet - <== Updates: 1
1
```

choose when otherwise

这三个标签是在一起使用的：



```
<choose>
  <when></when>
  <when></when>
  <when></when>
  <otherwise></otherwise>
</choose>
```

等同于：

```

if(){

}else if(){

}else if(){

}else if(){

}else{

}

```

只有一个分支会被选择！！！！

需求：先根据品牌查询，如果没有提供品牌，再根据指导价格查询，如果没有提供指导价格，就根据生产日期查询。

- CarMapper接口

```

/**
 * 使用choose when otherwise 标签查询
 * @param brand
 * @param guidePrice
 * @param produceTime
 * @return
 */
List<Car> selectWithChoose(@Param("brand") String brand, @Param("guidePrice") Double guidePrice, @Param("produceTime") String produceTime);

```

- CarMapper.xml

```

<select id="selectWithChoose" resultType="car">
    select * from t_car
    <where>
        <choose>
            <when test="brand != null and brand != ''">
                brand like #{brand}%"
            </when>
            <when test="guidePrice != null and guidePrice != ''">
                guide_price >= #{guidePrice}
            </when>
            <otherwise>
                produce_time >= #{produceTime}
            </otherwise>
        </choose>
    </where>
</select>

```

- CarMapperTest.testSelectWithChoose

```

@Test
public void testSelectWithChoose(){
    CarMapper mapper = SqlSessionUtil.openSession().getMapper(CarMapper.class);
    //List<Car> cars = mapper.selectWithChoose("宝马520Li", 41.0, "2022-09-01");
    //List<Car> cars = mapper.selectWithChoose("", 41.0, "2022-09-01");
    //List<Car> cars = mapper.selectWithChoose("", null, "2022-09-01");
    List<Car> cars = mapper.selectWithChoose("", null, "");
    System.out.println(cars);
}

```



```
D:\Java21\bin\java.exe ...
```

```
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectWithChoose - ==> Preparing: select * from t_car WHERE produce_time >= ?
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectWithChoose - ==> Parameters: (String)
[main] DEBUG lzt.mybatis.mapper.CarMapper.selectWithChoose - <== Total: 11
[Car{id=1, carNum='100', brand='宝马520Li', guidePrice=41.0, produceTime='2022-09-01', carType='燃油车'}, Car{id=2, carNum='101',
brand='奔驰E300L', guidePrice=54.0, produceTime='2022-08-01', carType='电车'}, Car{id=3, carNum='102', brand='丰田未来', guidePrice=40
.3, produceTime='2014-10-05', carType='氢能源'}, Car{id=4, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05',
carType='氢能源'}, Car{id=5, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}, Car{id=6,
carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}, Car{id=7, carNum='102', brand='丰田未来',
guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}, Car{id=8, carNum='102', brand='丰田未来', guidePrice=40.3,
produceTime='2014-10-05', carType='氢能源'}, Car{id=9, carNum='102', brand='比亚迪汉', guidePrice=30.23, produceTime='2018-09-10',
carType='电车'}, Car{id=10, carNum='102', brand='丰田未来', guidePrice=40.3, produceTime='2014-10-05', carType='氢能源'}, Car{id=29,
carNum='5262', brand='BYD汉', guidePrice=30.3, produceTime='2020-10-11', carType='新能源'}]
```

foreach标签

循环数组或集合，动态生成sql，比如这样的SQL：

- 批量删除

```
delete from t_car where id in(1,2,3);
delete from t_car where id = 1 or id = 2 or id = 3;
```

- 批量添加

```
insert into t_car values
(null, '1001', '凯美瑞', 35.0, '2010-10-11', '燃油车'),
(null, '1002', '比亚迪唐', 31.0, '2020-11-11', '新能源'),
(null, '1003', '比亚迪宋', 32.0, '2020-10-11', '新能源')
```

批量删除

- 用in来删除
 - CarMapper接口

```
/**
 * 通过foreach完成批量删除
 * @param ids
 * @return
 */
int deleteBatchByForeach(@Param("ids") Long[] ids);
```

- CarMapper.xml



```
<!--
collection: 集合或数组
item: 集合或数组中的元素
separator: 分隔符
open: foreach标签中所有内容的开始
close: foreach标签中所有内容的结束
-->
<delete id="deleteBatchByForeach">
    delete from t_car where id in
    <foreach collection="ids" item="id" separator="," open="(" close=")">
        #{id}
    </foreach>
</delete>
```

- CarMapperTest.testDeleteBatchByForeach



```
@Test
public void testDeleteBatchByForeach(){
    CarMapper mapper = SqlSessionUtil.openSession().getMapper(CarMapper.class);
    int count = mapper.deleteBatchByForeach(new Long[]{9L, 10L, 29L});
    System.out.println("删除了几条记录: " + count);
    SqlSessionUtil.openSession().commit();
}
```

执行结果：

✓ 测试 已通过: 1共 1 个测试 - 694毫秒

```
D:\Java21\bin\java.exe ...
[main] DEBUG lzt.mybatis.mapper.CarMapper.deleteBatchByForeach - ==> Preparing: delete from t_car where id in ( ? , ? , ? )
[main] DEBUG lzt.mybatis.mapper.CarMapper.deleteBatchByForeach - ==> Parameters: 9(Long), 10(Long), 29(Long)
[main] DEBUG lzt.mybatis.mapper.CarMapper.deleteBatchByForeach - <== Updates: 3
删除了几条记录: 3
```

- 用or来删除
- CarMapper接口



```
/**
 * 通过foreach完成批量删除
 * @param ids
 * @return
 */
int deleteBatchByForeach2(@Param("ids") Long[] ids);
```

- CarMapper.xml



```
<delete id="deleteBatchByForeach2">
    delete from t_car where
    <foreach collection="ids" item="id" separator="or">
        id = #{id}
    </foreach>
</delete>
```

- CarMapperTest.testDeleteBatchByForeach2



```
@Test
public void testDeleteBatchByForeach2(){
    CarMapper mapper = SqlSessionUtil.openSession().getMapper(CarMapper.class);
    int count = mapper.deleteBatchByForeach2(new Long[]{6L, 7L, 8L});
    System.out.println("删除了几条记录: " + count);
    SqlSessionUtil.openSession().commit();
}
```

运行结果：

```
D:\Java21\bin\java.exe ...
[main] DEBUG 1.mybatis.mapper.CarMapper.deleteBatchByForeach2 - ==> Preparing: delete from t_car where id = ? or id = ? or id = ?
[main] DEBUG 1.mybatis.mapper.CarMapper.deleteBatchByForeach2 - ==> Parameters: 6(Long), 7(Long), 8(Long)
[main] DEBUG 1.mybatis.mapper.CarMapper.deleteBatchByForeach2 - <== Updates: 3
删除了几条记录: 3
```

批量添加

- CarMapper接口



```
/**
 * 批量添加，使用foreach标签
 * @param cars
 * @return
 */
int insertBatchByForeach(@Param("cars") List<Car> cars);
```

- CarMapper.xml



```
<insert id="insertBatchByForeach">
    insert into t_car values
    <foreach collection="cars" item="car" separator=",">
        (null,#{car.carNum},#{car.brand},#{car.guidePrice},#{car.produceTime},#{car.carType})
    </foreach>
</insert>
```

- CarMapperTest.testInsertBatchByForeach



```
import java.util.Arrays;

@Test
public void testInsertBatchByForeach(){
    CarMapper mapper = SqlSessionUtil.openSession().getMapper(CarMapper.class);
    Car car1 = new Car(null, "2001", "兰博基尼", 100.0, "1998-10-11", "燃油车");
    Car car2 = new Car(null, "2001", "兰博基尼", 100.0, "1998-10-11", "燃油车");
    Car car3 = new Car(null, "2001", "兰博基尼", 100.0, "1998-10-11", "燃油车");
    List<Car> cars = Arrays.asList(car1, car2, car3);
    int count = mapper.insertBatchByForeach(cars);
    System.out.println("插入了几条记录" + count);
    SqlSessionUtil.openSession().commit();
}
```

运行结果：

```
D:\Java21\bin\java.exe ...
[main] DEBUG lzt.mybatis.mapper.CarMapper.insertBatchByForeach - ==> Preparing: insert into t_car values (null,?,?,?,?,?) ,
(null,?,?,?,?,?) , (null,?,?,?,?,?)
[main] DEBUG lzt.mybatis.mapper.CarMapper.insertBatchByForeach - ==> Parameters: 2001(String), 兰博基尼(String), 100.0(Double),
1998-10-11(String), 燃油车(String), 2001(String), 兰博基尼(String), 100.0(Double), 1998-10-11(String), 燃油车(String), 2001(String), 兰
博基尼(String), 100.0(Double), 1998-10-11(String), 燃油车(String)
[main] DEBUG lzt.mybatis.mapper.CarMapper.insertBatchByForeach - <== Updates: 3
插入了几条记录3
```

sql标签与include标签

sql标签用来声明sql片段

include标签用来将声明的sql片段包含到某个sql语句当中

作用：代码复用。易维护。

- my_mybatis-08-select模块中的CarMapper.xml

```
<sql id="carCols">id,car_num carNum,brand,guide_price guidePrice,produce_time produceTime,car_type carType</sql>

<select id="selectALLRetMap" resultType="map">
    select <include refid="carCols"/> from t_car
</select>

<select id="selectALLRetListMap" resultType="map">
    select <include refid="carCols"/> carType from t_car
</select>

<select id="selectByIdRetMap" resultType="map">
    select <include refid="carCols"/> from t_car where id = #{id}
</select>
```